

分类号	
学校代码	10700
学 号	2221920082

西安理工大学

硕士学位论文

(学术学位)

面向自然语言交互的电力巡检机器人任务规划

来钰钧

学 科 门 类: 工 学

学 科 名 称: 电气工程

所 在 学 院: 电气工程学院

指 导 教 师: 马文涛 栾智荣

申 请 日 期: 2025 年 6 月

论文题目：面向自然语言交互的电力巡检机器人任务规划

专业名称：电气工程

研 究 生：来钰钧

签 名：_____

指导教师：马文涛 副教授

签 名：_____

栾智荣 讲师

签 名：_____

摘 要

随着电力系统迅速发展，电气设备数量与种类持续增加，传统人工巡检的方式已不能满足现代电力系统高效安全的需求。电力系统包含的设备种类多样、分布广泛，并且每种设备的维护要求和故障表现都不同，这导致巡检工作的难度日益增大，为了解决这一难题在电力系统巡检任务中引入机器人技术能够提高作业效率。电力巡检任务属于开放场景中的开放式任务，通常巡检环境存在高度不确定性。当前机器人大都依赖预设的功能，并且无法与人进行自然语言交互。机器人需要面对复杂的巡检任务，同时还要应对动态环境以及任务变化引起的突发情况，这就要求巡检机器人具有较高的任务理解能力与任务规划能力。大型语言模型（Large Language Model , LLM）能够为巡检机器人赋能，但是该技术尚处于起步阶段，其技术难点主要体现在如何形成电力巡检任务的从自然语言到任务规划专业表述的内容生成？如何让机器人具备灵活多变的功能？如何结合实际环境对机器人行为做出规划？本文研究了面向自然语言交互的电力巡检机器人任务规划方法，以提升巡检机器人的任务理解，个性化的能力生成和任务规划等能力，以提高电力巡检任务的机器人执行效果。本文的主要研究内容包括：

（1）针对巡检机器人任务理解困难的问题，本文提出了一种基于自然语言处理的任务语义解析框架，通过构建一个电力巡检任务的向量知识库来解决 LLM 难以获得特定领域知识的问题，通过补充种子任务文本的形式能够有效减少由于上下文语义缺失导致的 LLM 幻觉现象，该方法能够提升 LLM 生成巡检任务规划的准确性。针对电力系统巡检任务中复杂任务难以与底层原子任务对齐的问题，本文提出了一种多智能体动态任务分解框架，该架构能够将复杂的多步骤任务分解为多个粗粒度的子任务，通过多个子代理（Agent）将粗粒度的任务分解为细粒度任务，这样做可以使任务不断地逼近机器人可以执行的原子任务，最终将任务分解为与底层功能对齐的数据形式。

（2）针对巡检机器人中未包含的个性化功能需求，本文提出了一种多智能体自协作框架，此框架依靠多个 Agent 协同制定机器人自动开发策略，生成个性化功能的开发程序。为保证多个 Agent 间有效协同，本文设计了协作规则模拟现实开发团队的协作模式，这种多

Agent 自协作框架的创新点在于，没有专业知识以及代码开发经验的个人，仅借助与该框架的自然语言交互就能够实现个性化的巡检功能开发。针对个性化功能与机器人动作行为难以对齐的问题，本文提出了一种基于余弦相似度的原子功能对齐方法，借助余弦相似度关联原子功能与机器人运动控制指令的语义相似性，达成了任务与指令的对齐。由于现有的评估基准缺乏准确反映 Agent 任务分解所需细粒度层级，本文提出一种基于语义度量的细粒度评价体系，该评价体系综合考量任务策略与机器人指令间信息传播准确性和任务分解有效性，能有效解决任务到指令过程中产生的语义偏差和传播错误。

(3) 针对电力系统巡检任务规划过程中难以获取环境信息的问题，本文提出了基于多模型交互的环境感知方法，将 LLM 与视觉语言模型 (Visual Language Model, VLM) 结合，其中 VLM 用于实时环境信息的感知，并将环境信息转化为自然语言与 LLM 进行交互，以生成面向现实环境的任务规划。同时为了实现更精确的环境感知，本文设计了一种用于开放环境感知的热值图导航算法，将摄像头采集到的视觉信息映射到二维可计算空间，并将 VLM 的结果转化为热值图用于机器人导航，热值图能够反映出物体在真实世界中的相对位置信息，通过贪心算法计算出最优路径，在热值图内生成密集的点状轨迹来指导机器人运动，这种映射关系允许机器人根据环境变化实时调整运动轨迹。此外，针对 LLM 生成结果出现的局部最优问题，本文提出了一种自回归循环语义对齐方法，旨在将语义对齐过程中出现语义偏差的任务文本进行修正，通过将错误任务和提示词合并构建一个反馈信息，使 LLM 在生成新一轮结果时在语义层面上偏向提示词所述的正确结果，能够有效的提高 LLM 输出结果与机器人控制指令之间的语义相似性。

关键词：大型语言模型；任务分解；任务规划；机器人；语义对齐；多代理协作

*本研究得到国家自然科学基金联合基金重点支持项目-区域创新发展联合基金(U21A20485)资助。

**Title: TASK PLANNING OF POWER INSPECTION ROBOTS BASED ON
NATURAL LANGUAGE INTERACTION****Major: Electrical Engineering****Name: Yujun LAI****Signature:_____****Supervisor: Associate prof. Wentao MA****Signature:_____****Lecturer. Zhirong LUAN****Signature:_____****Abstract**

With the rapid development of the power system, the number and types of electrical equipment continues to increase, the traditional manual inspection method can not meet the needs of modern power system efficiency and safety. The power system contains a variety of equipment, widely distributed, and the maintenance requirements and fault performance of each device are different, which leads to the increasing difficulty of the inspection work, in order to solve this problem in the power system inspection task to introduce robotics technology can improve operational efficiency. Power inspection tasks belong to the open scene in the open task, usually inspection environment has a high degree of uncertainty. Most of the current robots rely on preset functions and cannot interact with people in natural language. Robots need to face complex inspection tasks, but also to cope with the dynamic environment and unexpected situations caused by task changes, which requires inspection robots to have a high level of task comprehension and task planning capabilities. Large-scale language model (LLM) can empower inspection robots, but the technology is still in its infancy, and its technical difficulties are mainly reflected in how to form the power inspection tasks from natural language to the task planning professional expression of content generation? How to make the robot with flexible and changeable functions? How to plan the robot's behavior in the context of the actual environment? In this paper, we study the task planning method for power inspection robots oriented to natural language interaction, in order to improve the task understanding, personalized capability generation and task planning of inspection robots, so as to improve the robot execution effect of power inspection tasks. The main research content of this paper includes:

- (1) Aiming at the problem of inspection robot task understanding difficulty, this paper

proposes a task semantic parsing framework based on natural language processing, which solves the problem of LLM's difficulty in obtaining domain-specific knowledge by constructing a vector knowledge base of electric power inspection tasks, and effectively reduces the phenomenon of LLM's hallucination due to the lack of contextual semantics by supplementing the form of seed task text, and the method can enhance the ability of the accuracy of LLM in generating inspection task planning. Aiming at the problem that complex tasks in power system inspection tasks are difficult to be aligned with the underlying atomic tasks, this paper proposes a multi-intelligent body dynamic task decomposition framework, which can decompose complex multi-step tasks into multiple coarse-grained sub-tasks, and decompose coarse-grained tasks into fine-grained tasks through multiple subagents (Agents), which will enable the tasks to continuously approach the atomic tasks that can be executed by the robots, and finally decompose the tasks into tasks that are compatible with the atomic tasks that can be executed by the robots, and finally decompose the tasks into tasks that are compatible with the atomic tasks that can be executed by the robots. tasks, and finally decompose the tasks into data forms aligned with the underlying functions.

(2) For the needs of personalized functions not included in the inspection robot, this paper proposes a multi-intelligence body self-collaboration framework, which relies on multiple Agents to collaboratively formulate the robot automatic development strategy and generate the development program for personalized functions. In order to ensure effective collaboration among multiple Agents, this paper designs collaboration rules to simulate the collaboration mode of a real development team. The innovation of this multi-agent self-collaboration framework is that individuals with no professional knowledge and code development experience can achieve personalized inspection function development only through natural language interaction with the framework. To address the difficulty of aligning personalized functions with robot behaviors, this paper proposes a cosine similarity-based atomic function alignment method, which achieves task-command alignment by correlating the semantic similarity between atomic functions and robot motion control instructions with cosine similarity. Since the existing evaluation benchmarks lack the fine-grained level required to accurately reflect Agent task decomposition, this paper proposes a fine-grained evaluation system based on semantic metrics, which comprehensively considers the accuracy of information dissemination between the task strategy and robot instructions and the effectiveness of task decomposition, and can effectively solve the semantic bias and dissemination errors generated in the process of task-to-instruction.

(3) Aiming at the problem of difficulty in obtaining environmental information during the planning process of power system inspection tasks, this paper proposes an environment

perception method based on multi-model interaction, which combines the LLM with the visual language model (VLM), where the VLM is used for real-time environment information perception and transforms the environment information into a natural language for interacting with the LLM in order to generate task planning oriented to the real environment. Meanwhile, in order to realize more accurate environment perception, this paper designs a heat map navigation algorithm for open environment perception, which maps the visual information captured by the camera into a two-dimensional computable space and transforms the results of the VLM into a heat map for robot navigation, which reflects the relative position information of the object in the real world, and computes the optimal path through a greedy algorithm to generate the The optimal path is calculated by a greedy algorithm, and dense point-like trajectories are generated within the heat value map to guide the robot's movement, and this mapping relationship allows the robot to adjust its trajectory in real time according to changes in the environment. In addition, for the local optimization problem of LLM generated results, this paper proposes an autoregressive cyclic semantic alignment method, which aims to correct the task text with semantic deviation during the semantic alignment process, and construct a feedback message by merging the wrong task and the cue word, so that the LLM will bias towards the correct result described by the cue word at the semantic level when generating a new round of results, which can effectively improve semantic similarity between LLM output results and robot control commands.

Key words: Large Language Models; Task Decomposition; Mission planning; Robot; Semantic Alignment; Multi-Agent Collaboration

目 录

1 绪论.....	1
1.1 课题的研究背景及意义.....	1
1.2 国内外研究现状.....	2
1.2.1 电力巡检任务理解与规划.....	2
1.2.2 机器人功能自动开发.....	3
1.2.3 多机器人电力巡检任务规划.....	5
1.3 相关理论基础.....	6
1.3.1 自然语言处理.....	6
1.3.2 大型语言模型.....	7
1.3.3 LangChain 架构.....	9
1.4 本文主要研究内容.....	10
2 电力巡检任务的自然语言理解与分解	15
2.1 面向电力巡检任务理解的云边端架构设计	15
2.1.1 自然语言理解问题.....	15
2.1.2 云边端架构的任务理解框架设计.....	16
2.1.3 基于自然语言处理的任务语义解析框架	18
2.1.4 电力系统知识库 LLM 对比测试	20
2.2 基于大模型代理的电力巡检任务分解框架	22
2.2.1 复杂任务分解问题.....	22
2.2.2 任务分解机理与技能库的构建.....	22
2.2.3 多智能体动态任务分解框架构建.....	25
2.2.4 任务分解对比实验与结果分析.....	27
2.3 本章小结.....	29
3 巡检机器人功能自动开发与行为对齐	31
3.1 基于多智能体自协作框架的巡检机器人自动开发	31
3.1.1 个性化功能自动开发问题.....	31
3.1.2 多智能体自协作框架设计与实现.....	32
3.1.3 机器人功能自动开发实验.....	34
3.2 巡检机器人原子任务对齐与行为执行	36
3.2.1 任务与动作对齐问题.....	36
3.2.2 基于余弦相似度的原子任务对齐方法	37
3.2.3 基于语义度量的细粒度评价体系.....	38
3.2.4 语义度量基线实验和消融实验.....	43
3.3 本章小结.....	48

4 基于自然语言交互的机器人任务规划	51
4.1 多层模型嵌套架构的环境感知交互	51
4.1.1 多模型环境交互问题.....	51
4.1.2 多模型交互与环境感知方法.....	52
4.1.3 开放环境感知的热值图导航算法.....	55
4.1.4 机器人导航规划实验.....	57
4.2 巡检机器人自回归循环语义对齐	62
4.2.1 大模型结果局部最优问题.....	62
4.2.2 自回归循环语义对齐方法.....	63
4.2.3 基于自回归循环语义对齐方法的反馈实验	66
4.3 本章小结.....	68
5 结论与展望.....	69
5.1 本文的主要研究成果.....	69
5.2 下一步的研究方向.....	70
参考文献.....	72

1 绪论

1.1 课题的研究背景及意义

随着社会的不断发展导致用电需求也在持续的增长,电力系统是电力能源供应的支柱,保证电力系统的稳定运行成为社会运行关键步骤^[1]。传统电力系统巡检工作主要依赖人工操作,巡检人员需要根据巡检任务的内容在复杂工况和复杂环境中对电力设备进行数据采集和检查维护^[2]。随着社会用电量的不断增大,电力系统的规模也在不断扩大,为了满足电力系统的稳定运行,电力系统中的设备种类和数量也在急剧增加。在上述背景下传统人工巡检方式效率低并且存在安全问题^[3],传统的人工巡检方式在地形复杂的危险区域或者恶劣天气情况下作业人员的安全难以得到保证,基于上述情况传统的人工巡检方式已经难以满足现代电力系统安全高效的需求。

电力系统巡检任务之所以被视为一项复杂任务原因在于其具有不确定性,电力巡检任务的主要内容是对变电站以及输电线路等多种类型设备进行状态监测,这些电气设备种类繁多并且各项参数与检测方法也各不相同。由于多变的环境因素会随时对执行策略和结果产生干扰所以在开放环境中进行巡检往往难以预设任务路径。复杂且多变的环境容易导致漏检或误判现象的发生,可能造成供电中断甚至安全事故的发生^[4]。当某个电气设备出现故障时需要立即调整后续的巡检路径规划,这种动态调整机制必须依赖于电力系统的高水平感知能力和对任务语义的理解能力。

机器人在进行电力巡检任务时会面临一些困难,第一个问题就是巡检机器人如何正确理解真实环境中的巡检任务,虽然 LLM 具有优秀的语义理解能力与推理能力,但是 LLM 想要正确理解真实环境的电力系统巡检任务仍然是困难的。电力巡检任务涉及了大量的专业术语和电气设备功能描述,现有的 LLM 大多是通用的基座模型,缺乏专业的电力系统领域的知识和真实环境中的作业准则,所以不能完全理解电力巡检任务的深层语义信息。电力巡检任务不仅要处理静态的已知信息,还要同步处理任务规划与实时环境感知这类动态信息,如何使 LLM 能够获取实时变化的动态信息并且生成有效的任务规划是当前面临的一个瓶颈问题。

近年来电力系统中的电气设备数量和种类不断地增加,这就导致巡检任务也变得愈发复杂,越来越多的个性化需求已成为电力巡检任务的一个难题。在面对个性化需求时,基于预设程序与固定操作流程的巡检机器人其适配能力往往不足,难以应对多样化巡检任务的要求。不同型号的传感设备或作业工具,需由机器人根据具体任务信息进行携带进而执行精细化操作。复杂路径规划问题与电气设备维护需求频繁出现在电力巡检过程中,这类需求随着电气设备规模的扩大也引起了人们的关注^[5]。现有技术条件下,机器人个性化功能开发仍然存在明显的局限性。

在电力巡检任务中为了提升作业效率或者完成复杂度高的任务,往往会使用多台机器人协同工作,但现有技术条件下多机器人协同工作仍面临着极大的挑战^[6]。电力系统巡检任务的环境大多都是动态的开放场景,机器人在执行任务过程中遭遇环境变化的可能性较高^[7],此时需要为机器人提供灵活的任务规划与分配策略。单机器人独立系统完成特定任务是当前主流技术路径,主服务器负责向集群中每个单体机器人发送任务请求。动态环境变量角色被赋予其他机器人,仅在实际需求产生时做出响应。服务器端生成的任务策略必须具备极高精确度并且满足细粒度的要求,大型语言模型需要据此产生高效合理的多机协同方案。每个独立机器人的准确执行依赖于方案下发的精确程度。

通过上文得知机器人技术对电力系统巡检任务是至关重要的,在本文中为了实现电力系统巡检任务的任务理解与任务规划,需要将机器人技术和自然语言处理技术进行结合,使机器人能够应对不同形式的电力巡检任务^[8]。多层模型嵌套与环境感知技术的引入使得外部环境信息的实时感知成为可能,大型语言模型所具有的逻辑推理能力则被用于环境变化的分析及实时任务规划的生成,这种方法能够有效提升机器人在开放环境中的动态适应能力。探讨如何将机器人的行为动作与大型语言模型生成的运动控制指令对齐也是本文的重要研究内容,该方法的运用实现了任务指令与机器人运动模块间的精确匹配,实例证明了这种方法对于作业流程的顺利执行具有显著的促进作用,上述内容是本文研究的重点问题。

1.2 国内外研究现状

1.2.1 电力巡检任务理解与规划

复杂任务的理解与分解是一个重要的研究方向,随着电力系统中的设备与巡检场景的不断变化,电力系统巡检任务的复杂程度也在不断上升。以往电力巡检任务依赖于人工巡检,近年来随着电气设备种类的增加以及巡检场景的扩展,电力巡检任务对与巡检任务的效率和安全提出了更高的要求。传统电力巡检中的人工巡检方式已经难以应对这些需求,为了应对这些需求,国内外研究人员为了提升机器人的智能水平,开始将自然语言处理技术(Natural Language Processing, NLP)和大型语言模型与机器人技术进行结合。首先自然语言处理技术能够对巡检任务进行理解,使机器人能够根据任务约束执行一些简单的电力巡检任务重的行为动作。更加关键的是 LLM 技术的引用,LLM 具有强大的语义理解能力和自然语言推理能力,能够在为机器人行为策略做出指导。LLM 接收到一个电力巡检任务时,能够将任务进行分解,生成复杂度更低的任务便于机器人执行,这一步骤提升了任务执行的成功率,并且由于 LLM 所带来的强大推理能力,能够根据现场环境进行灵活的策略调整,这些技术的结合能够使巡检机器人理解一些复杂的电力巡检任务,不仅能够提升任务规划的精度,还能增加电力巡检任务中机器人面对多变环境的自适应能力。

由于 NLP 领域的快速发展,国内外研究者在任务理解方面进行了大量研究^[9]。研究表明语言能够对机器人行为进行约束并提供行为规范,这些语言规范可以用于推理人类

与机器人行为的中间过程^[10]。文献[11]描述了使用自然语言与机器人进行交互的方法,包括理解自然语言请求和利用语言推动机器人对外界的学习。文献[12]提出了一种两阶段的模型训练过程,并通过与环境的互动来增强模型训练的表现。先前的工作大多都使用词汇分析法和形式逻辑法这些经典方法提取任务中的有效指令,与之不同的是文献[13]设计了一个移动机器人对话代理,能够通过语义分析理解人类指令,由于早期理论不够成熟,所以该方案主要基于预设规则实现任务理解。现有研究的重点已经从在线控制转向了离线控制机器人运动,如何使机器人更精准的理解自然语言指令,并通过局部算术增强来执行局部端到端行为模式^[14]。接着大型语言模型的出现为任务理解提供了另一种思路^[15],文献[16]训练了 GPT-3,并展示了大型语言模型在零样本识别中的巨大改进。大量的研究工作围绕着通过自然语言注释来构建机器人数据集。文献[17]认为,语言的组合性质对学习不同的子技能并系统性地对新技能的泛化至关重要。通过语言指令与机器人进行行为交互的预测器控制模型与我们的思想更为接近。

在任务规划方面目前已经有了许多重要的研究工作,任务规划问题的核心是如何将一个自然语言形式的任务能够转化为机器可直接执行的任务序列。文献[18]探索了将预训练的语言模型生成的结果合并到代理生成决策的过程中,他们通过对预训练的语言模型进行初始化和微调操作,能够生成零样本的可执行策略,这种零样本的泛化能力可以为多种场景下的任务生成规划策略。文献[19]提出了一种名为“SayCan”的方法,该方法能够接收自然语言指令并引导机器人在真实环境中执行任务,通过观察环境对每个可行的动作进行评分,并结合提示推断下一步动作。前文中提到过一种主流的提示策略,可以通过提示工程提示生成特定目标的逐步计划。在这个方法的基础上,文献[20]扩展并提出了一种生成自适应环境行动计划的方法,该方法将环境中的物体及其相互关系作为生成行动计划的附加输入,使得代理在执行任务时能够更好地适应和理解所处的环境。这些文献表明了 LLM 可以为代理提供合理的计划与步骤,这是任务规划的核心功能。然而上述研究大多采用开环控制,这就意味着在 LLM 生成策略的过程中无法进行中间环节的干涉,不能形成人在回路的系统结构从而无法正确的识别出错误以及不能及时纠正错误。文献[21]使用闭环反馈来纠正代理执行中出现的错误,系统通过整合日志信息并且重新传输错误信息提示 LLM 对错误信息进行分析并生成修正后的结果。上述这些成果为我们的研究提供了重要的参考和启示。

现有研究在任务理解与规划方面虽然取得了一定的进展,但仍然难以解决复杂的任务理解与规划问题。如何使 LLM 能够准确的理解特定领域的专业问题以及如何将开放环境的动态信息融入到 LLM 输出的策略中仍然是难以解决的困难^[22]。在任务规划阶段中如何将动态信息合理的进行任务规划并且实现复杂任务的高效分解仍然需要进行研究^[23]。与此同时对于多机器人协作场景,现有研究主要集中在协同策略与任务分配这两方面^[24],而对复杂任务的动态调整和反馈优化仍然缺乏深入的研究^[25]。

1.2.2 机器人功能自动开发

随着电力巡检任务种类越发的多样化,传统的机器人大多都是基于预设任务的只能完成固定功能的机器人,这些机器人在面对不同场景的个性化需求时难以成功执行任务^[26]。为了成功执行电力巡检任务,机器人需要配备特定功能的传感器和工具,还要能根据任务的具体需求进行作业调整^[27]。传统的巡检机器人大多是针对特定任务或环境设计的,缺乏足够的适应性和扩展性,导致巡检机器人无法高效的应对各种动态变化的巡检需求^[28]。如何使巡检机器人应对不同巡检任务中的个性化需求,成为了电力巡检领域亟待解决的关键问题^[29]。在上述背景下基于 LLM 的 AI Agents 技术应运而生^[30],这些新技术的出现为机器人个性化功能开发提供了全新的思路和方法, AI Agents 能够通过提示工程^[31],将机器人的个性化需求以自然语言的形式进行理解与分析,通过来自基座 LLM 的语言理解能力与推理能力,结合机器人个性化需求,动态的生成巡检机器人所需要的功能模块相关代码,帮助巡检机器人实现个性化功能的快速开发^[32]。

AI Agents 在多个研究领域引起了广泛关注,以下研究展示了 LLM 的能力以及在不同情境中的应用^[33]。文献[34]提出了一种思维链提示方法,通过相互连接的提示词构成思维链的方法促进 LLM 的推理,该方法加强了 LLM 的推理能力,这种逐步思考的方式能够处理更加深层次的问题,对于多步骤任务该方法在任务执行成功率这一指标中取得了显著的提升。文献[35]探索了 LLM 具备零样本推理潜力,研究表明预训练模型在没有特定专业训练数据的前提下依然具有推理能力,这种不需要大量数据所产生的推理能力能够在开放环境中解决一些不能预设的问题。文献[36]训练了一种能够与人类进行交互的预训练模型,通过人类的反馈信息成功捕捉语义并进行分析,通过对指令进行反馈跟随的方式展示了 LLM 在学习互动中具有良好的适应能力。随着预训练模型的发展,指令微调技术的相关研究工作引起了广泛关注,文献[37]研究了一种通过指令增强微调模型的方法,该方法能够提高 LLM 在理解和执行指令方面的有效性。为了实现 LLM 开发个性化需求,越来越多的研究开始关注使用 LLM 生成可执行的代码,文献[38]深入探讨了使用 LLM 进行程序生成的方法,研究旨在探索如何使用自然语言生成高质量的可执行代码。文献[39]提出了一种将 LLM 与代码生成相结合的新型架构,该架构能够通过与 LLM 进行语言交互生成高质量的代码。文献[40]开发了一个能够通过用户反馈与交流生成可执行代码的 LLM,该模型可以通过多轮对话补充缺失的上下文信息,获取完整的代码需求从而生成更加符合使用者预期的代码。此外文献[41]研究了一种能够自我调试代码的 LLM 框架,该框架能够通过 LLM 的自我调试修改代码中可能出现的错误,这种方式可以生成准确率更高的代码。

此外,相关研究还探索了利用 LLM 进行软件开发的方法。文献[42]提出了一种能够从 LLM 中提取结构化知识的方法,通过提示工程的方式能够从模型中提取特定领域的深层知识并且获得层次化结构。文献[43]通过一套全新的提示词与多 Agent 结构实现了一种泛化的自动编程模式,通过这种方式增强了 LLM 在生成代码时的表达能力,同时该研究还考察评估了现有 LLM 对未知知识理解的策略是否合理。文献[44]提出了一种能够判断

LLM 所掌握知识的方法,通过这种方法能够判断 LLM 生成的结果是否具有幻觉现象从而判断出生成内容的可靠性。

1.2.3 多机器人电力巡检任务规划

随着电力巡检场景逐渐变得复杂以及任务需求越发多样化,单一机器人难以独立完成复杂度高的任务,原因在于单个机器人的功能不能全面覆盖巡检需求并且难以对多个任务进行并行处理^[45]。多机器人系统能够有效的进行任务分配,在面对复杂任务时可以通过协同作业的方式使巡检工作更加安全可靠^[46]。因此在电力巡检任务中引入多机器人系统是保障任务顺利执行的关键要素。传统的多机器人协同作业模式通常依赖于预设规则和固定流程,这种作业模式难以应对动态变化的巡检场景^[47]。与此同时,传统的人机交互方式大都为人工控制机器人执行巡检动作,这种方式效率较低并且容易由于操作失误而导致任务失败^[48]。因此如何实现多机器人的任务规划与自然语言交互,使多机器人能够通过对话形式自动分配任务并协同配合执行复杂任务,成为了近年来研究的热点。

执行电力系统巡检任务的多机器人通常由多台不同型号且功能不同的机器人构成,这些设备通过协同合作的方式能够相互弥补机器人个体功能的不足,并且这种相互配合的模式能够提升执行任务的效率。目前对于多机器人任务规划的研究主要集中在多机器人任务分配与路径规划等方面。多机器人任务分配的相关研究点在于将任务需求合理的进行分解并且根据机器人的现有功能将拆解后的任务分配到对应的机器人,并且通过协调机器人的任务列表,使任务顺利完成^[49]。传统人工控制机器人运动的方式难以适应越来越密集的巡检任务需求,通过自然语言的形式与机器人进行交互是一种更加直观便捷的方式,操作人员可以直接对机器人进行任务描述,机器人能够自主的执行操作人员的指令。同时机器人能够通过配备的传感器实时的向操作人员反馈日志,操作人员能够同时监视多台设备的工作状态,通过反馈信息可以及时发现错误,这种方式简化了繁杂的人机交互过程^[50]。自然语言交互方式的优点在于电力巡检任务中操作人员可以通过对话的形式与多台设备进行事实交互,并且能够处理多台设备的反馈信息,而不需要通过手动的方式人为进行设备的硬编码设置^[51]。

随着 LLM 的不断研究,研究者们开始使用大量的机器人数据训练垂直领域的语言模型,以解决物理世界中的实际问题。大量研究的关注点在于模型的自然语言理解能力与交互能力,文献[52]展示了预训练模型具备通用的知识,预训练模型作为通用知识的载体需要结合本地场景信息来生成专业知识,许多研究集中在将环境信息与 LLM 结合,使得模型能够与环境进行互动,文献[53]展示了大型语言模型能够从根据需求文档生成相关代码,并提出了一种层次化代码生成方法能够生成复杂度更高的代码。文献[54]通过大型语言模型的通用常识构建动作序列来实现机器人行为,LLM 在获取环境信息后根据模型中储存的通用常识能够理解这些环境信息并且生成后续的任务规划策略,但机器人仍然缺乏执行这些策略的能力,因为调用机器人运动模块需要大型语言模型调用机器人运动控制指令,这通常需要提供预先生成的动作指令库。文献[55]发现 LLM 具有出色的总结和归纳能力,

LLM 能够总结用户偏好并且生成相应的机器人运动策略。相比之下我们的研究重点是提高大型语言模型在控制机器人行为方面的准确性,使机器人能够理解复杂的语义信息并成功执行任务。

还有很多研究探讨了利用学习型方法进行机器人轨迹优化,尽管机器人轨迹优化相关研究非常广泛但这些研究大致可以分为两类,第一类是使用学习模型进行机器人轨迹优化^[56],另一类是通过构建成本奖励函数或约束目标函数的形式使机器人轨迹达到最优^[57],这些方法是通过特定领域的运动数据进行模型训练,通过约束函数得到最优的路径^[58]。为了实现跨领域的泛化,另一类研究探索了从大规模离线数据中学习任务规范^[59],从大量离线数据中拟合最优路径,特别是通过大量的视频数据^[60]或是利用预训练的基础模型^[61]所包含的通用知识,将学习到的数据进行数据清洗后进行强化学习^[62]后生成机器人路径优化的轨迹^[63]。在本研究的第五章中我们利用 LLM 来处理自然语言指令,不需要人为设定硬编码的形式进行任务规划,并且我们在多组任务中实现了良好的泛化性。

1.3 相关理论基础

1.3.1 自然语言处理

自然语言处理的目标是使人能够和计算机进行有效交互,使计算机能够理解人类复杂的自然语言并且能够对自然语言进行推理生成可执行的策略。NLP 领域随着深度学习的不断发展也取得了显著的进展,并且在多个领域得到了广泛的应用。

NLP 相关研究起源于20世纪50年代,当时的研究大都围绕着语法解析等固定规则的方式,通过构建规范的语法体系和语义规则来实现自然语言的分析。这种符号主义研究范式虽在基础语言单位解析方面取得初步成效,但自然语言是多样且动态的,其应用边界存在明显的局限性,特别是在处理复杂语义关系和语境依赖时面临难以扩展的瓶颈。随着时代的发展,计算机的计算能力与数据存储能力得到了大幅度的提升,得益于上述技术的进步 NLP 逐渐向着数据驱动的方向转变。20世纪90年代,随着统计学方法的发展,这一时期的主流研究方向是数学概率模型,统计学理论的引入成功的使数学概率模型进行了突破,研究的重心变成了数学概率模型在语言建模中的使用。

以隐马尔可夫模型 (Hidden Markov Model, HMM) 为代表的概率模型成为主流研究方法,这种范式迁移使得语言处理从离散符号操作转向连续概率空间的特征学习。研究的重点变为了对语言库的统计规律发掘,通过最大似然估计等参数优化方法提升了语言模型的泛化能力与预测精度。例如,基于马尔可夫的 n-gram 模型,这个模型是一个典型的统计语言模型,该方法的核心思想是当前词汇出现的概率仅与少数上文有关。使用数学方法解释该方法认为当前词汇出现仅与前 n-1 个词相关。对于一个给定的句子 $S=w_1, w_2, \dots, w_T$, 其概率可以表示为式(1-1):

$$P(S) = \prod_{t=1}^T P(w_t | w_{t-1}, w_{t-2}, \dots, w_{t-n+1}) \quad (1-1)$$

这里, $P(w_t|w_{t-1}, w_{t-2}, \dots, w_{t-n+1})$ 表示当前词 w_t 在给定前 $n-1$ 个词条件下的概率。在 n -gram 模型中, 模型的目标是计算词汇出现的条件概率。

随着深度学习技术的发展, 尤其是卷积神经网络 (CNN) 和循环神经网络 (RNN) 的广泛应用, 自然语言处理进入了一个新的时代。深度学习模型通过构建大量数据样本自动学习特征, 极大地提高了文本理解任务的性能。尤其是在语言模型领域, 深度的神经网络能够捕捉抽象的长距离语义信息以及语法关系, 能够根据上下文信息生成流畅的具有逻辑的自然语言。这里我们以 RNN 为例, 假设一个 RNN 的隐藏状态 h_t 依赖于前一个时刻的隐藏状态 h_{t-1} 和当前输入 x_t , 其更新规则为式(1-2):

$$h_t = f(W_h h_{t-1} + W_x x_t + b_h) \quad (1-2)$$

这里 W_h 和 W_x 是权重矩阵, b_h 是偏置项, $f(x)$ 是激活函数, 通常选择 ReLU 激活函数。

近年来, 随着对 Transformer 架构的深入研究, Transformer 的核心是自注意力机制 (self-attention), 它能够对输入序列中的所有词进行并行计算, 从而有效捕捉长距离依赖关系。假设输入序列 $X = \{x_1, x_2, \dots, x_n\}$, 自注意力机制的计算可以表示为式(1-3):

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (1-3)$$

其中, Q 是查询矩阵, K 是键矩阵, V 是值矩阵, d_k 是键向量的维度。通过这一机制, 模型能够为每个词生成一个加权的表示, 关注的重点在于相关性高的向量, 从而能够根据上下文理解并且生成关联性强的文本。

大型预训练模型是近几年来 NLP 领域最为重要的突破之一, 预训练模型通过从海量语料库进行自监督学习, 提取文本特征并且能够将这种特征迁移到平级甚至下游任务中, 这一过程展现出了极强的泛化能力。通过从海量文本中进行特征提取学习到的丰富语言表达能力能够将模型适配各种具体的任务, 这一策略的成功使得大规模预训练语言模型成为 NLP 领域的主流方法。

1.3.2 大型语言模型

大型语言模型是基于 Transformer 架构训练的模型, 训练大型语言模型的过程就是从海量的文本中提取出语言的特征, 进而能够理解复杂的文本信息。LLM 的训练过程涵盖五个关键步骤, 依次为数据预处理、训练架构的选择、预训练、模型微调以及推理应用。

LLM 训练的数据来源于互联网上公开的大量文本数据, 由于原始数据中存在大量噪声, 所以需要对数据进行文本清洗和去噪并且执行文本标准化操作。这一步骤的目的是将数据转换成适合训练的格式, 为了便于模型训练通常会将文本进行向量化操作, 将文本转换为向量形式进行后续的训练步骤。

接着通过 Transformer 架构特有的自注意力机制计算词与其他词之间的关系, 即每个词的查询向量会与其他词的键向量进行相似度计算, 这个相似度衡量了词与词之间的关联程度, 为了正确的使词之间的相似度能够表征关联程度的重要性, 模型需要对相似度的结

果进行归一化操作。由于相似度的数值分布可能有较大的差异,比如某些相似度可能非常大,而有些则非常小。直接使用这些分布不均匀的数值进行训练会导致某些词的影响力过大,影响整体模型结构。归一化操作将相似度数值转化为一个范围在 0 到 1 之间的权重,这一步骤的目的是平衡每一个词对于整体文本的影响力。我们可以对每一个词进行加权平均,能够得到新的词向量的表示,通过 Transformer 特有的机制在进行推理的过程中动态的关注上下文中的多数词汇信息,这种方式能够良好的处理长文本中的语义逻辑关系。

在 LLM 的预训练阶段,模型通过无监督学习在大量没有标签的文本数据上进行训练,目标是学习到语言的基本模式和结构。这一阶段主要采用两种预训练方法。分别是掩码语言建模和自回归语言建模。

掩码语言建模的特点是,模型在训练过程中会随机屏蔽输入序列中的一些元素,并且模型的训练目标是模型根据上下文关系,预测出这些被遮蔽的词汇。在这一过程中,模型通过学习输入中未遮蔽的词来推断出被遮蔽部分的词语,从而学习到上下文中的依赖关系。在掩码语言建模中,模型的目标函数是最大化预测出屏蔽词的概率。对于输出序列 $X=\{x_1, x_2, \dots, x_n\}$, 假设某些词 x_i 是被屏蔽的,我们的任务须通过上下文 X_{context} 预测被屏蔽的词汇 x_i 。该过程表示为式(1-4):

$$P(x_i | X_{\text{context}}) = \text{softmax}(W_o h_i) \quad (1-4)$$

其中, h_i 是输入序列中的隐藏状态,由 Transformer 的编码器计算得到。 W_o 是输出层的权重矩阵,用于将隐藏状态 h_i 转换为输出词汇的概率分布。

自回归语言建模的特点是让模型根据输入序列的前部分词来预测下一个词。这是一种依赖序列数据性的方法,该方法没预测一个词,就将预测得到的结果作为下一个时刻的输入。假设我们有一个序列, $X=\{x_1, x_2, \dots, x_t\}$, 我们想要预测下一时刻的词 x_{t+1} , 同样的模型的目标函数依旧是最大化下一个词的条件概率,如式(1-5)所示:

$$P(x_{t+1} | x_1, x_2, \dots, x_t) \quad (1-5)$$

在训练过程中,模型会根据当前序列的上下文 $x_1, x_2, \dots, x_t$ 来预测 x_{t+1} 。在生成过程中,模型会逐步预测下一个词,并将其作为输入继续生成后续的词。

在预训练阶段,模型的目标是通过最大化的预测正确词汇的概率分布进而优化参数模型,通常上述两种方法需要构建相应的损失函数,如式(1-6)所示。

$$L_{\text{MLM}} = - \sum_{i=1}^N \log P(x_i | X_{\text{context}}) \quad (1-6)$$

对于自回归语言建模,损失函数为式(1-7):

$$L_{\text{AR}} = - \sum_{t=1}^T \log P(x_{t+1} | x_1, x_2, \dots, x_t) \quad (1-7)$$

模型训练的过程就是不断地优化上述损失函数进而迭代模型权重,最常见的方法是 Adam 优化器,该方法通过梯度计算以及自适应学习率机制,能够有效的更新模型权重。

在预训练阶段结束后,我们已经得到了一个模型权重。此时 LLM 训练会进入微调阶段,该阶段中模型进行有监督训练,通过数据微调进一步优化模型性能。微调阶段模型的

目标是通过优化特定任务的损失函数提高任务准确性。针对于构建的损失函数最小化为目标，该阶段损失函数可以表示为式(1-8)：

$$L = -\sum_{i=1}^N y_i \log(p(y_i | x_i)) \quad (1-8)$$

其中， y_i 是第*i*个样本的真实标签， $p(y_i|x_i)$ 是模型预测的概率， L 是整个数据集上的平均交叉熵损失。通过微调模型能够保留预训练阶段提取到的文本特征，同时还能够针对于特定任务进行优化提升准确度。

最后一个阶段是推理阶段，模型能够根据输入的文本生成回答，通过预训练和微调阶段训练的权重模型保留了海量文本特征，这一过程保证了权重模型能够被应用到各种自然语言处理任务。

1.3.3 LangChain 架构

LangChain (Language Chain) 是一种用于构建知识库与大型语言模型交互的框架，单一LLM由于模型缺少了专业领域的相关知识，导致LLM不能给出相关领域精确的文本信息。该架构旨在简化模型与外部数据之间的集成，LangChain的核心功能是将LLM最核心的语言处理能力与外部数据库的数据访问功能集成在一起，从而支持数据库问答的功能实现。

LangChain架构由多个组件构成，主要包括链、节点、数据源以及工具。通过组件之间的协调配合使用，能够构建一套多步骤且高效的工作流。如图1-1中所示，该图清晰地展示了LangChain全流程工作过程。在该架构中，链是重要且关键的，作用是将上一个功能模块的输出传输到下一个功能模块中，如图所示，箭头的指向就代表信息传递的方向，这是基于链的传递性构建的引导式数据传输方式。这种设计可以使系统易于管理，每一个子模块可以进行单独的调试，类似于程序开发过程中的断点操作，可以将每一个模块作为基本单元进行功能调试，每一个模块只负责一个输入以及一个输出，这样做的好处是可以最大程度的减小系统的兼容性问题。并且模块化的分布有助于开发人员根据自己的喜好自由的设计模块化结构以及独特的应用。

在LangChain的结构中，文本向量化后会将文本转化为向量形式，这些向量会被存储于向量数据库中。这一过程对应数据源模块的概念，该架构允许用户构建自己的本地知识组成的数据库并且允许大型语言模型访问数据信息。众所周知LLM由于预训练和微调过程已经更新过模型权重，重新训练一个大型语言模型成本高昂，也就是说一个固定版本的大型语言模型的知识是固定的。当LLM不具备某些领域知识的时候，数据库就起到了重要作用，用户可以构建该领域的独特知识，由于其数据可以被LLM访问，LLM通过知识的提取结合本身具有的语言处理能力，可以生成专业领域的精准回答。

为了能够从数据源中有限的检索正确的信息，一个关键的技术是检索增强生成技术 (RAG)，大型语言模型在收到问题时，将输入文本转化成向量形式，通过在外数据库计算向量相似度，动态的将知识库中的相关向量关联，再将检索到的知识作为上下文投

喂给LLM，此时LLM就得到了外部知识，对于某些专业领域的问题，LLM既能通过自身优秀的语言生成能力输出逻辑性文本，同时结合专业数据库的知识提供真实的资料，该过程就是增强了知识的正相关性从而提升模型回答的精准度。

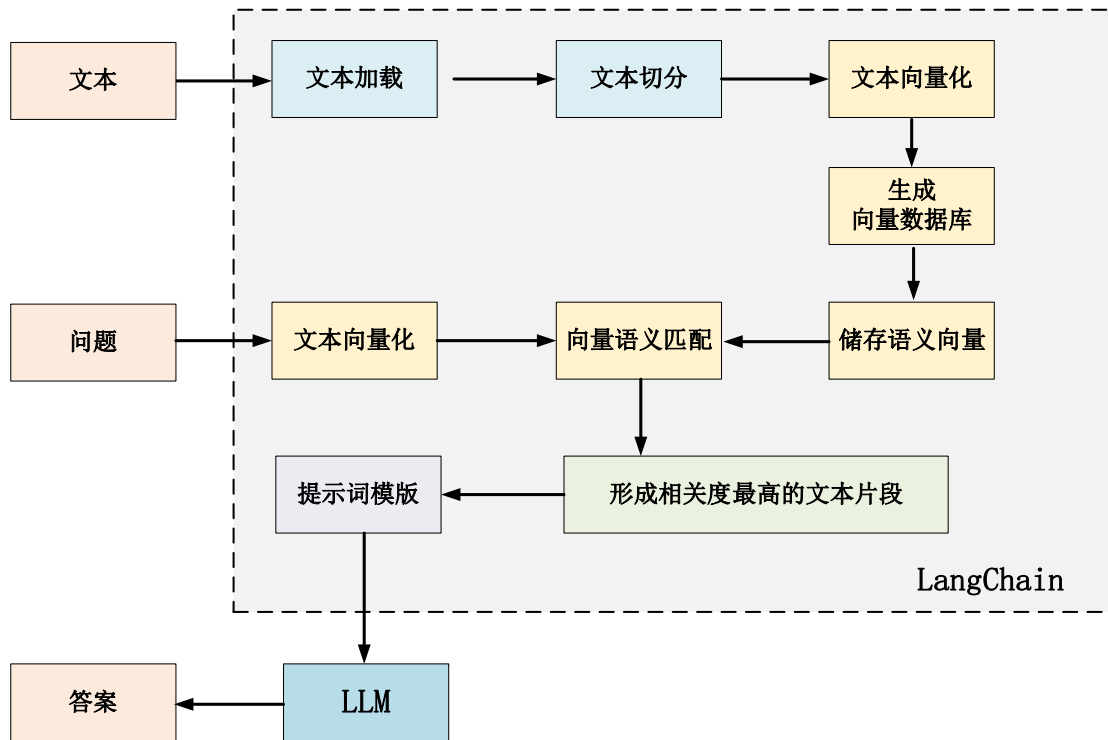


图 1-1 LangChain 全流程架构图

Fig. 1-1 LangChain full process architecture diagram

1.4 本文主要研究内容

基于上述电力巡检任务中机器人交互与任务规划的研究现状，本文将聚焦于研究电力系统巡检任务中面临的自然语言理解问题，电力系统巡检任务中的复杂任务难以分解的问题，巡检机器人个性化功能难以自动开发的问题，任务策略与机器人动作行为之间难以对齐的问题，电力系统巡检任务中模型与环境交互困难的问题，大模型生成结果出现的局部最优等问题。并且本章节阐述了 NLP、大型语言模型及 LangChain 架构的相关理论。首先介绍了 NLP 的发展历程，重点讲解了自然语言处理技术从规则驱动方法到统计学方法再到深度学习技术的演变，其中深度神经网络在自然语言理解与生成。接着，本章节详细说明了大型语言模型的训练过程，包括数据预处理、预训练、微调和推理阶段，并且介绍了掩码语言建模和自回归语言建模两种主要模型预训练方法。最后介绍了 LangChain 架构，它通过集成大型语言模型与外部数据源，利用检索增强生成技术，为复杂任务提供个性化解决方案。本章节层次化的论述了 NLP、LLM 及 LangChain 框架的核心概念以及工作原理。本文的研究内容如图 1-2 所示，各章节主要研究内容具体为：

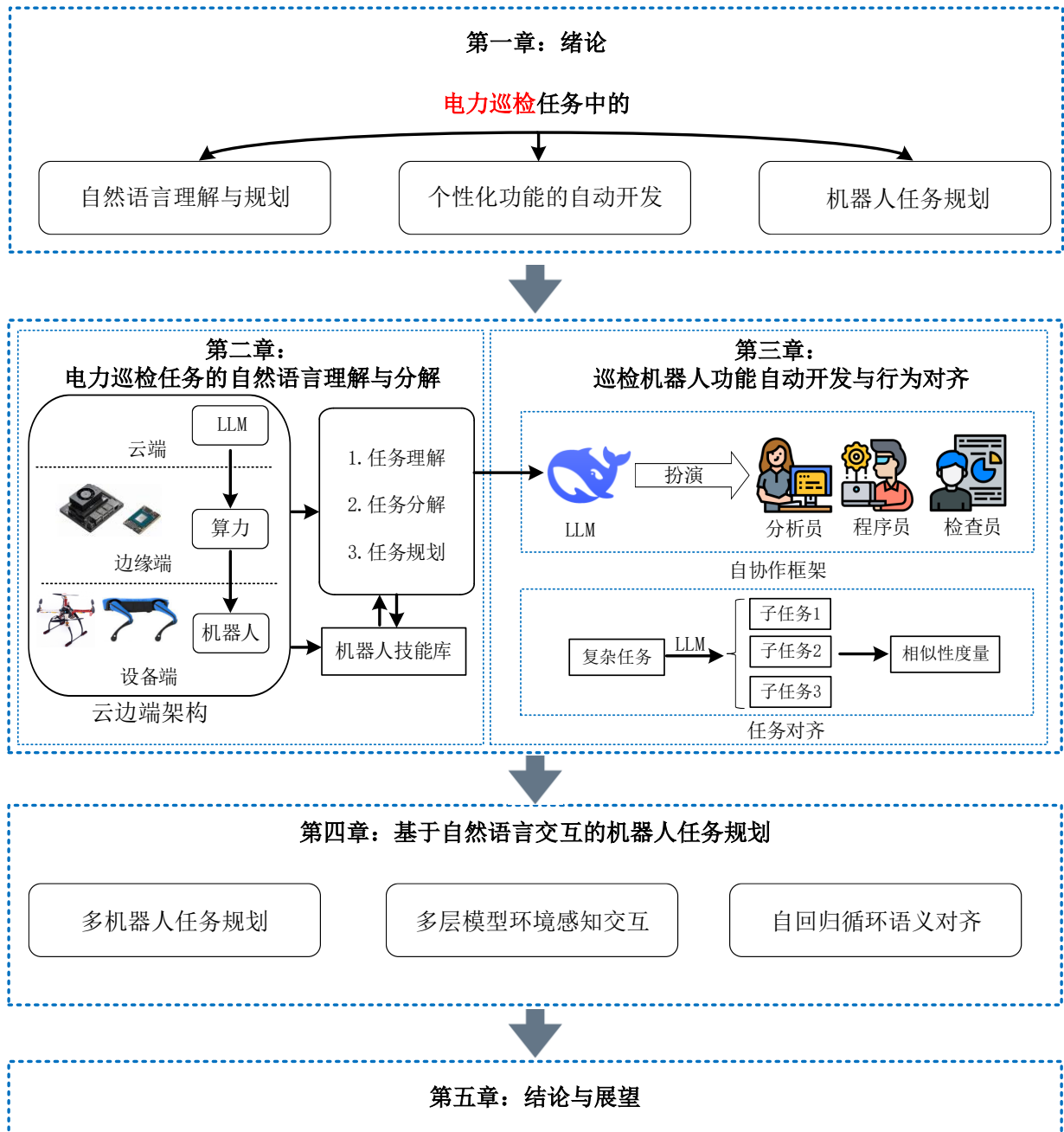


图 1-2 研究内容与章节安排

Fig. 1-2 Research content and chapter arrangement

第 2 章针对电力系统巡检任务中面临的自然语言理解问题,本文提出了基于自然语言处理的任务语义解析框架,该框架通过 LangChain 架构接入了具备本地知识的向量库,能够将本地知识通过向量的形式与大型语言模型进行交互,为了解决大型语言模型难以获得自然语言中特定知识的问题,我们构建了一个电力系统巡检场景的数据库,并且在 LangChain 架构中我们使用了检索增强生成(RAG)这一技术,通过动态检索外部知识增强了大型语言模型的本地知识获取能力。实现了电力系统巡检任务的准确理解。与此同时,针对电力系统巡检任务中的复杂任务难以分解的问题,本文提出了多智能体动态任务分解框架,该框架展示了一个多步骤的复杂任务经过主代理分解为细粒度的子任务序列,然后动态的将这些子任务分配给多个子代理的过程。该框架能够将复杂的多步骤任务分解为复

杂度低的更加易于执行的多个子任务。这个过程降低了任务的复杂度,增加了每个任务的可执行性,最终目的是使子任务在细粒度层面逼近原子任务,从而在行为一致性上进行对齐。针对本文提出的多智能体动态任务分解框架,通过 2.2.4 章节中的对比实验成功的验证了该架构的性能优于传统单 LLM 任务分解架构。

第 3 章的内容在上一节任务理解的基础上,针对巡检机器人个性化功能难以自动开发的问题,本文提出了一种多智能体自协作框架,该框架通过多个 LLM 相互协作共同制定机器人自动开发策略,并生成机器人可执行代码。为了保证多个模型之间的有效协同,该框架设计了明确的协作规则来模拟现实中机器人开发团队的协作模式,该框架的创新之处在于开发人员不需要掌握多个领域的专业知识,依靠语言描述就可以开发机器人个性化功能。在 3.1.3 章节中通过机器人功能自动开发实验我们能够得出以下结论,与传统的机器人开发团队以及单一 LLM 在处理机器人开发任务相比,多智能体自协作框架在效率方面具有明显优势。并且,针对任务策略与机器人动作行为之间难以对齐的问题,本文提出了一种基于余弦相似度的原子任务对齐方法,通过余弦相似度关联任务规划与机器人运动控制指令之间的语义相似性,通过语义相似性这一指标,本文进行了相似度匹配操作,能够将语义相近的文本进行一个映射,实现了任务与指令之间的对齐。同时由于现有的评估基准缺乏能够准确反映代理任务分解所需的细粒度层级,本文提出了一种基于语义度量的细粒度评价体系,该评价体系综合考虑了任务策略与机器人指令之的信息传播准确性和任务分解有效性,通过权衡这两种语义层面的重要属性能够有效解决任务到指令这个过程中产生的语义偏差和传播错误。并且本文在 3.2.4 章节中进行了语义度量基线实验和消融实验,证明了该方法的有效性。

第 4 章针对电力系统巡检任务中模型与环境交互困难的问题,本文提出了多模型交互与环境感知方法,将大型语言模型与视觉语言模型结合,能够对真实环境信息进行解析与理解,同时为了实现更精确的环境感知,我们设计了一种用于开放环境感知的热值图导航算法,通过构建一个可计算的二维空间,将摄像头采集到的视觉信息映射到二维可计算空间,能够反映出物体在真实世界中的相对位置信息。并且通过贪心算法的思想计算出最优路径,在热值图内生成密集的点状轨迹来指导机器人运动,这种映射关系允许机器人根据环境变化实时调整运动轨迹。在 4.1.4 章节的无人集群复杂任务执行实验中我们分别设计了基线实验和消融实验,验证了本文所提出的多模型交互与环境感知方法以及开放环境感知的热值图导航算法的有效性。同时,针对大模型生成结果出现的局部最优问题,本章节提出了一种自回归循环语义对齐方法,旨在将语义对齐过程中出现语义偏差的任务文本进行修正,这个过程通过将错误任务和提示词合并构建一个反馈信息,使大型语言模型在生成新一轮结果时在语义层面上偏向提示词所述的正确结果,能够有效的提高大型语言模型输出结果与机器人控制指令之间的语义相似性。为了进一步验证该方法的有效性,本文进行了基于自回归循环语义对齐方法的反馈实验,4.2.3 章节的实验中讨论了最佳反馈次数,并且验证了该方法能够有效提升任务执行的成功率。

第 5 章：结论与展望。本章节总结了本文的主要研究成果与贡献，并进一步分析了本文将来的研究方向。

2 电力巡检任务的自然语言理解与分解

本章主要针对电力系统巡检任务中面临的自然语言理解和任务分解问题展开,针对电力系统巡检任务中常见的自然语言理解困难的问题,本章节提出了一种基于 LangChain 架构的任务语义解析框架,该框架通过接入包含本地知识的向量库,结合大型语言模型实现高效的知识检索并且能生成更加精准的结果,这种方法能有效解决大型语言模型在面对专业领域问题时难以生成正确结果的问题。该框架能够检索外部知识库从而增强专业知识的获取能力,这种方式在提升电力系统巡检任务理解的准确性与可靠性方面表现出显著优势,对比实验的结果进一步验证了向量知识库的引入能够降低 LLM 输出结果的幻觉现象并且提升回答的准确率。

此外针对电力系统巡检任务中的复杂任务难以与底层原子任务对齐的问题,本章还提出了多智能体动态任务分解框架。该框架通过主代理对复杂任务进行动态分解,将其转化为细粒度的子任务,并自动将子任务分配给多个子代理执行,从而降低了任务执行的复杂度,提升了任务的可操作性。与传统的单一 LLM 任务分解架构相比,该框架能够将任务分解到更加细粒度的层面,通过实验验证该框架在多步骤分解任务中能够将任务分解到原子任务层面,这一步骤能够有效降低后续任务执行的难度。

2.1 面向电力巡检任务理解的云边端架构设计

2.1.1 自然语言理解问题

电力系统巡检任务通常会涉及到复杂操作以及大量自然语言形式的任务,在传统的电力巡检方法中这些任务是由人工完成,随着巡检工作量的增加人工巡检的效率逐渐无法满足现代电力系统的需求。为了提高巡检效率并降低人工成本,引入机器人技术成为一种有效的解决方案。机器人执行电力巡检任务的过程中首先要正确的理解任务才能顺利的开展后续工作,为了巡检任务的顺利执行,机器人必须能够准确理解巡检任务的复杂指令并将其转化为可执行的步骤,然而现有技术在任务理解方面面临诸多困难。

传统的基于规则的自然语言处理方法难以处理复杂的任务,这是因为基于规则的方法依赖人工设置的语法规则或者使用特征提取等手段,如字段匹配和关键词检索来解析任务描述的文本,然而这种方法在面对语言中的模糊性或非标准表述时表现不佳。电力巡检任务往往是灵活多变的,通常情况下基于规则的方法并不能适配这种非固定格式的语法规则,这种情况下就难以提取文本的特征并且会导致任务执行过程中出现理解偏差和误解。

随着 LLM 在自然语言处理领域的表现受到了广泛关注。LLM 凭借其强大的文本生成和理解能力能够快速的处理大量文本数据。尽管 LLM 展现出了较高的语言理解能力,但是幻觉现象会导致 LLM 输出错误的结果,幻觉现象指的是 LLM 在理解文本的过程中依赖少量的上下文信息而忽略真实情况导致输出出现错误的一种现象,尤其是在文本中存在

上下文缺失或模糊的表述时幻觉现象尤为明显。尽管 LLM 可以生成灵活的结果，但是由于幻觉现象和上下文缺失导致 LLM 输出的结果与实际内容出现偏差，在这种情况下会影响任务执行的准确性。

在进行自然语言理解时，传统的基于规则的方法和单独依赖 LLM 推理的方式都存在各自的缺陷。在电力系统巡检任务中，复杂的巡检任务需要一种新的方法能够有效理解任务信息并且输出基于真实环境的任务执行策略。

2.1.2 云边端架构的任务理解框架设计

在 2.1.1 中我们阐述了任务理解的重要性以及机器人理解任务是执行任务的前置条件。通过上文可知，LLM 作为任务理解的基座有着不可替代的作用，LLM 能够将非规范化的文本转换成结构化文本，通过信息传输的方式传递到下游单位逐级执行，这个结构化文本生成依赖与 LLM 的任务理解能力，当 LLM 正确的理解了问题，才能给出正确的回答。但对于一个系统来讲，执行整个流程需要一个完备的架构提供完整且稳定的逻辑推理到任务执行的所有步骤。

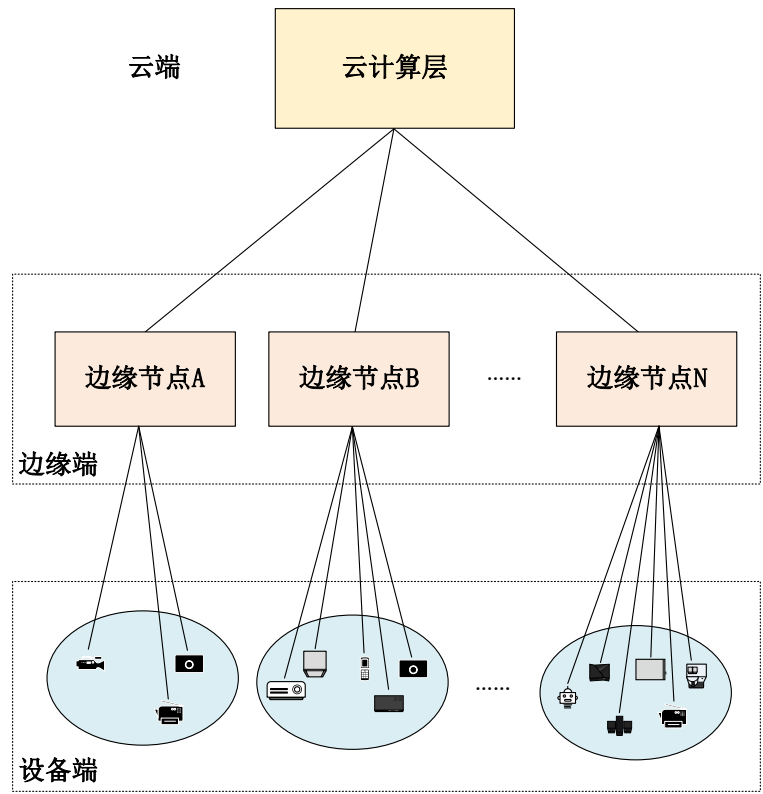


图 2-1 云边端架构框架图

Fig. 2-1 Cloud-edge architecture framework

为了实现复杂任务的理解，我们设计了一种基于云边端架构的任务理解框架。首先我们介绍云边端架构，如图 2-1 所示，该架构分为三个层级、分别是云端、边缘端、设备端。首先云端指的是部署在大型服务器集群的云计算平台，例如华为云、阿里云等云计算平台。边缘端也就是本地环境，通常由具备一定算力的设备构成多个边缘节点，用于与云端以及设备端进行通信。由于云端发送的策略难以直接控制机器人运动，需要边缘端的计算节点

进行数据处理才能转换为机器可执行指令。通常边缘节点在处理云端信息后会将信息下发给设备端的多台机器。

由于任务理解依赖于 LLM 的性能，而 LLM 的性能与参数量成正相关，即参数量越大的模型在性能上也会越强。目前主流的大型语言模型如，DeepSeek-R1 大模型完全体参数规模是 671b,本地部署这种主流大型语言模型是非常昂贵的。所以我们使用云计算平台部署超大规模模型。利用绪论中描述过的 LangChain 架构实现知识向量库的提取与存储，通过提示工程（prompt Engineering）的方法能够使大型语言模型回答我们期望的信息。如图 2-2 所示，该图为云边端架构任务理解框架。云端能够将 LLM 生成的结果下发到边缘端，同时也能够接收边缘端上传的任务。

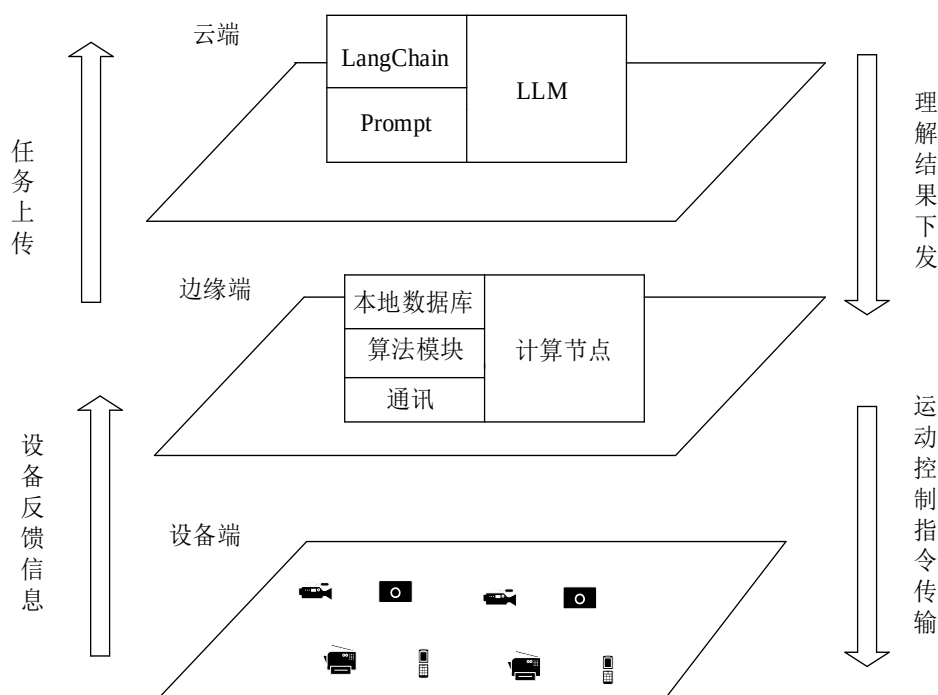


图 2-2 云边端架构任务理解框架

Fig. 2-2 Task understanding framework of cloud-edge architecture

边缘端通常是本地环境，通过具有一定算力的设备构成计算节点，边缘端的设备具有本地数据知识以及轻量级程序模块，可以将任务内容以及本地数据知识通过通讯模块上传到云端，在云端部署的 LLM 会根据接收内容的上下文关系进行语义理解与任务分析，将生成的策略信息下发到边缘端节点。当边缘端接收到云端发送的策略信息后，边缘端计算节点中的轻量级 LLM 通过解析策略信息的内容调用本地数据库以及算法模块生成机器可执行操作或指令，接着将指令下发到设备端执行。

设备端是云边端架构的下级执行单位，控制设备端机器人运动的指令由边缘端的通讯模块发送到具体设备，设备依据可执行指令完成任务。同时搭载传感模块的设备能够及时反馈任务执行情况，交由边缘端进行处理。边缘端将反馈信息上传至云端进行逻辑判定，然后云端 LLM 进行策略生成后将输出回传到边缘端的计算节点，最终由边缘端的通信模

块将信息下发到具体执行设备上，这一过程形成了信息传输的闭环。

2.1.3 基于自然语言处理的任务语义解析框架

在 2.1.1 章节中，本文探讨了 LLM 在处理复杂任务时，由于文本中会存在上下文缺失导致的 LLM 输出的结果出现幻觉问题。大型语言模型能够提供灵活的输出，但其生成的信息不一定符合实际任务需求，因为 LLM 不能获取到环境的真实信息导致无法推理出正确的结果。例如在执行一个电力巡检任务时，可能涉及到当地地点名称或者任务代号，此时大型语言模型因为缺失了本地知识导致不能正确理解任务需求，这种情况下 LLM 就不能输出正确的结果。这种现象在本文的 1.3 章节也有所讨论，原因在于 LLM 经过预训练和微调后模型的参数是固定的，此时 LLM 的知识容量已经确定。大型语言模型只能依靠接收到的上下文信息来进行逻辑推理。

LLM 具有上下文推理能力可以在一定程度上通过前文信息做出合适的判断，但它无法真正理解不在训练数据中的新知识，LLM 的能力虽然强大，但它仅限于已知的信息，并且会受到训练时使用的文本数据范围的约束，简而言之 LLM 的权重停止更新意味着在预训练和微调之后，模型的能力和知识库是固定的，无法自动获取新的领域知识或实时更新。当任务描述中涉及未见过的信息或非常专业的知识时，LLM 仅能根据已有的上下文进行推理和生成输出，而无法通过额外的学习或记忆补充缺失的知识。这就是为什么 LLM 在处理复杂或专业的任务时，容易产生幻觉现象，原因在于 LLM 不能从真实环境中学习和更新自己的知识库。

为了使 LLM 能够准确理解专业领域的相关任务，我们需要构建一个基于 LangChain 架构的语义解析框架。我们面临的另一个问题是如何准确的通过大型语言模型自动化的提取出任务信息，因为使用者在执行巡检任务的过程中难以对系统提供有效的上下文，具体来讲，我们希望系统能够对复杂任务的文本信息分析后直接输出任务的关键信息，该过程中我们需要使用检索增强生成这一技术，通过动态检索外部知识来增强 LLM 的文本生成能力。

在本章节中提出了一种基于自然语言处理的任务语义解析框架，值得说明的是，该框架是结合 LLM 与向量数据库的一种系统性架构，该框架部署在 2.1.2 章节中所述的云端，通过云端部署的 LLM 接入向量数据库实现专业领域知识的获取。如图 2-3 所示，展示了任务语义解析框架。首先，我们将需要解析的电力巡检任务文本传输给系统，接着系统对电力巡检任务文本进行文本切分操作，具体来讲，就是将整个文本切分为多个段落。这是由于过长的文本会影响后续信息提取的效率。切分后的文本可以将打断的字符分离成更小的短语，在后续步骤中有利于 LLM 高效的识别关键词和其他重要信息。接着通过 Embedding 操作将切分后的文本转换成向量形式储存在向量数据库中。

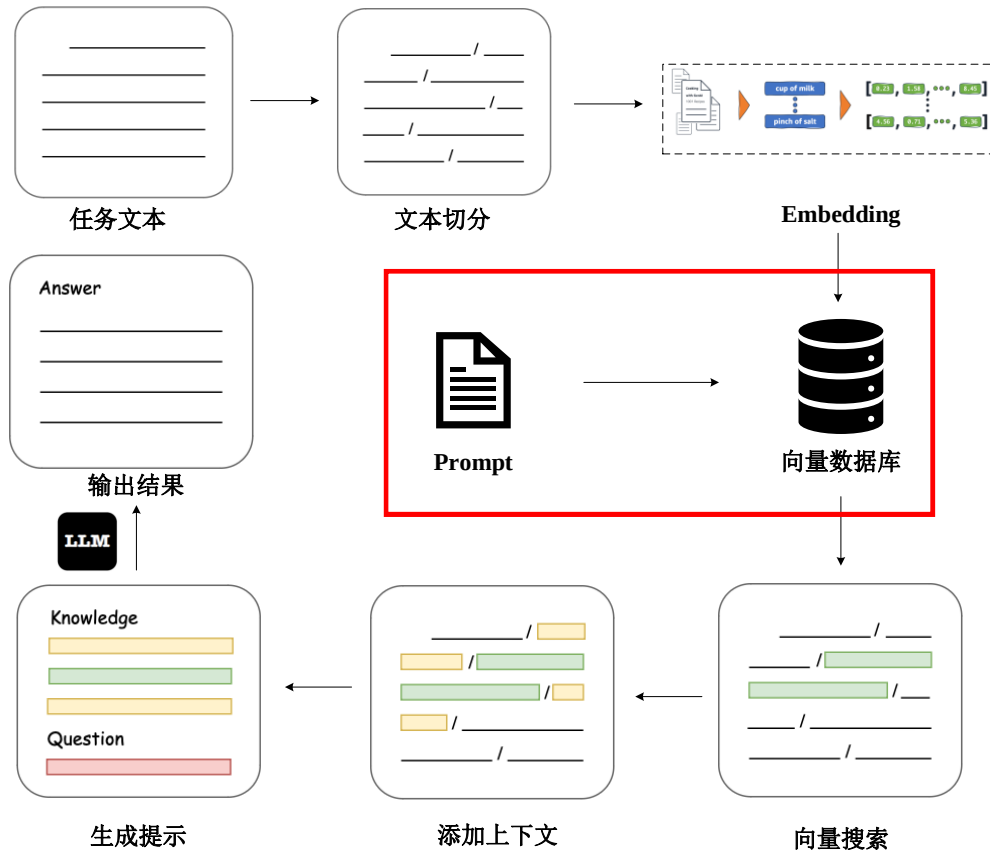


图 2-3 任务语义解析框架结构图

Fig. 2-3 Task semantic analysis framework structure diagram

为了实现自动化的任务理解而不需要人为手动输入问题，我们通过提示工程构建了 Prompt 提示，将 Prompt 向量化后存储在向量数据库中进行后续操作。此时我们的查询需求已经通过 Prompt 加载到了向量数据库中，系统会使用检索引擎从数据库中获取相关文档。文档和查询在向量数据库中都被映射为向量形式，通过计算向量之间的余弦相似度就能够得到查询与切分文本之间的相关性，相关性计算公式如式(2-1)所示。

$$\text{Similarity}(q,d) = \frac{q \cdot d}{|q| \cdot |d|} \quad (2-1)$$

其中， q 是查询向量， d 是文档向量。同时为了得到文档中的重要词汇，本文采用的方法是 TF-IDF 公式为(2-2)，该方法通过计算查询词的频率以及整个文档中出现该词的逆比值结合，评估文本重要性。

$$\text{TF-IDF}(w,d) = \text{TF}(w,d) \cdot \log \left(\frac{N}{|\{d : w \in d\}|} \right) \quad (2-2)$$

其中， w 是词， d 是文档， N 是文档总数。上述步骤我们对文档进行了初步的筛选，接着需要对初步筛选的文档再次进行二次筛选。这一过程的目的是确保最终筛选出的文档之间具有最高的相关性。通过损失函数对整个文档进行筛选最终得到最高相关性的向量，该向量的表示如式(2-3)所示。

$$L = -\sum_{j=1}^M y_j \log(p(y_j | q, d_j)) \quad (2-3)$$

其中 y_j 表示文档的真实标签,也就是是否相关的标签, $p(y_j | q, d_j)$ 指的是模型对于文档是否相关的预测概率。 M 是本次检测的文档数量。此时得到了与查询向量具有高相关性的文本向量集合。下一步需要将这些被筛选出来的文档转换为一种LLM能够接受的上下文类型的格式。首先,系统将筛选出来的文档进行拼接,生成一个长文本序列,为了将长文本序列转换为LLM可接受的格式,需要进行编码操作。使用Transformer的编码器(Encoder)对拼接好的文档进行编码,以供后续的生成步骤使用。编码后的上下文标记为 c , c 的计算如式(2.4)所示。

$$c = \text{Encoder}(d_1, d_2, \dots, d_k) \quad (2-4)$$

这里 $d_1, d_2, \dots, d_k$ 是经过筛选的相关文档, k 是文档数量。此时我们得到了一个与查询问题高度相关的长文本向量。通过提示工程,我们将长文本知识以及我们 prompt 中的问题组合,构成图 2-3 中的提示部分。

我们将得到的提示输入 LLM,这时 LLM 接受用户的原始查询和检索到的上下文信息,以生成最终回答。通过上述流程我们能够将本地知识接入 LLM,能够根据现场环境,理解电力巡检任务中的关键信息,并且能够生成符合本地真实情况的策略。

2.1.4 电力系统知识库 LLM 对比测试

在 LLM 理解复杂的任务时会出现幻觉现象导致任务失败,原因是 LLM 在预训练和微调完成后,模型的权重不再更新。即 LLM 的知识储备已经固定,此时大型语言模型只能沟通过接收到的上下文来进行逻辑判断。在 2.1.3 章节本文提到了使用外接向量数据库的方法遏制由于上下文缺失导致的 LLM 幻觉现象,向量知识库能够通过计算向量之间的相似度,迅速从庞大的数据集中找到相关信息。这种方法不依赖于语言模型的固定知识储备,而是能够根据上下文从数据库找出最相关的内容补充上下文缺失的信息。通过这种方式在执行复杂任务时,LLM 能够根据知识向量库得到环境真实信息,从而推理出准确度更高的结果。

表 2-1 变电站知识库区域信息示例

Tab. 2-1 Substation Knowledge Base Area Information Example

区域	区域信息
A 区域	"#1 主变", "220kV GIS 室", "继保室"
B 区域	"#2 主变", "110kV 开关场", "蓄电池室"
C 区域	"SVG 无功补偿装置", "接地变"

要解决电力系统巡检任务中复杂指令理解的问题,并证明向量知识库在任务理解中的重要性,我们设计了一个实验,通过比较不同的任务理解方法来验证知识库的有效性。我们构建了一个结构化变电站知识库。表 2-1 为我们构建的变电站知识库的区域信息的部分场景示例,我们将变电站的不同功能区域进行划分,并且将详细的区域信息储存在数据库

中。表 2-2 是变电站知识库的设备信息示例，我们在知识库中详细给定了设备信息与对应的参数，并通过格式化的 JSON 格式进行储存，便于 LLM 后续的检索与调用。

表 2-2 变电站设备信息示例

Tab. 2-2 Substation equipment information example

设备信息	参数
#1 主变型号	"SFSZ11-240000/220"
#1 主变 油温报警阈值	85°C
#1 主变 最近检修日期	"2023-11-20"
220kV GIS 室 气室数量	0.45MPa
220kV GIS 室气 SF6 压力报警值	8

对于接入向量知识库的模型以及纯 LLM,我们设计了三个问题,分别是 220kV 问题 1: GIS 室在哪个区域? 包含几个气室? 问题 2: 检查继电保护装置需要带什么工具? 最近的装置在哪里? 问题 3: #1 主变当前运行状态是否正常? 我们将这些问题分别询问接入向量知识库的 LLM 以及纯 LLM,结果如表 2-3 所示。

表 2-3 电力系统知识库 LLM 对比测试结果

Tab. 2-3 Comparative test results of power system knowledge base LLM

模型	回答结果
220kV GIS 室在哪个区域? 包含几个气室?	
知识库增强模型	根据先锋变电站布局数据: 220kV GIS 室位于 A 区, 该 GIS 设备共包含 8 个独立气室。当前 SF6 压力报警阈值为 0.45MPa, 请注意监测。
纯 LLM 模型	220kV GIS 设备通常布置在配电装置区域, 常规设计包含 4-6 个气室。具体位置需要查看现场布置图。
检查继电保护装置需要带什么工具? 最近的装置在哪里?	
知识库增强模型	继保室位于 A 区, 标准巡检工具包应包括: 万用表、继保测试仪、绝缘电阻表。根据 2023 版作业规程, 还需携带最新的定值单。
纯 LLM 模型	常规需要携带万用表和测试笔, 继电保护装置通常靠近主控楼或变压器区域。
#1 主变当前运行状态是否正常?	
知识库增强模型	#1 主变 (SFSZ11-240000/220 型) 最近检修时间为 2023-11-20, 当前油温 72°C (低于报警阈值 85°C), 建议结合负荷率进行状态评估。
纯 LLM 模型	主变正常运行温度通常在 65-80°C 范围内, 具体状态需要查看实时监测数据。

通过本实验可清晰验证, 向量知识库的引入有效降低了 LLM 的幻觉现象。从结果能够看出纯 LLM 回答的结果是不正确的, 由于缺失了上下文中的关键信息, 仅仅依靠 LLM 的推理能力不能生成准确的结果, 向量知识库的引入能够补全上下文缺失的信息从而使

LLM 能够生成正确的结果。

2.2 基于大模型代理的电力巡检任务分解框架

2.2.1 复杂任务分解问题

复杂任务分解问题是任务分解领域的一个核心挑战, 尽管 LLM 已经能够有效的理解自然语言, 但是现有方法在面对复杂问题时依然难以将问题拆解为多个可执行的子任务。现有的任务分解方法只能将任务分解为粗粒度的任务, 难以实现任务的精细化分解, 并且现有的任务评估方法也只能在宏观层面判断任务是否执行成功, 缺少了中间过程的理解与分析。在现实场景中, 一个任务通常包含多个细粒度的子任务, 仅用任务是否成功完成来评估任务, 无法反映细粒度子任务的真实完成情况。例如, 在电力巡检任务中, 尽管最终任务未能完成, 其中的某些子任务可能已经部分完成, 这些未完成的工作为后续工作的成功执行具有一定的作用, 现有方法未能有效应对这种情况。

从方法论的角度来看, 现有方法是将一个复杂的多步骤任务拆解为多个环节的子任务, 相比与拆解前的总任务来讲多个子任务的复杂度更低, 更易于成功的执行。然而以上方法存在两个缺点, 错误传播和适应性有限。在许多研究中, 对复杂任务进行任务分解时, 最小单元往往被视为固定且不可更改的, 这是由于底层动作执行单元功能的限制。这种设置导致子任务变得固定, 不具备灵活性。这意味着, 一旦某个早期子任务出现失败, 错误就会在后续步骤中传播, 导致整个任务的失败。错误传播现象使得任务分解无法准确反映代理在任务中的增量进展, 从而导致人们无法评估任务的可执行性。此外, 传统方法通过子代理硬编码来处理子任务, 这种静态设计不仅耗时费力, 而且缺乏足够的普适性和灵活性。因此, 在面对现实世界中动态变化的任务需求时, 这类方法难以适应不同的任务情境。

综上所述, 复杂任务的拆解不仅仅要考虑将任务拆解为多个子任务, 还需要能够对拆解后的子任务进行跟踪评估, 并在评估时充分考虑子任务的完成情况。现有的任务分解方法在细粒度分解和错误传播的处理上存在不足。需要一种能够将任务进行分解又能够评估任务执行完成度的方法。

2.2.2 任务分解机理与技能库的构建

如 2.2.1 中所述, 电力系统巡检过程中任务分解是一个复杂的问题, 多步骤的任务分解中, 由于存在错误传播和有限的适应性问题, 必须要对任务分解的程度进行控制。一个任务能否被成功执行有多方面的原因, 首先任务分解的最下游就是任务执行端, 任务是否能够成功执行取决于执行端能否完成任务。机器人执行任务的能力是有限的, 因为需要遵从机器人硬编码提供的任务执行能力, 这些能力是提前人为设定的。机器人的能力与任务分解的结果相匹配, 该任务就能够被机器人执行。由此可见, 系统需要将任务分解为机器人能够执行的多步骤子任务, 机器人才能够顺利执行任务。

传统任务分解方法依赖于人工经验, 丰富经验的人能够依赖自身所学会的知识以及对现场环境的理解将复杂任务拆解为便于执行的子步骤。人工分解的目标是将一个多步骤的

复杂任务分解成可以单独完成的子步骤，关键点就是分解的子任务的层级。对于机器人来讲，能够执行的任务是固定的，直接执行一个复杂的任务或者多步骤的任务是难以执行成功的，我们需要将任务分解为多步骤的细粒度任务，在执行层级上与机器人执行任务的基本单元相匹配。即子任务与原子任务处于一个细粒度层级，这是机器人自动执行任务的前提条件。

现阶段复杂任务的自主分解通常依赖于 LLM 所提供的语言理解能力和逻辑推理力。通常如果想将一个任务分解成多个步骤，我们需要为 LLM 提供包含任务信息的上下文。LLM 在分解任务过程中，的确能够将多步骤任务分解为复杂度较低的子任务，但是这与机器人可以直接执行的任务还是具有较大差异。例如，我们得到了一个电力巡检任务：巡检电力变压器的运行状态，这个任务通常分为多个步骤执行，将这个任务交给 LLM 进行拆解我们会得到复杂度降低的任务执行流程，如检查变压器的外部状态、内部温度、油位、运行声音、连接线是否存在异常等。如果出现任何异常，机器人需要记录并上传数据，或者进行简单的处理和调整。从任务的复杂性角度来评判，任务分解后的子任务执行难度下降了，此时人工执行每一步子任务会相对容易完成，但机器人无法完成这些任务。原因在于，机器人可执行的任务是基于人为编码定义的多个功能模块，其执行任务的层次是更加细粒度的，必须将任务分解的层次与机器人现有功能匹配，即生成细粒度子任务，这是调用机器人运动模块的前置条件。

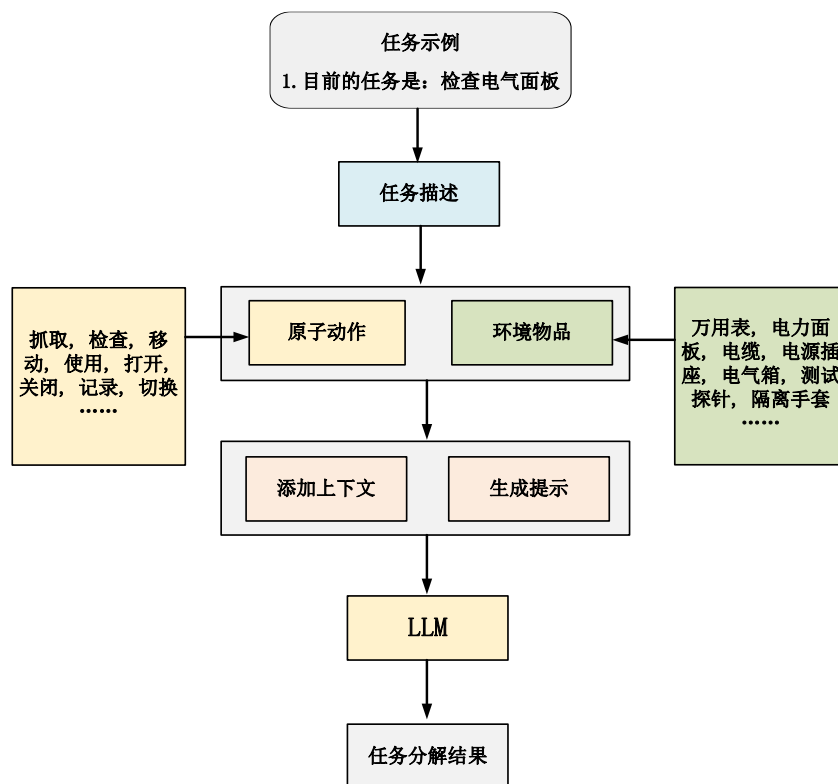


图 2-4 电力系统巡检中的任务分解示例

Fig. 2-4 Example of task decomposition in power system inspection

更加重要的一点是，在执行多步骤任务的过程中，我们将任务拆分为多个子任务，任务执行的流程是第一个子任务执行成功然后开始执行第二个子任务，以此类推，当上一个

任务判定执行成功后开始执行下一个任务。那么任务执行成功的判定就显得尤为重要，本章节我们只讨论不同细粒度任务判定的影响，具体任务成功判定方法我们将在 4.2 章节进行讨论。多步骤任务在进行判定时遵从二元分数进行判断，也就是一个任务的判定结果只存在两种可能，任务成功和任务失败。在现实场景中，使用这种宏观的判断方法是无法正确反映细粒度子任务的真实完成情况。就如 2.2.1 章节阐述的例子，一个任务被判定执行失败，假设该任务是一个未被分解的多步骤任务，失败的原因可能是发生了更加细粒度层面的错误传播导致后续子任务序列停滞。一个任务被分解为最基本执行单元，我们认为该任务就是细粒度的最小单元，在这里我们称为原子任务。从原子任务的视角来看，即使最终任务没有完成，某些子任务在可行性上仍会部分成功并为后续工作奠定基础。所以将任务分解到原子任务级别是必要的。

为了实现 LLM 将任务分解到原子任务级别，需要给 LLM 标准化知识体系。也就是 LLM 通过知识库的形式能够识别原子任务并且将任务分解的目标映射到该知识体系。本文构建了一个机器人可执行技能库，以便在 Agents 成功完成任务分解后保存相关信息。这个技能库不仅存储基本执行单元，即原子任务的相关信息，还允许多个 Agents 调用其中的知识以进行逻辑匹配和进一步的任务分解。

图 2-4 中我们展示了一个电力巡检任务分解示例，能够模拟现实世界中动态多步骤任务分解流程。本章节关注的核心问题是任务是否被分解彻底，即分解到原子任务层面。我们会输入一个宽泛的任务，例如我想要检查电缆连接，这种宽泛的一个开放场景下的问题，通过查看我们给定的技能库获得原子动作以及环境信息进一步的进行任务规划和任务分解。最终的目标是，将任务分解为技能库中设定的一系列原子任务，即任务被彻底分解。

如图 2-4 所示，我们将一个 LLM 的 API 端口作为一个代理(Agent)，代理收到我们发送的任务请求，同时能够调用技能库中的原子任务以及环境中的物品，技能库中的原子任务视为可以直接执行的动作，技能库中的物品视为当前环境具备的物品，我们期望能够将输入的任务进行任务分解，分解为原子任务级别的动作序列，通过这种方式就能够使下游执行端直接执行这种抽象且宏观的任务了。并且我们兼顾了当前环境物品信息，最大程度的保证了任务分解过程中的动作能够对应的找到环境中具备的物品，如果没有该物品，我们可以在后台的日志中发现，并且即时的添加该物品或者修改不完备的动作，将新的信息添加到技能库中。如表 2-4 所示。

表 2-4 技能库相关功能与模块划分

Tab. 2-4 Skill library related functions and module division

原子动作	环境物品	已成功执行的任务
1.保存大量已有原子任务 2.能够动态添加原子任务	1.保存环境中大量已有物品 2.能够动态添加环境中的物品	1.能够保存任务分解的结果，作为成功执行的经验知识便于后续相关问题的调用。

如图 2-5 所示，为了构建技能库，我们收集了大量电力系统环境中巡检任务所需的作业工具以及机器人操作系统中的常见操作对象，将上述信息作为技能库中的环境物品，可

以通过原子任务进行调用。并且我们构建了包含作业场景中常见的动作以及日常行为活动中的动作作为原子动作存储在技能库中。值得强调的是环境物品不只是工具类物品，而是指能够通过原子任务调用的工具，举一个抽象的例子，我们需要通过计算机上传一个文件，上传就是一个原子任务，文件在这里就可以被视为生产环境中的物品被原子动作调用。

为了使 LLM 生成符合用户偏好的结果，我们通过提示工程的方法在技能库中储存了规范化的提示文本对任务分解的过程进行指导。在技能库中原子动作库代表了执行的基本动作单元，而环境物品库则保存着不同环境中的物品信息。同时本文还构建了一个包含人类先验知识的动态分解库，通过动态检索的方式将提示传输给 LLM，便于 LLM 在历史信息中找到最符合要求的结果。已经执行的任务会以日志的形式进行存储，所有信息都以标准化的 JSON 格式存储，便于数据库的调用。

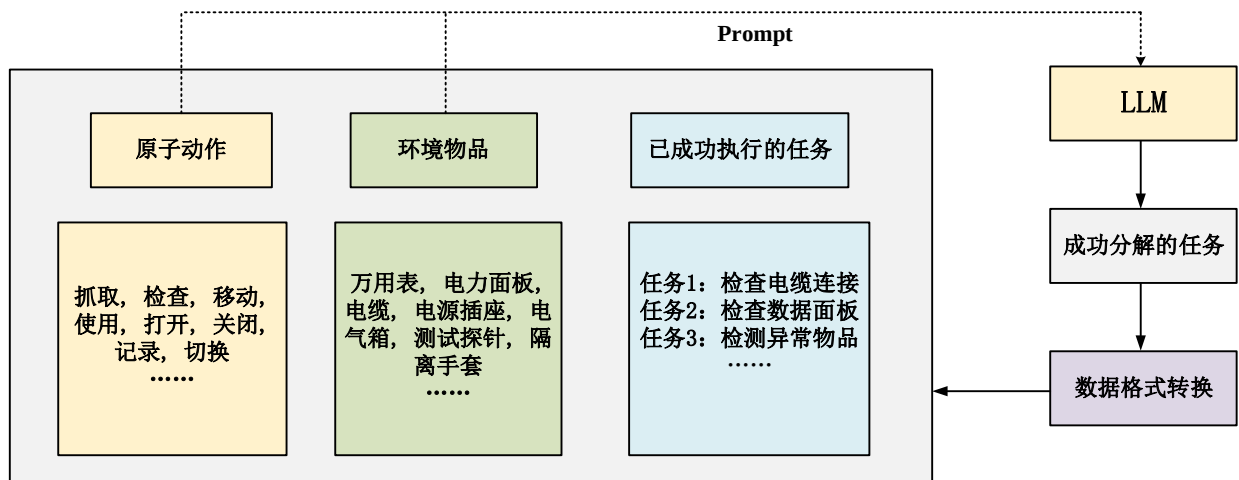


图 2-5 技能库组成元素示意图

Fig. 2-5 Schematic diagram of the components of the skill library

2.2.3 多智能体动态任务分解框架构建

为了应对 2.2.1 章节中描述的问题，本章节提出了一种动态任务分解框架，该框架能够针对现实世界中不同复杂度的任务进行有效的分解。框架能够将复杂的多步骤任务分解为复杂度低的更加易于执行的多个子任务。这样的好处是降低了任务的复杂度，增加了每个任务的可执行性，最终目的是使子任务在细粒度层面逼近原子任务，从而在行为一致性上进行对齐。

如图 2-6 所示，该框架展示了一个多步骤的复杂任务经过代理分解为多个细粒度的子任务，然后动态的将这些子任务分配给多个子代理的过程。我们将任务需求输入系统，一个后台接入 LLM 的主代理首先将任务进行初步分解，生成粗粒度的任务，这个过程可以减轻后续精细化分解的计算开销。第二步将粗粒度的任务分配给多个子代理，为了确保任务分解的质量，每一个子代理只负责一个粗粒度任务的分解。子代理参照技能库的原子动作以及环境物品，将粗粒度任务分解为原子动作级别的原子任务。

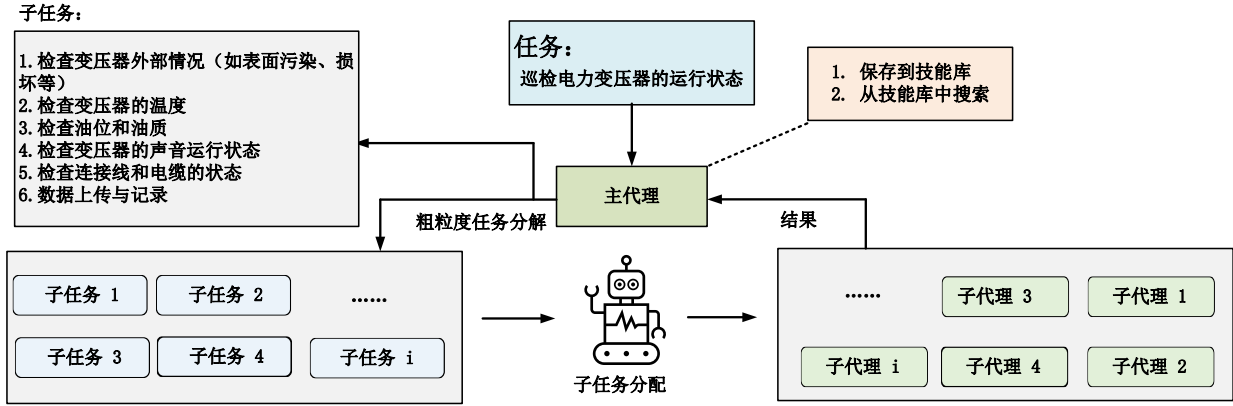


图 2-6 多智能体动态任务分解框架

Fig. 2-6 Multi-agent dynamic task decomposition framework

为了应对高复杂度问题，本框架采用多智能体动态任务分解策略，将任务分解为更小的子任务，并动态调整子任务的执行策略以提高整体任务解决效率和准确性。主代理接收原始任务 T ，并将其分解为一组子任务 $\{t_1, t_2, \dots, t_n\}$ ，表示为式(2-5)：

$$(t_1, t_2, \dots, t_n) = \text{MainAgent}(T). \quad (2-5)$$

其中 t_i 是第 i 个子任务，子任务序列 $\{t_1, t_2, \dots, t_n\}$ 是主代理根据任务目标和技能库动态生成的。每个子任务 t_i 被分配给子代理执行，执行结果 r_i 表示为式(2-6)：

$$r_i = \text{SubAgent}(t_i, \{t_1, \dots, t_{i-1}\}, \{r_1, \dots, r_{i-1}\}) \quad (2-6)$$

子代理在执行 t_i 的过程中，会参考前序任务的定义 $\{t_1, \dots, t_{i-1}\}$ 和结果 $\{r_1, \dots, r_{i-1}\}$ ，以动态调整其任务策略生成准确的结果。子任务的定义在执行过程中是动态更新的，更新后的子任务表示为式(2-7)。

$$t_i = \text{Update}(t_i, \{r_1, r_2, \dots, r_{i-1}\}) \quad (2-7)$$

其中，动态更新函数 Update 考虑了子任务 t_i 的当前定义以及前序任务结果的反馈信息，用以修正可能的任务偏差或适应新信息。子代理在成功完成子任务 t_i 后，会生成任务过程摘要 s 并更新技能库 L 由下式(2-8)可得。

$$L = \text{Processor. Update Skill Library}(L, s) \quad (2-8)$$

子代理能够在技能库的历史数据中查阅先验知识，在每一轮任务执行完成后，主代理根据当前任务状态和执行结果，动态调整任务列表 List 表示为式(2-9)。

$$\text{List} = \text{MainAgent.UpdateTasks}(\text{List}, \{r_1, \dots, r_n\}) \quad (2-9)$$

该调整确保了任务执行的实时优化，适应任务环境的变化。为了保证任务执行过程的一致性，定义一致性误差度量 D 表示为式(2-10)，衡量更新前后的子任务定义偏差：

$$D(t_i, t'_i) = t_i - t'_i \leq \epsilon \quad (2-10)$$

其中 ϵ 为允许误差范围， t'_i 是更新后的信息，通过严格控制任务更新偏差确保任务目标的连贯性。假设每个子任务成功的概率为 $P_{\text{success}}(t_i)$ 表示为式(2-11)，则整体任务成功率为：

$$P_{\text{success}}(T) = \prod_{i=1}^n P_{\text{success}}(t_i) \quad (2-11)$$

通过降低每个子任务的复杂性，提升了 $P_{\text{success}}(t_i)$ ，从而提高任务的成功率。引入动态调整机制后，子任务能够实时根据新的上下文和执行结果进行判断重新调整任务策略。这一过程体现为式(2-12)：

$$t'_i = \text{Update}(t_i, \{r_1, r_2, \dots, r_{i-1}\}) \tag{2-12}$$

从而实现了任务分解策略的动态调整。整个流程如表2-5中的算法1所示，本框架成功地实现了复杂任务的动态分解与执行优化，提升了多步骤任务的完成效率。

表 2-5 多智能体任务分解算法

Tab. 2-5 Multi-agent task decomposition algorithm

算法 1 多智能体任务分解算法	
1:	Input 任务 T ，工具组件 D ，技能库 L
2:	主体算法：
3:	$List = \text{MainAgent. Decompose}(T)$ ：
4:	将任务 T 分解成若干个子任务列表。
5:	对每个任务 t_i 进行如下操作
6:	$\text{subagent}_i = \text{Agent Generator. Generate}(D, L, t_i)$
7:	生成子任务 t_i 对应的代理任务。
8:	$r_i = \text{subagent}_i. \text{Execute}(t_i, \{t_1, \dots, t_{i-1}\}, \{r_1, \dots, r_{i-1}\})$
9:	执行子任务，返回任务结果
10:	$List = \text{MainAgent. UpdateTasks}(List, \{r_1, \dots, r_n\})$
11:	动态更新任务列表
12:	$R = \text{MainAgent. Submit}(List, \{r_1, r_2, \dots, r_n\})$
13:	提交任务结果
14:	return 返回更新后的技能库 L 和任务结果 R 。
15:	Output 更新后的技能库 L ，任务结果 R 。

2.2.4 任务分解对比实验与结果分析

本章节论述的主要观点为任务分解需要将任务分解为足够细粒度的子任务，通过这种方式能够反映任务的真实完成情况。我们设置了多组对照试验证明多智能体动态任务分解框架在任务分解方面的优越性。我们针对电力系统巡检任务设计了三个子任务，分别使用本章节提出的智能体动态任务分解框架和传统LLM分解进行对比，每个子任务分别进行了100次分解，最终判断任务是否达到原子任务级别。

我们构建了巡检场景中的三个子任务进行分解，分别是1.检查电缆连接。2.检查电器面板。3.检查发电机状态。判断标准为任务分解到原子任务层级并且能够正确生成任务的执行逻辑。每一组结果我们进行了AI双盲实验，即寻找三位实验人员，三位实验人员能够得知的信息为，需要被分解的任务以及分解的结果。进行人工打分，每一个任务有100次分解结果，实验人员认为结果合理则给一分，反之结果不合理则不给分，每一个任务的满分为100分，最终结果取三位实验人员得分的均值。

为了使实验保证较高的智力上线，本实验选择了4种主流LLM基座，分别为ChatGPT-4.0（API 调用）、Zhipu-GLM4-Plus（API 调用）、DeepSeek-R1（API 调用），和Claude3.7（API 调用）。针对于三种子任务，我们给LLM相同的提示词，即“请根据数据库所提供的原子动作以及环境物品，将任务分解为多步骤可执行的细粒度子任务序列。”传统框架的分解方法为，系统通过读取JSON文本的方式调用知识库，允许LLM访问数据库中的原子动作和环境物品，一次性的进行任务分解。本章节所提框架进行任务分解的流程为算法1中所示，即每一步分解后调用新的Agent对知识库与已完成步骤执行遍历操作，生成下一步原子动作以及环境物品，直到生成最终步骤并且输出最终结果。

表 2-6 单一 LLM 任务分解（Baseline）对比动态多 Agent 任务分解（Proposed）

Tab. 2-6 Single LLM task decomposition (Baseline) vs. dynamic multi-agent task decomposition (Proposed)

ChatGPT-4.0		
任务	基线方法	动态任务分解框架（本文）
检查电缆连接	51	90
检查电器面板	55	92
检查发电机状态	59	88
总分	165	270
Zhipu-GLM4-Plus		
任务	基线方法	动态任务分解框架（本文）
检查电缆连接	53	88
检查电器面板	58	91
检查发电机状态	62	94
总分	173	273
DeepSeek-R1		
任务	基线方法	动态任务分解框架（本文）
检查电缆连接	48	89
检查电器面板	52	93
检查发电机状态	55	90
总分	155	272
Claude 3.7		
任务	基线方法	动态任务分解框架（本文）
检查电缆连接	59	94
检查电器面板	57	90
检查发电机状态	54	91
总分	170	275

从表 2-6 能够看出基于多 Agent 的动态任务分解框架在任务分解中的得分高于传统 LLM 分解方法，原因在于传统 LLM 方法虽然能够快速生成任务分解，但由于其一次性处理的特性，无法对任务进行深入细化，特别是对于多步骤任务，分解结果在精度方面有所欠缺。本章节提出的动态任务分解框架能够根据实时任务步骤和通过下文灵活调整分解结果。逐步优化的方式使每一步分解更加细致，生成更加连贯的动作序列。

2.3 本章小结

本章节针对电力系统巡检任务中面临的自然语言理解问题,提出了基于自然语言处理的任务语义解析框架,该框架通过 LangChain 架构接入了具备本地知识的向量库,能够将本地知识通过向量的形式与大型语言模型进行交互,为了解决大型语言模型难以获得自然语言中特定知识的困难,我们构建了一个电力系统巡检场景的数据库,并且在 LangChain 架构中我们使用了检索增强生成(RAG)这一技术,通过动态检索外部知识增强了大型语言模型的本地知识获取能力。实现了电力系统巡检任务的准确理解。通过电力系统知识库 LLM 对比测试,验证了在专业领域任务理解中,向量知识库的引入使模型在准确性、可靠性等关键指标上产生显著提升,有效降低 LLM 的幻觉现象。

进一步,针对于电力系统巡检任务中的复杂任务难以分解的问题,本文提出了多智能体动态任务分解框架,该框架展示了一个多步骤的复杂任务经过主代理分解为细粒度的子任务序列,然后动态的将这些子任务分配给多个子代理的过程。该框架能够将复杂的多步骤任务分解为复杂度低的更加易于执行的多个子任务。这个过程降低了任务的复杂度,增加了每个任务的可执行性,最终目的是使子任务在细粒度层面逼近原子任务,从而在行为一致性上进行对齐。针对本文提出的多智能体动态任务分解框架,通过任务分解对比实验成功的验证了基于多 Agent 的动态任务分解框架的性能高于传统单 LLM 任务分解架构。

3 巡检机器人功能自动开发与行为对齐

本章主要解决了巡检机器人个性化开发与机器人执行方面的困难。首先,针对巡检机器人个性化功能开发难以自动化的问题,本章节提出了一种基于多智能体自协作的框架。该框架通过多个大型语言模型之间的协作,共同制定机器人自动开发策略并生成可执行代码。通过调用 LLM 的 API 接口并构建不同角色的智能体能够有效地引导机器人进行二次开发,使得没有专业知识或编程经验的用户也能完成复杂的机器人开发任务。实验结果表明,与传统的团队协作方式和单一模型相比,这种多智能体自协作框架在效率和准确性上表现出显著的优势。

此外,针对巡检任务中机器人动作行为与任务策略之间的对齐问题,本章提出了一种基于余弦相似度的原子任务对齐方法。通过量化 LLM 输出的任务策略与机器人运动控制指令之间的语义相似度,本文实现了任务与指令的精准匹配避免了由于语义偏差所带来的错误传播。为了更好地评估这一对齐过程的有效性,本章还提出了一种基于语义度量的细粒度评价体系,该方法综合考虑了任务策略和原子任务之间的语义相似性,这种方法有效地保证了任务分解与指令执行的顺畅衔接,实验结果证明该方法在解决任务与指令对齐问题上具有显著效果。

3.1 基于多智能体自协作框架的巡检机器人自动开发

3.1.1 个性化功能自动开发问题

随着电力巡检任务的日益多样化,传统的基于固定功能的机器人难以面对不同场景的个性化需求。此时,就需要机器人具备高度适应性,还要求其能够根据不同的任务需求执行任务。现实世界中开发机器人功能是非常复杂的,原因在于机器人开发需要掌握多个领域的专业知识和编程技能。

近年来,LLM 因其具有强大的语义理解能力与编程能力能够将机器人开发需求进行分析并且生成合适的代码,LLM 为机器人开发带来了新的思路。尽管如此,生成满足个性化需求的代码仍然是一个巨大的挑战。近年来代码生成受到广泛关注因为成功的代码生成能够显著提升机器人开发的效率和质量代码生成的步骤在机器人的开发过程中至关重要,但面对机器人开发中的复杂任务单纯依靠大型语言模型无法有效生成正确代码,这是因为机器功能自动开发并非单纯的代码生成过程而是需要复杂的逻辑和理解能力。通常情况下,需要分析人员根据需求进行功能需求分析,此外程序员需要具备丰富的知识来应对多个需求,最重要的是代码生成后需要经过测试环节,因为并不能保证代码与硬件以及系统环境的是适配与兼容性问题。这就导致普通的大型语言模型框架难以解决机器人功能自动开发这一问题。

综上所述,机器人功能自动开发需要多个环节和步骤才能生成适配不同机器人的代码,

同时需要具备丰富的行业知识才能应对多种专业需求。所以我们需要提出一种新的方法,解决机器人开发中的复杂任务需求以及生成正确代码这一问题,满足不同任务中机器人功能的多样化需求。

3.1.2 多智能体自协作框架设计与实现

机器人功能二次开发是一项复杂且具有挑战性的任务,这项任务的难点在于对业务的深入理解以及生成正确的代码。生成准确的代码是一项极具挑战性的任务,现有方法可以利用大型语言模型来实现代码生成,但传统方法需要大量资金和数据来训练大型语言模型,这一过程对数据质量和资金要求极高,虽然 AI Agent 技术能够缓解这些难题但仍然需要大量的人力来完成代码生成任务。



图 3-1 Agent 在机器人开发过程中扮演不同角色

Fig. 3-1 Agents play different roles in the robot development process

本章节提出了一种多智能体自协作框架,通过多个 LLM 之间的协作来制定机器人自动开发策略,并生成精确的可执行代码。在机器人开发过程中我们采用多层次任务处理系统模拟专业开发团队的工作流程。该系统包含基于 LLM 的智能代理分别承担分析师、程序员和测试人员的职能分工。分析师负责深度解析用户需求并生成精确的开发指导文档,程序员依据文档内容进行程序编写,测试人员则通过用户反馈调整参数以确保代码在实际机器人环境中的适用性。每个代理专注于解决开发流程中特定环节的问题并精细化执行对应任务。

如图 3-1 所示,我们通过调用 LLM 的 API 构建了三种不同角色的智能代理,并

采用提示工程的方法使它们分别承担三种角色。在多智能体自协作框架的指导下，这三个代理共同协作完成机器人的功能二次开发。为了保证多个模型之间的有效协同，本文设计了明确的协作规则模拟真实开发团队的工作模式以确保不同模型之间的高效配合。该框架的创新性在于使不具备机器人技术或编程经验的用户仅需与系统交互即可完成复杂开发任务。相比传统人工团队或单一 LLM 在复杂任务中的局限性，该框架在效率和准确性上具有明显优势。

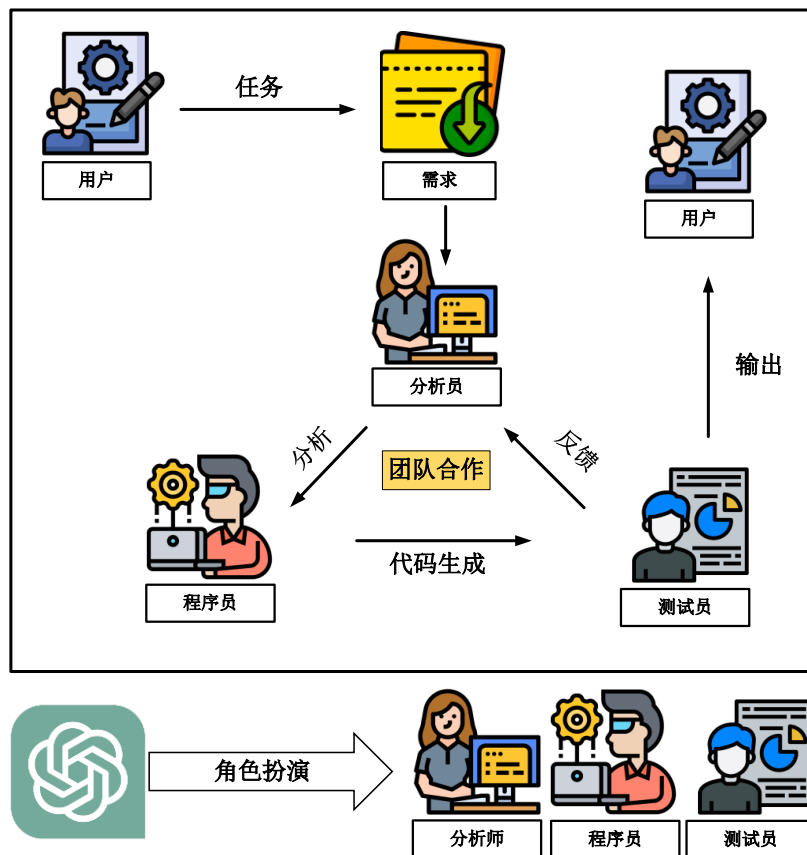


图 3-2 多智能体自协作框架示意图

Fig. 3-2 Schematic diagram of multi-agent self-collaboration framework

为了确保不同的 Agent 能够处理不同的子任务，我们为多个 Agent 输入不同的指令，将代理引导到合适的角色，并指定需要完成的子任务。我们要求智能体之间相互协作，还描述了我们期望 Agent 在协作中扮演的角色用来模拟一个真实的机器人开发团队。通过这种方式，智能体能够表现出与专业机器人开发人员相似的特征，即具备丰富的开发经验和专业知识。通过实验，我们发现直接将需求提交给单一的智能体并不能生成我们期望的结果，由于机器人开发的多个步骤与环节之间的关联性和知识领域有所不同，单一 LLM 会给出相当泛化的结果，对于具体任务而言，泛化的结果并不足以解决细分领域的问题，这是不能被接受的。本章节提出的多智能体自协作框架，能够将机器人个性化功能开发这个复杂需求细化，使每一个 Agent 尽量做一个细分领域的工作，我们通过观察得到 LLM 在仅处理一个细分领域的问题时，能够输出精细化的结果。

如图 3-2 所示，为了使多智能体自协作框架能够自动化运行，本文定义了一系列连续

步骤来确保流程清晰且逻辑严谨。分析人员首先获取用户任务的目标、性质及约束条件，并将其转化为程序员可执行的决策和指导方针。该过程的关键在于将复杂任务分解为可操作的具体子任务，使程序员能准确生成符合需求的代码。生成的代码交由测试人员根据实际机器人反馈调整参数，若多次调整仍不达标则将问题反馈给分析人员。这种闭环机制通过持续优化确保最终代码质量。

3.1.3 机器人功能自动开发实验

在本实验中，我们选择了一个基于 ROS（机器人操作系统）的四足机器人，如图 3-3 所示。我们在四足机器人上使用自协作框架开发了一个目标跟踪功能，旨在使机器人能够跟踪人类的运动，并执行避障和保持距离的功能。实验的主要目的是验证 3.1.2 章节中提出的多智能体自协作框架对机器人个性化开发的有效性。



图 3-3 用于实验的四足机器人搭载 NVIDIA Jetson Xavier NX 开发板

Fig. 3-3 The quadruped robot used for the experiment is equipped with the NVIDIA Jetson Xavier NX development board

图 3-4 展示了多智能体自协作框架机器人开发过程的一个例子，我们验证了多智能体自协作框架在四足机器人目标跟踪功能开发中的有效性。由分析人员进行任务分解和策略生成，并由程序员生成代码，并且所获得的最终代码在实际测试中由测试人员进行验证。本实验证明了本章节所提出的多智能体自协作框架在机器人开发领域的实际应用价值，为机器人开发提供了一种更有效、更精确的方法。

我们向分析人员提供了四足机器人的设备参数，这些参数包括机器人感知开发接口、机器人运动开发接口以及需要开发的功能需求，涵盖了机器人的物理特性、传感器配置和运动控制方法。机器人开发的功能需求明确包含移动跟踪、避障和距离保持等任务，这些信息为分析人员提供了必要的任务分解依据。分析人员将目标跟踪等复杂任务拆解为可执行的子任务，并制定相应的实现策略，明确机器人行为规范以供程序员实施。

程序员依据分析人员提供的策略编写了代码，实现了机器人的运动控制、避障和距离维护功能。Agent 的角色指令帮助程序员理解分析人员的策略并生成符合功能需求的代码。测试人员在真实四足机器人平台上评估代码性能，模拟人物移动、避障和距离保持场景以验证代码是否满足需求。根据实验结果测试人员调整参数确保机器人在现实环境中正常运行，测试过程需要测试人员与用户共同完成。

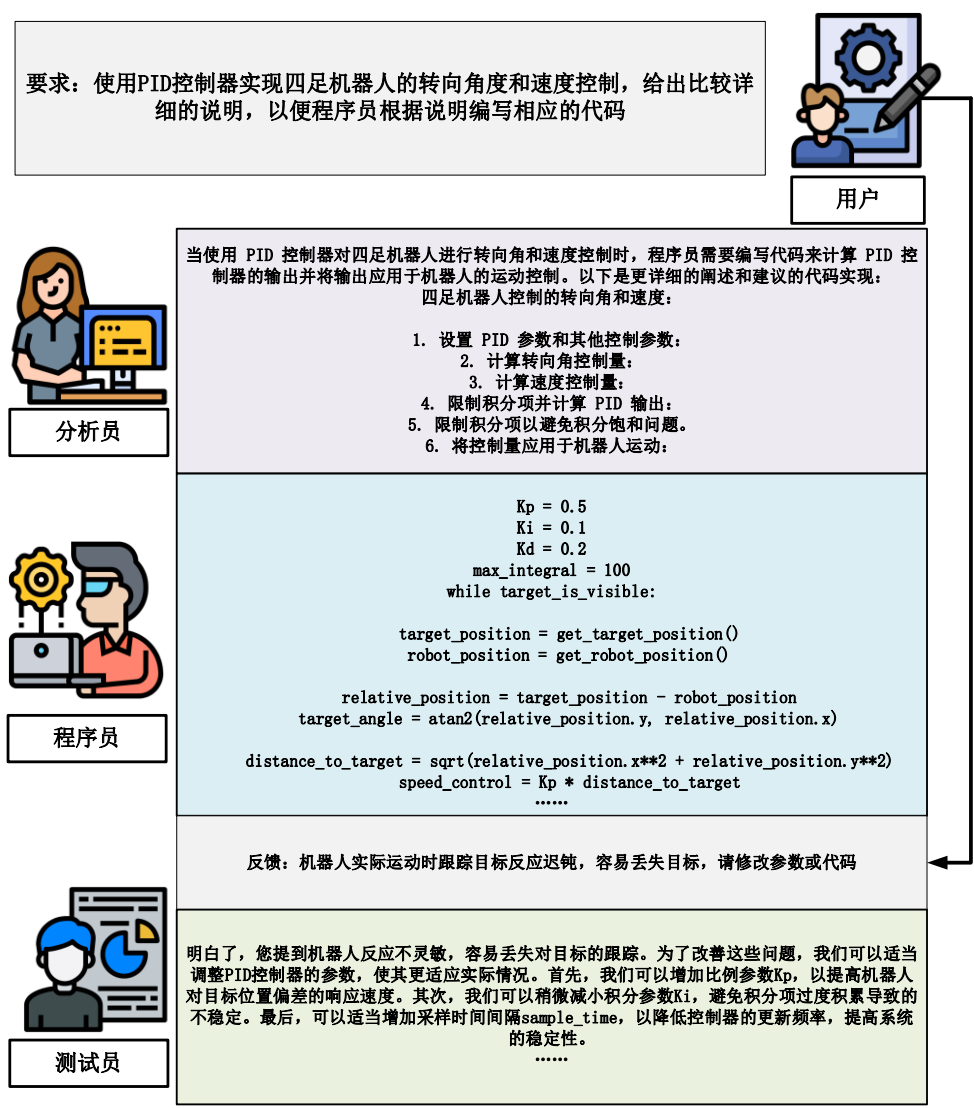


图 3-4 多智能体自协作框架机器人开发示例

Fig. 3-4 Multi-agent self-collaboration framework robot development example

如图 3-5 所示，实验阶段的应用验证了该协同开发框架的有效性，成功实现了四足机器人跟踪人体运动轨迹的功能。实验结果证明该方法在实际机器人应用中具备可行性，同时为满足个性化需求的自动化功能开发提供了新的参考方案。

通常情况下，实现四足机器人的自动跟随功能需要专业的团队协作以及耗费大量的时间和资源。然而，通过我们提出的多智能体自协作框架能使没有机器人开发经验的人员成功完成复杂任务，证明该框架有效降低了开发门槛并提升了效率。实验表明单一 LLM 驱动的智能体无法独立完成此类任务，而采用多智能体协作框架则实现了机器人复杂功能的开发。实验成功实现了四足机器人跟随功能，在现实应用中表现优异。该方法显著提升了开发效率并为单个 LLM 模型难以处理的复杂任务提供了创新解决方案。实验验证了多智能体自协作框架的有效性，切实满足了用户需求。



图 3-5 四足机器人执行目标跟踪任务

Fig. 3-5 Quadruped robot performs target tracking task

3.2 巡检机器人原子任务对齐与行为执行

3.2.1 任务与动作对齐问题

大型语言模型控制机器人行为是近年来的热门话题,但是现有方法通常依赖大型语言模型调用固定格式的硬编码,这种方式的弊端是大型语言模型在调用硬编码时被严格限制输出形式,以确保系统能够准确索引到需要执行的运动控制函数,在本质上这种调用方法和传统基于规则的运动控制调用方式接近,传统的调用方式依赖于认为判断动作序列再进行指令下达。核心问题是机器人运动模块的调用方式为固定编码格式,大型语言模型想要成功调用运动功能,就必须和传统方法类似,输出能够精准指向固定编码的数据格式。但是从另一个方面考虑,这种方式并没有解决大型语言模型幻觉所带来的输出偏差。

实际上,复杂任务分解为可执行的子任务这一过程是困难的,复杂的任务包含多个环节,这些环节在宏观上被认为是一项任务,想要细化到执行层面,每个环节都需要被拆解为可执行的小步骤,并且这一过程需要保证任务语义不变。例如,在一个具体的任务场景中,若任务为“让小明为小红倒水”,该任务可分解为寻找杯子、取水、递送等子任务。虽然每个步骤看似简单,但需要机器人精确执行。现有方法中,由于 LLM 存在幻觉问题,难以生成标准格式的运动指令或准确调用运动模块完成相应动作。

LLM 生成的控制指令需人工转换为可执行机器人指令,这一过程增加了时间成本并降低系统效率。在复杂任务中,LLM 直接输出的代码往往无法准确匹配机器人动作需求,导致执行偏差和性能下降。复杂任务要求机器人动作指令与任务需求精确匹配。以“让小

明为小红倒水"为例,每个动作既要满足任务目标又要符合机器人操作规范。LLM 生成的非结构化指令与机器人可执行动作之间存在适配难题,这是当前需要解决的关键问题。

3.2.2 基于余弦相似度的原子任务对齐方法

由于 LLM 的任务规划的策略与机器人的运动控制指令之间存在差异,通过自然语言策略调用 LLM 控制机器人的运动是困难的。这是由于 LLM 输出的任务规划内容在特定描述上存在变化,这导致了机器人需要执行的具体动作指令的调用出现模糊性错误。正如 3.2.1 所描述的问题,调用机器人运动控制函数通常是一个固定的语法结构或者硬编码,LLM 输出的内容可能会出现一种情况,那就是 LLM 输出一个与机器人运动控制指令高度相似的文本,但这并不能正确的调用机器人的运动控制函数。

为了解决这个问题,我们采用了语义相似性来关联任务规划与机器人运动控制指令。来自 LLM 的任务描述与机器人控制指令本质上是两段语义相似的文本,我们通常使用余弦相似度来衡量这两段文本的相似性。语义之间的相似性计算可以通过将文本嵌入特征空间中,然后在特征空间中执行余弦相似度计算来实现。具体来说,本文使用余弦相似度来计算语义相似性。

表 3-1 语义相似度算法流程

Tab. 3-1 Semantic Similarity Algorithm Process

算法 2 语义相似度算法流程

- 1: **Input:** text message 输入文本信息
- 2: 量化:
- 3: $\mathbf{v}_A = (w_{A1}, w_{A2}, \dots, w_{An})$ 文本向量化和位置信息
 $\mathbf{v}_B = (w_{B1}, w_{B2}, \dots, w_{Bm})$
- 4: 标准化:
- 5: $\mathbf{e}_A = \frac{\mathbf{v}_A}{|\mathbf{v}_A|}$ 向量标准化
- 6: $\mathbf{e}_B = \frac{\mathbf{v}_B}{|\mathbf{v}_B|}$ 向量标准化
- 7: 余弦相似度计算
- 8: $\text{Similarity}(\mathbf{e}_A, \mathbf{e}_B) = \frac{\mathbf{u}_A \cdot \mathbf{u}_B}{|\mathbf{v}_A| \cdot |\mathbf{v}_B|}$ 余弦相似度计算公式
- 9: $\text{Similarity}(\mathbf{e}_A, \mathbf{e}_B) = \frac{\sum_{i=1}^n w_{Ai} \cdot w_{Bi}}{\sqrt{\sum_{i=1}^n w_{Ai}^2} \cdot \sqrt{\sum_{i=1}^m w_{Bi}^2}}$ 计算文本 A 和文本 B 之间的相似度
- 10: 相似度判断
- 11: **if:** $\text{Similarity}(\mathbf{u}_A, \mathbf{u}_B) > \text{threshold}$: 如果相似度大于阈值,返回相似结果
- 12: **else:**
- 13: 如果相似度不大于阈值,返回不相似结果

首先我们对 LLM 生成的文本进行向量化操作,假设大型语言模型输出的任务规划文本为 A,而机器人控制命令是文本 B。我们首先将它们表示为向量 $\mathbf{v}_A$ 和 $\mathbf{v}_B$,具体计算如式 (3-1)所示,通过词嵌入的方法将文本进行向量化,这些向量表示文本在向量空间中的位置。

$$\begin{aligned}\mathbf{v}_A &= (\mathbf{w}_{A1}, \mathbf{w}_{A2}, \dots, \mathbf{w}_{An}) \\ \mathbf{v}_B &= (\mathbf{w}_{B1}, \mathbf{w}_{B2}, \dots, \mathbf{w}_{Bm})\end{aligned}\quad (3-1)$$

其中 n 和 m 分别表示文本A和B中词汇的数量， w_{Ai} 和 w_{Bi} 是相对应的词汇权重。为了消除文本长度的影响，可以对文本向量进行归一化。归一化后的向量表示为 $\mathbf{e}_A$ 和 $\mathbf{e}_B$ ，具体计算如式(3-2)和(3-3)所示。

$$\mathbf{e}_A = \frac{\mathbf{v}_A}{|\mathbf{v}_A|} \quad (3-2)$$

$$\mathbf{e}_B = \frac{\mathbf{v}_B}{|\mathbf{v}_B|} \quad (3-3)$$

其中， $\mathbf{e}_A$ 和 $\mathbf{e}_B$ 表示归一化的文本向量，而 $|\mathbf{v}_A|$ 和 $|\mathbf{v}_B|$ 分别表示向量 $\mathbf{v}_A$ 和 $\mathbf{v}_B$ 的范数。归一化操作本质上是通过将向量中的每个元素除以该向量的范数来进行的。这样做的效果是，无论原始文本向量的长度如何，归一化后的文本向量都具有单位长度。因此，在计算文本向量之间的距离或比较相似度时，它们不会受到文本长度的影响，从而更好地比较文本相似度。余弦相似度是通过计算两个向量的点积并除以这两个向量范数的乘积来衡量的。余弦相似度的公式如式(3-4)和式(3-5)所示：

$$\text{Similarity}(\mathbf{e}_A, \mathbf{e}_B) = \frac{\mathbf{u}_A \cdot \mathbf{u}_B}{|\mathbf{v}_A| \cdot |\mathbf{v}_B|} \quad (3-4)$$

$$\text{Similarity}(\mathbf{e}_A, \mathbf{e}_B) = \frac{\sum_{i=1}^n w_{Ai} \cdot w_{Bi}}{\sqrt{\sum_{i=1}^n w_{Ai}^2} \cdot \sqrt{\sum_{i=1}^m w_{Bi}^2}} \quad (3-5)$$

通过文本相似度匹配，我们能够在任务和指令之间执行对齐。具体步骤见表 3-1 中的算法 2。在生成任务计划的过程中，可能发生的一种情况是，大型语言模型结合环境信息生成的任务不正确，该错误导致任务的语义不能与指令的语义对齐，导致无法调用机器人控制命令。对于不能在语义级别对齐的文本，我们将大型语言模型的任务输出重新输入模型，进行新一轮分解重新生成任务序列，这个过程一直持续到 LLM 的输出与机器人控制指令的余弦相似度高于某个阈值，即系统判定两段文本在语义层面接近，可以进行对齐此时整个流程停止。

3.2.3 基于语义度量的细粒度评价体系

任务对齐操作是将复杂任务分解为细粒度任务后与机器人控制指令进行对齐。从评估基准的角度来看，目前的评估标准无法准确衡量 Agent 在任务分解过程中是否表现良好。现有评估指标通常无法准确衡量 Agent 任务分解所需的精细程度。复杂多步骤任务需要多层次分解才能转化为可执行的原子任务，大模型驱动的代理可能在部分环节表现良好，但往往难以完成最终的任务分解目标。

在评估 Agent 的多步骤任务分解能力时，需要同时考察中间步骤和最终原子任务的分解效果。仅依据最终结果难以全面反映代理的真实能力，必须建立包含中间环节的评估体

系才能准确衡量其任务分解水平。如果仅通过最终结果来判断任务分解的成功与否,那么评价代理能力的相关性将大打折扣。为了全面评估代理的能力,我们提出了一个基于语义度量的细粒度评价体系,该体系从两个层面评估代理的表现,任务分解的有效性和传播错误的发生情况。区别于传统固定答案的基准测试,我们的评估方法基于代理在模拟环境中执行任务时的动态表现进行评分。针对多步骤任务分解的特点,设计了包含任务分解有效性和信息传递准确性的分层评分体系,能够适应无固定流程的多解情况。为了全面评估自适应代理在动态多步骤任务分解中的表现,我们设计了一个分层的综合评分体系,目的是量化代理在任务分解过程中的有效性以及信息传播的准确性。这个评分体系包含两个主要部分。

1) 信息传播准确性: 通过任务分解总体语义相似度 (Overall Task Decomposition Similarity Score, TDS) 评分来衡量信息传播准确性, 基于输入任务与生成的任务分解结果之间的余弦相似度, 用于量测任务文本的整体语义, 能够反映出任务分解的过程中整体任务的语义是否发生变化。

设输入任务 $T \in R^m$ 表示为一个 m 维向量, 任务分解结果 $\{d_1, d_2, \dots, d_n\}$ 由 n 个子任务组成, 且每个子任务 $d_i \in R^m$ 。总体语义相似度可定义为式(3-6):

$$TDS = S(T, D) = \frac{T \cdot D}{|T| \cdot |D|} \quad (3-6)$$

其中, D 是所有任务分解结果的平均向量, 如式(3-7)所示, 也就是将子任务序列向量平均化, 定义为:

$$D = \frac{1}{n} \sum_{i=1}^n d_i \quad (3-7)$$

若 $TDS < 0.8$, 则认为存在显著的语义偏差, 表示信息传播错误的发生。若 TDS 值小于 0.8, 表示输入任务的语义信息在分解过程中产生了显著的变化。这是信息传播错误的一种指示, 突显了代理在任务分解时的失误或不足。因真实世界中的任务往往并非单一固定过程, TDS 提供了一种动态评估机制, 对不同上下文中的任务结果给予反馈, 从而促使代理自我调整与学习。

2) 任务分解有效性: 通过子任务分解有效性评分 (Subtask Decomposition Validity Score, SDS) 来衡量任务分解有效性, SDS 可以表示为式(3-8)。计算每个子任务与其对应原子任务间的余弦相似度来评估分解的准确性。能够反映出任务分解是否彻底, 最终生成的子任务是否与原子任务对齐。对于每一子任务 d_i 及其对应原子任务 a_i , 定义子任务有效性评分为:

$$SDS = \frac{1}{n} \sum_{i=1}^n S(d_i, a_i) = \frac{1}{n} \sum_{i=1}^n \frac{d_i \cdot a_i}{|d_i| \cdot |a_i|} \quad (3-8)$$

若存在 " $SDS < 0.8$ ", 则表示该子任务 d_i 的分解为失败。通过余弦相似度的计算, SDS 提供了对每个子任务的精细评估, 帮助研究者了解每个子任务在满足整体任务目标方面的有效性。该指标通过余弦相似度计算, 可精确评估各子任务对整体目标的贡献度。0.8 的阈值能有效定位问题子任务, 为后续优化提供依据。综合评分 C 通过加权的方式, 将总

体评分与子任务评分结合起来，如式(3-9)所示。

$$C = \alpha \cdot TDS + \beta \cdot SDS \quad (3-9)$$

其中，权重 α 和 β 满足 $\alpha + \beta = 1$ 。在本评分体系中，我们设定 $\alpha = 0.5$ 和 $\beta = 0.5$ ，从而可表示为式(3-10)，最终评分 C 的范围为0到100分，最终评分可以表示为式(3-10)：

$$C = 0.5 \cdot TDS + 0.5 \cdot SDS \quad (3-10)$$

本章节提出的细粒度评价体系的评估顺序如下，首先计算总体相似度评分 TDS，以分析输入任务与生成分解结果之间的语义一致性，然后评估每个子任务的有效性 SDS，通过计算子任务与原子任务之间的相似度。通过将 TDS 和 SDS 综合形成评分 C，评分 C 能够全面反映代理在任务分解和执行中的整体表现，为评估提供更加精准的量化结果。该评分体系通过整合信息传播准确性和子任务分解效果，建立了一套适用于复杂任务的细粒度评估标准，能够全面衡量自适应代理处理多样化非结构化问题的实际性能。

在任务分解过程中，确保子任务序列与原始任务以及子任务与原子任务之间的语义一致性，是实现高效任务分解的关键所在。如图 3-6 所示，本框架基于余弦相似度构建了多层语义相似度判定框架，该框架任务分解部分沿用了本文 2.2.3 章节的多智能体动态任务分解框架。该框架在任务分解后引入语义相似度评估模块，通过量化子任务语义一致性动态优化分解策略，同时识别并纠正潜在的语义偏差和传播错误。

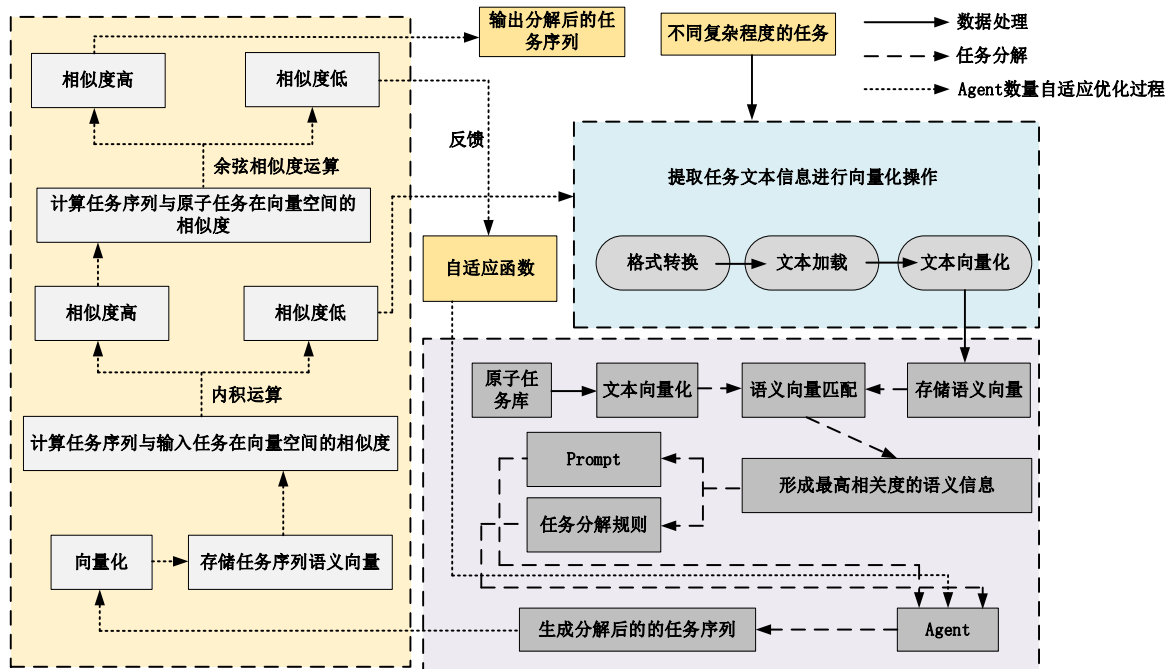


图 3-6 多层语义相似度判定结构框架图

Fig. 3-6 Multi-layer semantic similarity determination structure framework diagram

图 3.6 描述了完整的任务文本输入以及向量化的过程，其中通过子任务与输入任务的余弦相似度判定来判断是否在信息传播过程中出现传播错误，和子任务与原子任务的余弦相似度判断任务分解是否彻底。任务文本首先经过数据处理，数据处理是非常重要的过程，包括格式转换以及文本向量化，首先对任务文本信息进行格式转换与向量化操作，将语义

信息映射到高维向量空间进行语义向量的存储,与原子任务库的语义向量进行匹配,形成相关度最高的语义信息。之后,通过 **prompt** 提示工程结合任务分解规则对上述最高相关度语义信息进行文本重构,将文本信息传输至具有层级结构的 **Agent** 模块,对文本信息进行任务分解,生成一系列任务分解后的子任务序列。

将分解后的任务序列进行向量化处理生成语义向量,计算任务序列与输入任务在向量空间的相似度,通过余弦相似度运算得到两种分类结果,高相似度以及低相似度。高相似度的任务序列向量进入下一个运算模块,低相似度则返回初始位置重新进行任务分解。

最后再将任务序列与原子任务进行相似度计算,计算向量之间的余弦相似度作为判断依据,如果余弦相似度高,表示任务分解序列与原子任务库有较高相似度,判定任务分解成功。如果余弦相似度低,表示任务分解序列与原子任务相似度低,此时反馈信息传输至自适应函数,自适应函数通过重新对这个子任务分配 **Agent** 进行更加彻底的任务分解,实现任务分解序列与原子任务具有较高匹配度,具体见表 3-2 中的算法 3。

多层语义相似度判定框架允许使用定制化的任务分解方案,这一点和已有的 **Agent** 不同,我们的个性化任务分解是通过技能库中增加组件来实现的,该技能库在 2.2 章节给出了具体的描述,具体来讲是通过增加技能库中的信息和增加 LLM 的 **prompt** 来实现的。有两个模块共同构建了这个功能,第一点是我们的技能库能够手动的增加现有环境中的物品以及更加复杂的组合动作,增加了代理在任务分解过程中的适应性。第二点,我们的技能库能够储存历史成功执行任务分解的案例,通过日志的形式保存成 JSON 文件,也能够人为的增加符合个人偏好的执行步骤,通过提示词我们能够使代理在分解的过程中对技能库中的知识进行参考,生成符合个人偏好的任务分解步骤。

表 3-2 多层语义相似度判定算法

Tab. 3-2 Multi-layer semantic similarity determination algorithm

算法 3 多层语义相似度判定算法

- 1: **Input:** Task T 输入任务集合T
- 2: **save:**
- 3: $\{\phi(d_1), \phi(d_2), \dots, \phi(d_n)\}$ 子任务文本向量化和储存
- 4: 总体语义相似度:
- 5: $TDS = S(T, D) = \frac{T \cdot D}{|T| \cdot |D|}$ 计算任务与子任务之间的余弦相似度
- 6: 任务分解有效性:
- 7: $SDS = \frac{1}{n} \sum_{i=1}^n S(d_i, a_i) = \frac{1}{n} \sum_{i=1}^n \frac{d_i \cdot a_i}{|d_i| \cdot |a_i|}$ 计算任务与原任务之间的相似度
- 8: $Consistency(d_i, \{d_1, \dots, d_{i-1}\}) = \max_{j < i} \frac{d_i \cdot a_i}{|d_i| \cdot |a_i|}$ 判断语义一致性
- 9: $d_i' = Adjust(d_i, \{r_1, \dots, r_{i-1}\}, T)$ 动态调整任务的相似度与偏差
- 10: 在任务分析过程中, 确保子任务语义向量能够被正确保存并达到预期一致性
- 11: **return:** $S(TDS, SDS) \geq 0.8$ 返回最终分析结果, 判断是否通过相似度判定

个性化任务分解通过增强技能库功能和优化提示机制实现动态任务适应,其核心包含两个关键模块:一是支持手动添加环境物品和组合动作的动态技能库,使代理能够应对多样化任务需求;二是基于历史分解案例和用户偏好的参考系统,帮助生成更符合用户习惯的任务方案。技能库扩展功能允许用户上传环境物品数据和配置组合动作,使代理能自动调用这些信息生成环境适配的分解策略。

动态技能库作为个性化任务分解的核心组件,支持用户根据实际需求添加环境物品信息和组合动作指令。通过系统接口可上传设备清单和配置特定场景的操作序列,使代理能自动调用这些数据生成符合环境特征的任务分解方案。

表 3-3 个性化任务分解算法

Tab. 3-3 Personalized task decomposition algorithm

算法 4 Customized proxy generation process

```

1: Input: Task  $T$ , skill library  $L$ , Prompt
2: Output: 任务分解方案  $D_{\text{custom}}$ , 执行结果集合  $R_{\text{sub}}$ 
3:  $\phi(T) \leftarrow \text{Vectorization}(T)$  提取输入任务的语义向量
4:  $\text{Prompt} \leftarrow \text{GeneratePrompt}(\phi(T), L)$  生成初始提示词
5:  $D = \{d_1, d_2, \dots, d_n\}$  生成子任务序列
6:  $A = \{\}$  初始化代理列表
7:  $\phi(d_i) \leftarrow \text{Vectorization}(d_i)$  计算子任务的语义向量
8:  $A_i \leftarrow \text{GenerateAgent}(\phi(d_i), L)$  该代理是与技能库结合起来动态生成的
9:  $R_i \leftarrow A_i.\text{Execute}(d_i)$  执行子任务并生成结果
10:  $S(R_i, \phi(T)) = \frac{\phi(R_i) \cdot \phi(T)}{\|\phi(R_i)\| \cdot \|\phi(T)\|}$  对于每个执行结果, 计算其与输入任务的余弦相似度
11:  $d_i' \leftarrow \text{Adjust}(d_i, R_i, \phi(T))$ 
    如果  $S(R_i, \phi(T)) < \delta$  它会触发子任务调整过程, 并调整子任务定义
12:  $A_i \leftarrow \text{GenerateAgent}(\phi(d_i'), L)$  重新分配代理
13:  $R_i \leftarrow A_i.\text{Execute}(d_i')$  再次执行子任务
14:  $s_i \leftarrow \text{SummarizeProcess}(R_i)$   $L \leftarrow \text{UpdateSkillLibrary}(L, s_i)$ 
    如果子任务执行成功, 则将任务流程摘要存储到技能库中
15: return:
16:  $D_{\text{custom}} = D$ ,  $R_{\text{sub}} = \{R_1, R_2, \dots, R_n\}$  返回分解后的任务序列和执行结果

```

技能库可以存储历史任务分解案例,将代理成功完成的任务分解过程以 JSON 日志形式保存。日志内容包括详细记录子任务序列和执行过程的任务分解步骤、任务分解过程中生成的关键语义一致性数据以及用户手动添加的个性化任务分解步骤。通过调用这些历史记录,代理在生成新任务分解方案时能够参考过往执行步骤,从而更符合用户习惯和偏好。

提示词优化模块是个性化任务分解的核心组件,通过动态调整 LLM 的提示词设计使代理在分解任务时有效利用技能库内容并适应用户需求。代理基于任务语义向量从技能库检索相似历史记录或动作信息,动态调整提示词以融入关键内容,包括历史成功案例的分解步骤、用户偏好的执行顺序以及当前任务环境中的相关物品和动作信息。

将技能库信息整合到提示词中使代理能够生成更贴合用户需求的任务分解方案,同时根据环境变化自动调用适配当前场景的动作单元。提示词会优先参考用户偏好的执行步骤,

生成更加个性化的分解方案。在任务分解过程中，代理可以通过多层语义相似度判定结构（见 3.2.3 章节图 3-6）动态检测分解结果的语义一致性，并根据反馈调整提示词以优化分解策略。具体算法见表 3-3 中的算法 4。

3.2.4 语义度量基线实验和消融实验

本节实验验证自适应多层语义相似度判定框架的有效性，并测试细粒度评估基准在复杂任务分解中的表现。实验在自适应多层大模型规划框架下展开，通过基线对比和消融分析探究多智能体规划框架的优势。为检验语义度量细粒度评价体系的有效性，采用 3.2 章节构建的技能库，该数据集包含定义最小分解粒度的原子任务库、记录场景物品信息的环境物品库以及通过标准化提示工程提供分解指导的动态分解提示库。

1) 基线对比实验设置

本章节通过基线实验和对照实验验证自适应多层大模型结构规划框架的任务分解效果。基线实验采用主流 LLM 直接进行任务分解，对照实验在相同模型基础上引入语义度量细粒度评价体系，通过对比两组实验结果评估该框架在提升任务分解准确性和可执行性方面的实际效果。

1.1) 基线实验配置

基线实验中，任务分解直接依赖以下模型的能力 ChatGPT-3.5（API 调用）、ChatGPT-4.0（API 调用）、Gemini（API 调用）、LLaMA 3-405B（API 调用）、Zhipu-GLM4-Plus（API 调用）、Zhipu-GLM4-Flash（API 调用）、DeepSeek-R1（API 调用）、和 Claude3.7（API 调用）、ChatGLM3-6B（离线部署）、ChatGLM13B（离线部署）。

为了展示不同 LLM 的能力，我们选择了市面上主流的基座大型语言模型，同时兼顾一些离线开发需求，我们也同时测试了两个离线部署的规模更小的 LLM。在基线实验中，LLM 直接接收任务文本输入并尝试将其分解为子任务序列，整个过程不依赖额外框架辅助，仅依靠模型自身推理能力完成任务分解。实验设计涵盖电力系统巡检场景中三种不同复杂度的任务

a.简单任务如“检查变压器温度是否正常、油位是否符合要求并记录结果”或“确认断路器分合状态是否正常并记录外观损坏情况”，用于测试 LLM 在单一目标任务上的分解能力；

b.中等复杂任务如“全面检查断路器触头磨损情况、灭弧室状态并进行必要校准”，要求分解为多个并行步骤，评估 LLM 在多步骤任务中的表现；

c.高复杂任务如“制定并执行变电站预防性维护计划，包括设备清洁、紧固检查、润滑维护并评估效果”，涉及多层次结构，测试 LLM 对高度复杂任务的分解和适应能力。

1.2) 对照实验配置

本章提出的多层语义相似度判定结构框架基于相同的大型语言模型，接入该框架后，通过动态代理生成模块与语义相似度判定模块优化任务分解过程。具体配置为，采用动态规划策略逐步分解复杂任务，将高复杂任务拆解成多个子任务，进一步细化到原子任务级

别。结合余弦相似度判断任务分解中的语义偏差，确保子任务语义与原始任务一致，避免信息传递中的错误。系统为每个子任务动态生成专用子代理执行，根据实时反馈调整代理数量与分工，以适应任务变化。引入多层语义相似度判断框架后，模型在处理复杂任务时的分解能力显著提升，更好地满足高复杂度场景的需求。

1.3) 实验评估指标

实验评估采用以下量化指标，该指标在 3.2.3 章节基于语义度量的细粒度评价体系中进行了详细的描述，这一评价体系全面衡量任务分解的语义一致性和执行有效性，衡量模型生成的任务分解序列与输入任务语义的一致性，基于任务向量余弦相似度计算如式(3-11)所示：

$$TDS = \frac{\mathbf{v}_{input} \cdot \mathbf{V}_{output}}{|\mathbf{v}_{input}| \cdot |\mathbf{V}_{output}|} \quad (3-11)$$

若 TDS 值小于 0.8,则认为任务分解过程中存在显著语义偏差。评估生成的子任务是否与原子任务对齐，同样基于余弦相似度：

$$SDS = \frac{1}{n} \sum_{i=1}^n \frac{\mathbf{v}_i \cdot \mathbf{a}_i}{|\mathbf{v}_i| \cdot |\mathbf{a}_i|} \quad (3-12)$$

若某子任务的 SDS 值低于 0.8,则视为分解失败，SDS 的计算如式(3-12)所示。TDS 和 SDS 按权重组合计算，反映总体分解效果如式(3-13)所示：

$$C = \alpha \cdot TDS + \beta \cdot SDS \quad (3-13)$$

其中，权重参数设定为 $\alpha=0.5, \beta=0.5$ 。

2) 基线对比实验结果分析

本小节分析各大模型在基线实验和对照实验中的表现，比较其在任务分解语义一致性（TDS）、子任务有效性（SDS）以及综合评分（C）上的差异。

2.1) 基线实验结果

在基线实验中，不同模型直接生成任务分解结果，各模型在简单任务、中等复杂任务和高复杂任务的评分如表 3-4 所示。

从表中可以看出：

简单任务：所有模型在简单任务中表现较好，得分均在 80 分左右。ChatGPT-4.0 表现最佳，得分为 88.7。

中等复杂任务：模型在任务分解的多步骤和层次结构处理上有所下降，得分集中在 70-80 分之间，ChatGPT-4.0 依然领先，Zhipu-GLM4-Plus 和 Zhipu-GLM4-Flash 次之。

高复杂任务：高复杂任务对语义一致性和多步骤规划的要求更高，大多数模型的评分明显下降，平均得分低于 70。

并且能够得出结论，性能与大模型本身的能力具有很大的关系，我们测试离线模型的得分普遍较低，这是由于离线模型参数量小且本身性能不足的原因造成的。

表 3-4 基线实验评分结果
Tab. 3-4 Baseline Experiment Scoring Results

闭源大模型调用 API			
模型	简单任务	中等任务	复杂任务
ChatGPT-3.5	82.1	75.4	68.3
ChatGPT-4.0	88.7	81.3	72.8
Gemini	85.6	78.2	69.7
LLaMA3-405B	81.3	73.9	67.2
Zhipu-GLM4-Plus	87.4	80.6	71.5
Zhipu-GLM4-Flash	86.8	79.9	70.4
DeepSeek-R1	88.4	80.5	73.2
Claude3.7	88.5	81.5	72.5
离线部署的小型模型			
Model	简单任务	中等任务	复杂任务
ChatGLM3-6B	68.9	62.3	56.1
ChatGLM13B	70.2	64.5	57.5

2.2) 对照实验结果

表 3-5 对照实验评分结果
Tab. 3-5 Control experiment scoring results

闭源大模型调用 API + 本文方法			
模型	简单任务	中等任务	复杂任务
ChatGPT-3.5	90.4 (+8.3)	84.1 (+8.7)	77.6 (+9.3)
ChatGPT-4.0	94.2 (+5.5)	89.6 (+8.3)	84.7 (+11.9)
Gemini	92.1 (+6.5)	86.8 (+8.6)	79.2 (+9.5)
LLaMA 3-405B	89.7 (+8.4)	83.2 (+9.3)	75.8 (+8.6)
Zhipu-GLM4-Plus	93.8 (+6.4)	88.7 (+8.1)	83.4 (+11.9)
Zhipu-GLM4-Flash	92.9 (+6.1)	87.5 (+7.6)	81.9 (+11.5)
DeepSeek-R1	94.4 (+6.0)	90.1 (+9.6)	84.1 (+10.9)
Claude3.7	93.8 (+5.3)	89.5 (+8.0)	83.8 (+10.6)
离线部署的小型模型+ 本文方法			
模型	简单任务	中等任务	复杂任务
ChatGLM3-6B	78.3 (+9.4)	78.3 (+9.4)	64.5 (+8.4)
ChatGLM13B	80.1 (+9.9)	73.6 (+9.1)	66.9 (+9.4)

在对照实验中，将各模型接入本章节提出的多层语义相似度判定结构框架，通过动态

代理生成和语义相似度判定优化任务分解过程。优化后的评分如表 3-5 所示。

实验结果显示，简单任务得分普遍提升 5-10 分，表明语义相似度判定模块有效增强了任务分解精度。中等复杂任务在多智能体框架支持下获得 8-10 分的提升，验证了该框架处理多步骤分解的有效性。高复杂任务改善最为显著，通过动态代理分工和多轮优化，ChatGPT-4.0 得分从 72.8 升至 84.7，Zhipu-GLM4-Plus 从 71.5 提升至 83.4，充分体现了框架在复杂场景下的优势。

2.3) 基线实验结果分析

对比结果表明，接入多智能体任务分解框架后各模型的任务分解性能均有明显改善，尤其在中高复杂度任务中表现突出。该框架整合动态分解策略、语义一致性校验和实时反馈机制，成功克服了单一模型在独立分解时产生的语义偏移和错误传播的问题。

从 TDS 和 SDS 的评分变化来看，多智能体任务分解框架在保持语义一致性和提高任务分解程度的方面效果显著。例如，Zhipu-GLM4-Plus 在高复杂任务中的评分提升了 11.9%。在基线实验中，高复杂任务对所有模型的评分都有显著影响，尤其是离线部署的 ChatGLM3-6B 和 ChatGLM-13B 模型。离线模型评分较低主要受限于其固有理解能力，但在引入多智能体框架后性能显著提升，证明该框架能有效增强模型处理复杂任务的能力。

3) 消融实验设置

为了深入探讨本文提出的语义度量评价体系中各个模块对任务分解的作用，消融实验通过逐步移除框架功能模块并对比性能差异，证实语义相似度判断和多智能体协作机制对任务分解效果具有决定性影响。

3.1) 消融实验设计

如图 3-7 所示，消融实验基于以下四种配置进行：

完整框架：包括多智能体框架、语义相似度判断模块以及大模型任务分解能力，作为对比基准。

移除多智能体框架：仅保留语义相似度判断模块和大模型任务分解能力，直接由大模型输出任务分解序列，不进行多智能体分工和动态调整。

移除语义相似度判断模块：仅保留多智能体框架和大模型任务分解能力，任务分解过程中不检测和修正语义偏差，仅依赖动态代理和大模型分解能力完成任务。

单独大模型：完全移除多智能体框架和语义相似度判断模块，仅依赖大模型直接完成任务分解，作为对比实验的基线。

3.2) 模型选择

消融实验选择了以下具有代表性的模型：ChatGPT-3.5（API 调用）、ChatGPT-4.0（API 调用）、Zhipu-GLM4-Plus（API 调用）、DeepSeek-R1（API 调用）。在基线对比实验中，这些模型已经表现出较强的任务分解能力，消融实验的设计旨在分析框架的适应性和各模块的贡献。

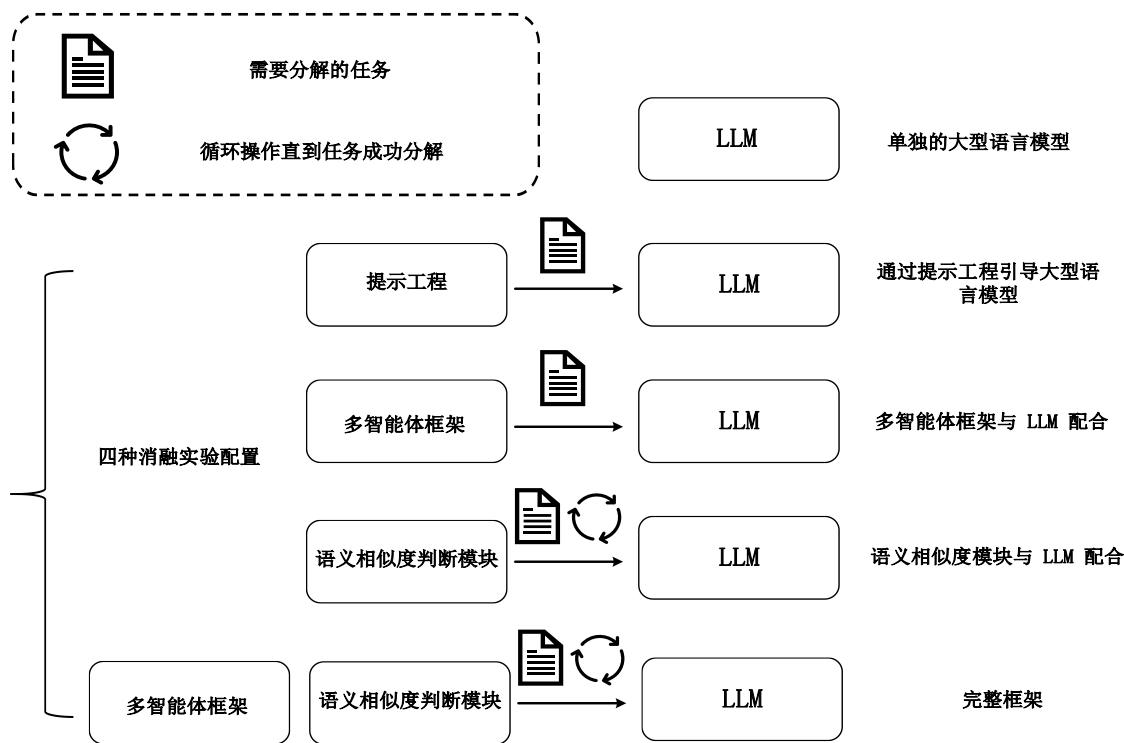


图 3-7 消融实验的四种不同配置

Fig. 3-7 Four different experimental configurations for ablation experiments

4) 消融实验结果分析以及模块贡献

本节通过分析消融实验结果，探讨多智能体框架和语义相似度判断模块在任务分解中的实际贡献。实验结果如表 3-6 所示。

4.1) 消融实验结果分析

在完整框架配置下，所有模型在简单任务、中等复杂任务和高复杂任务上的评分均达到最高。例如 DeepSeek-R1 在高复杂任务中的评分达到 84.9，同一级别的 ChatGPT-4.0 在高复杂任务中的评分达到 84.7，这种显著提升表明，完整的框架通过动态分解和语义调整，能够使 LLM 输出符合语义的任务分解结果。

消融实验结果显示，移除多智能体框架导致模型性能显著下降，Zhipu-GLM4-Plus 在高复杂任务得分从 83.4 降至 75.6，证明该框架在降低分解难度和协调代理方面具有关键作用。移除语义相似度模块主要影响高复杂任务，ChatGPT-4.0 得分从 84.7 降至 81.5，表明该模块有效维护语义一致性并抑制错误传播。

单纯依赖大模型完成任务分解时，所有评分均为最低。例如，ChatGPT-3.5 在高复杂任务中的得分仅为 68.3。这显示出，大模型单独任务分解能力在面对多步骤复杂任务时存在显著局限性，容易出现语义偏差和信息传播错误。

4.2) 模块贡献分析

多智能体框架通过动态生成子代理和分工合作结合错误反馈机制显著提升了任务分解的全面性，尤其在处理复杂任务时框架的动态调整使综合评分平均提高 8-10 分。语义相似度模块对维持任务分解的语义一致性至关重要，移除该模块后模型在高复杂任务中的

TDS 评分下降超过 15%，证明该模块能有效保障信息传播准确性和分解结果对齐。完整框架下 ChatGPT-4.0 在高复杂任务中的评分为 84.7，明显优于无多智能体框架的 77.2 和无语义相似度模块的 81.5，这表明模块协同作用能全面优化复杂任务分解能力。

消融实验证实，多智能体框架通过降低任务复杂性和提升分解全面性显著优化了任务分解性能，同时语义相似度模块减少了信息传播误差并维持了语义一致性。两模块协同作用进一步提升了复杂任务分解效果，其模块化设计能够灵活高效地适应不同难度任务的需求。

表 3-6 消融实验评分结果
Tab. 3-6 Ablation experiment scoring results

模型	配置	简单任务	中等任务	复杂任务
ChatGPT-3.5	完整的框架	90.4	84.1	77.6
	无多代理架构	85.3	78.6	70.2
	无语义相似模块	87.1	81.2	73.4
	单独 LLM	82.1	75.4	68.3
ChatGPT-4.0	完整的框架	94.2	89.6	84.7
	无多代理架构	89.7	84.3	77.2
	无语义相似模块	91.6	87.1	81.5
	单独 LLM	88.7	81.3	72.8
Zhipu-GLM4-Plus	完整的框架	93.8	88.7	83.4
	无多代理架构	89.3	83.6	75.6
	无语义相似模块	91.2	86.3	79.8
	单独 LLM	87.4	80.6	71.5
DeepSeek-R1	完整的框架	94.4	90.5	84.9
	无多代理架构	90.1	84.8	77.8
	无语义相似模块	91.2	87.7	80.2
	单独 LLM	89.1	82.4	73.2

3.3 本章小结

本章节针对巡检机器人个性化功能难以自动开发的问题，本章节提出了一种多智能体自协作框架，该框架通过多个 LLM 相互协作共同制定机器人自动开发策略，并生成机器人可执行代码。通过调用 LLM API 构建不同角色的 Agent，并在该框架的指导下进行机器人二次开发。为了确保多个模型之间的有效协同，本章节基于现实机器人开发团队的合作模式设计了明确的智能体协作规则。该自动化协作框架的创新性在于使非专业人员无需编码经验即可通过交互完成复杂机器人开发任务。3.1.3 节的机器人功能开发实验表明，相比传统多团队协作方式和单一大型语言模型，多智能体自协作框架在复杂任务处理的效率和准确性上具有明显优势。

针对任务策略与机器人动作行为之间难以对齐的问题，本章节提出了一种基于余弦相似度的原子任务对齐方法，通过余弦相似度关联任务规划与机器人运动控制指令之间的语义相似性，将语义相近的文本映射到向量空间，以实现任务与指令之间的对齐。针对现有

评估基准难以衡量代理任务分解的细粒度层级问题,我们开发了基于语义度量的新型评价体系。该体系通过同时评估任务策略与机器人指令间的信息传播精度和分解质量,有效解决了任务到指令转换过程中的语义偏差问题。3.2.4 节的基线对比和消融实验验证了该评价方法的可靠性。

4 基于自然语言交互的机器人任务规划

本章针对电力系统巡检任务中模型与环境交互的难题,并提出了创新的解决方案以增强模型的环境感知能力与任务执行效率。首先针对电力系统巡检任务规划过程中难以获取环境信息的问题,本文提出了一种多模型交互与环境感知方法,该方法整合大型语言模型与视觉语言模型,实现对真实环境信息的深度解析,为巡检任务提供精准的语义理解。针对开放环境导航需求,开发了基于热值图的导航算法,将视觉数据映射至二维计算空间,实时获取物体位置信息,并通过贪心算法规划最优路径,指导机器人精确运动。该方案支持复杂环境下的动态轨迹调整,有效提升巡检效率与环境适应能力。

此外,针对大模型生成过程中可能出现的局部最优问题,本章提出了一种自回归循环语义对齐方法,旨在修正语义对齐过程中产生的语义偏差。该方法将错误任务与提示词合并生成反馈信息,使语言模型在后续生成中更准确匹配任务文本与机器人指令的语义关系,显著提升任务成功率。实验验证了自回归循环语义对齐方法的有效性,确定了最优反馈次数,结果表明该方法在语义匹配精度和任务完成率方面具有明显提升效果。

4.1 多层模型嵌套架构的环境感知交互

4.1.1 多模型环境交互问题

在开放场景中执行任务时,机器人会面临环境的动态变化和突发状况。这些变化使机器人必须实时动态调整策略,以适应不断变化的环境。目前,传统的环境感知方法分别是激光雷达结合 SLAM (同步定位与地图构建) 技术和通过训练视觉模型进行物品识别。

激光雷达结合 SLAM 技术擅长解决路径规划和避障问题,但机器人与环境进行交互时,该方法并不能有效识别物品种类信息。因为激光建图技术主要通过还原物体的形状和位置信息来构建环境模型,但无法识别环境中物体的具体名称和种类。尽管激光雷达能够生成精密度极高的点云地图并且提供高精度的地理信息,但是并不能提供智能决策过程中所需要的语义信息。

另外一种方法是通过视觉模型训练实现物品识别,但是能够识别物品的前提条件是提前将这类物品进行标签化处理后,通过神经网络训练后得到模型权重。也就是说研究需要提前知道环境中有哪些物品,然后对这些物品进行采样后通过训练才能得到视觉模型。这样显然是不现实的,开放场景中的物品对于研究来说是未知的。既要保证能够识别出物品的种类同时又不能提前对物品进行视觉模型训练,这样就无法提供所需的完整环境信息,所以这两种方法在环境感知方面存在着显著的局限性。

为了给推理的大型语言模型提供完整的环境信息,必须要在传统方法的基础上开发一个新的模块,补全现有方法的缺陷。针对上述问题本文考虑在系统中引入视觉语言模型 (VLM),该模型因其超大参数的图像理解能力,可以在无需训练的情况下自动识别物品的

类别与基本信息。这一方法与我们的需求相吻合。该方法带来的新问题在于，我们需要与环境进行交互，也就是需要使系统具备环境理解能力与逻辑判断能力，这些能力显然是 VLM 不具备的。因为从得到环境中的物品信息再到策略生成仅仅依赖于单纯的大型语言模型是不现实的，需要一种全新的方法将 VLM 提取的环境信息与其他功能模块结合，实现开放环境中的策略生成与环境交互。因此如何有效地将 VLM 感知到的环境信息与 LLM 进行交互，成为当前机器人系统面临的一个重要问题。

4.1.2 多模型交互与环境感知方法

人类依靠经验积累能够通过简单交流理解复杂任务，经过大规模数据训练的语言模型可以模仿人类语言理解能力并识别对话深层意图。该方法发挥模型在语义分析和文本生成方面的优势，将复杂任务拆解为可执行的子步骤，这种分步处理机制特别适合机器人任务规划系统，能有效提升人机协作效率。

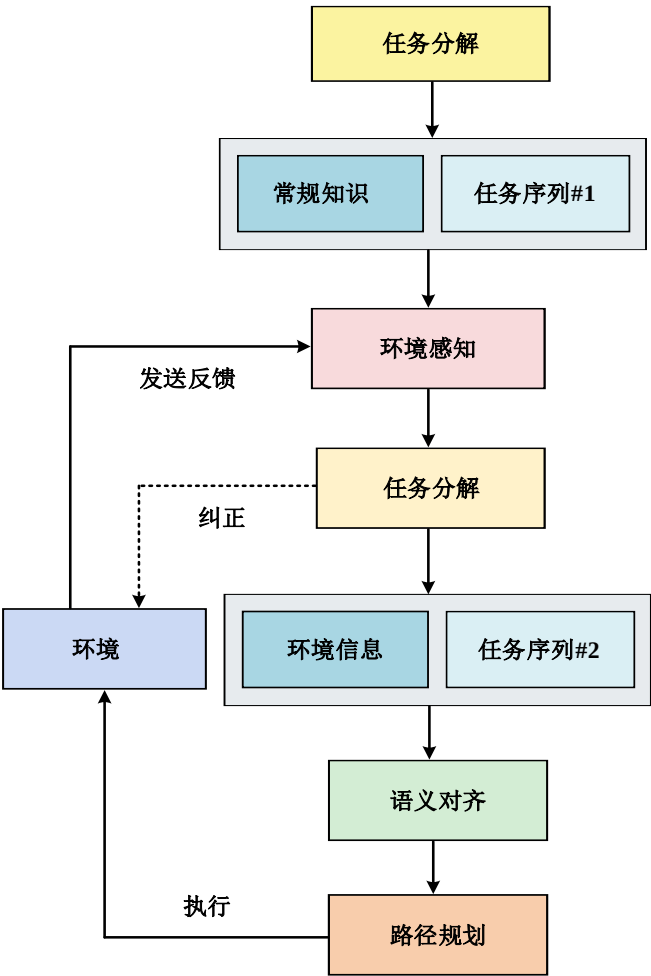


图 4-1 环境感知方法概述

Fig. 4-1 Overview of Environmental Perception Methods

针对上述技术挑战，本研究开发了 VLM 用于识别真实场景中的物体特征。该模型与 LLM 实时交换数据，首先获取当前环境的关键信息，然后将这些信息与初步拆解的任务指令结合形成更详细的操作步骤。系统通过文本向量相似度匹配将分解后的任务需求转化为机器人可执行指令，实现语言模型生成内容到具体动作的准确转换。图 4-1 展示了该环

境感知方法的整体架构。

为实现精确的环境感知，设计了一种基于热值图的算法，详见 3.1.3。该方法通过构建二维可计算空间，将摄像头采集的视觉信息映射为反映物体真实位置的数据。系统结合贪心算法与热图中的密集点状轨迹生成动态映射，实时指导机器人运动。这种图像级映射能够根据环境变化自动调整运动轨迹。

值得注意的是，本研究方法的关键创新点在于构建了多模态信息融合机制。具体而言，通过视觉语言模型（VLM）实时提取环境的空间特征后，再通过特征绑定模块与大语言模型实现跨模态交互。该架构使 LLM 能够利用动态环境特征生成上下文感知的决策策略，克服传统方法依赖预训练策略的限制，在未训练场景中具备零样本迁移能力，实现开放环境下的机器人自主控制。通过融合热值图算法与规划框架，结合多模态语言模型协同工作，机器人能够理解并执行抽象语义指令。该集成方法不仅提升任务执行精度，还支持通过自然语言控制高复杂度任务。实验验证了该方法在自然语言理解和机器人行为引导方面的有效性。

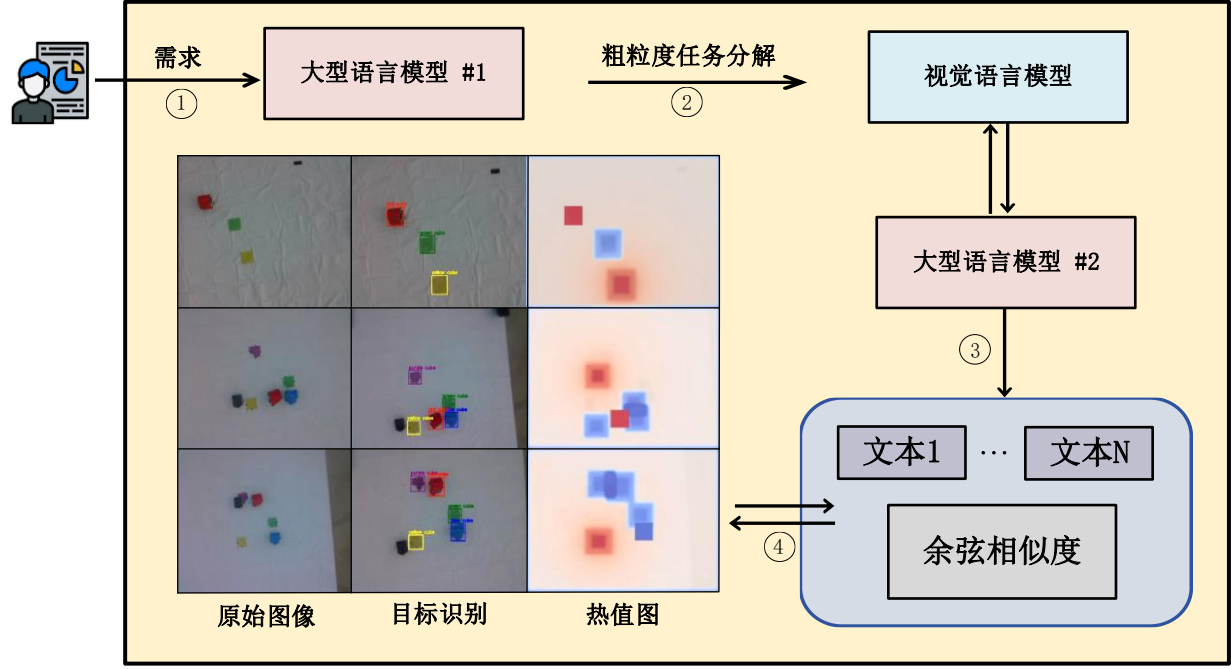


图 4-2 多模型交互与环境感知方法的功能模块与整体结构

Fig. 4-2 Functional modules and overall structure of multi-model interaction and environment perception methods

在多模型交互与环境感知方法中，我们整合了两个大型语言模型与一个视觉语言模型，如图 4.2 所示。第一个大型语言模型用于解析人类指令并为机器人生成可执行的粗粒度计划，该过程依赖常识进行任务分解。由于常识本身存在局限且系统无法获取完整环境信息，模型生成的任务序列可能偏离实际机器人行为，导致粗粒度任务分解结果在真实环境交互中表现出不足。

为提高大语言模型的环境泛化能力，系统将粗粒度任务分解结果逐步输入后续功能模

块,结合多模态信息增强模型对不同场景的适应性。该架构采用级联处理流程,由 GPT-3.5 架构的 LLM-1 生成初始任务框架后,传递至联合处理模块进行细化。该模块整合了 LLM-2 与基于 OWL-ViT 架构的视觉语言模型 VLM,后者通过大规模跨模态预训练具备实时环境实体检测与属性识别功能。两阶段模型通过 API 调用与本地部署方案实现异构系统间的数据通信。在迭代优化阶段,LLM-2 基于 VLM 提供的环境信息能够更新初始任务并且结合环境信息生成更准确的任务分解结果。

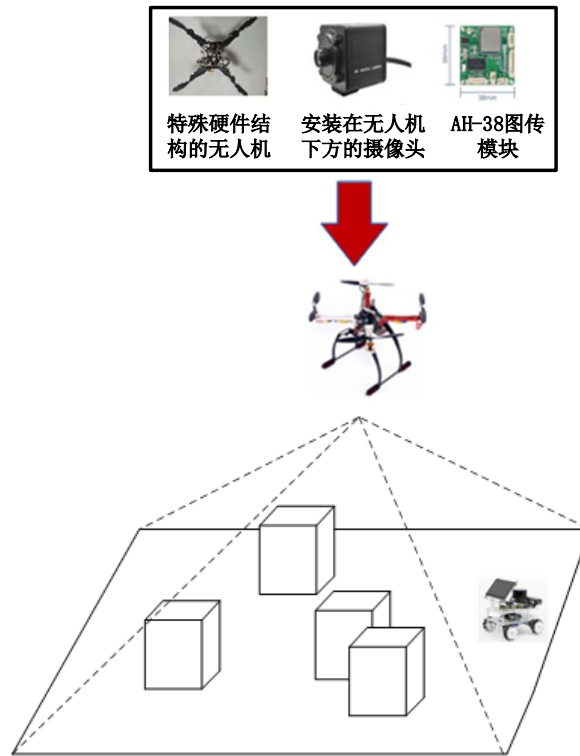


图 4-3 无人机获取地面信息方法示例

Fig. 4-3 Examples of how drones can obtain ground information

虽然 LLM 生成的任务序列明确了机器人应该执行的动作,但是由于任务序列与机器人运动指令之间的对齐问题,导致机器人不能顺利执行 LLM 生成的任务序列。为了便于大语言模型调用机器人运动,我们采用了一种语义相似性评估方法(具体见 3.2 章节),将任务序列与指令序列在向量层面上进行对齐。在这个过程中,首先将文本进行向量化,将任务和指令表示为数字格式。随后通过向量归一化来减轻文本长度对向量的影响。最后在向量层面上对任务和指令进行对齐。上述过程能够使 LLM 生成的任务序列能够在向量层面与机器人运动对齐,使任务序列能够转化为机器人可执行的动作指令。

如图 4-3 所示,本方法通过视觉模型处理摄像头采集的视频流数据,驱动机器人在接收指令后执行导航至指定坐标点的操作任务。基于视觉识别结果构建的二维热力分布图中,目标坐标呈现高热值特征,非目标区域热值呈梯度衰减分布,形成连续的空间能量场表征。实验系统中,机器人实体采用红蓝双色标识进行可视化,其中高热值目标区域以红色渐变光谱表示,常规区域则映射为蓝色渐变光谱。通过贪心算法构建的路径规划模块能够生成向全局热极值收敛的最优轨迹。热值图动态更新,实时与无人机视觉转换同步,保证快速

响应环境变化。因此，基于热值图的导航方法能够迅速适应环境并生成导航指令。

4.1.3 开放环境感知的热值图导航算法

本章节我们基于 4.1.1 和 4.1.2 的内容，对自然语言L控制机器人具体行为这一步骤进行建模，例如让机器人先到达位置 a，然后再去位置 b。在人类的主观理解上，这是一个简单的任务，但是对于机器人来讲这个过程并不容易。基于L生成机器人轨迹非常困难，因为“L”提供的信息是粗粒度的，缺乏任务执行的详细过程，同时缺少了必要的环境信息。机器人执行任务的环境并不是一个静态的空间，因此有必要在机器人执行任务过程中实时为机器人传输环境信息。假设大型语言模型将L分解为几个子任务 $(l_1, l_2, \dots, l_n)$ ，在任务生成的这个阶段，我们关注于实时环境来创建详细的任务 l_i 来生成机器人可以执行的运动控制命令。

上述方法将复杂任务分解为低复杂度的子任务，并为每个低复杂度任务感知环境。本小节的核心问题是如何充分利用环境信息为每个任务 l_i 生成机器人运动轨迹 τ_i^r 。在 4.1.4 章节中我们构建了机器人集群来执行运动轨迹 τ_i^r ，是由多台麦克纳姆轮无人小车来执行运动的，更为重要的是我们将环境信息与轨迹生成问题相结合，构建了一个数学模型来阐述这个问题，问题总结如下：

$$\min_{\tau_i^r} \{F_{\text{task}}(T_i, l_i) + F_{\text{control}}(\tau_i^r)\} \quad (4-1)$$

公式(4-1)为无人车在动态环境中的约束条件，其中 T_i 为传感器采集到的环境信息， $\tau_i^r \subseteq T_i$ 为无人车在动态环境中的运动轨迹， $F_{\text{task}}(T_i, l_i)$ 表示在动态环境范围内完成相应任务的情况， $F_{\text{control}}(\tau_i^r)$ 指定最短路径或最小任务执行时间所需的控制成本。

在基于自然语言计算任务函数 $F_{\text{task}}(T_i, l_i)$ 时，存在很大的困难，一方面是因为自然语言与机器人任务 l_i 之间的难以对齐问题，另一方面则是由于缺乏动态环境信息和实时机器人位置信息。在这种背景下，我们提供了一种二维可计算空间，用来描述在动态环境中对象的相对位置信息，记作 $V \in R^{w \times h}$ ，我们称之为热值图。该热图反映了环境中我们感兴趣或不感兴趣的对象，对于研究感兴趣的对象，热图的值较高，而对于不感兴趣的对象，则显示较低的热值。热图通过为高热值的物体提供引导力，影响物体的移动，形成机器人与感兴趣物体之间的轨迹曲线。热值图根据大语言模型定义的子任务，给环境中的物体分配不同的热值。在此基础上，任务目标被标记为具有较高热值的区域，从而吸引机器人朝目标区域移动。相反感兴趣之外的物体则在热图上具有较低的热值，从而对机器人产生排斥作用，引导机器人远离非目标区域。

我们将高热量目标表示为 e ，机器人轨迹表示为 τ^e 。对于子任务 l_i 在 $F_{\text{task}}(T_i, l_i)$ 中，我们可以通过热力图的方式，在二维空间 $V \in R^{w \times h}$ 中对任务进行数值化。对应的任务 F_{task} 在环境中可以通过在二维空间 $V \in R^{w \times h}$ 中对 e 的连续累积来近似。其公式如式(4-2)所示：

$$F_{\text{task}} = - \sum_{j=1}^{|\tau_i^e|} V(p_j^e) \quad (4-2)$$

其中 $p_j^e \in N^2$ 是第j步中的离散位置 (x, y) 。



图 4-4 机器人执行任务提示工程实例

Fig. 4-4 Robot execution task prompt engineering example

本研究采用嵌入式提示架构，将任务目标、预期输出及相关实体逻辑关系整合，引导LLM生成符合需求的内容。LLM基于上下文感知动态响应，能够根据输入提示生成对应输出。针对机器人路径优化问题，如图4-4所示，系统通过环境特征融合机制有效集成物体语义信息。利用LLM-2将问题分解为细粒度任务序列，结合环境信息使机器人能够在真实场景中执行任务。

通过提示工程操作能够准确识别目标物体及其空间关系特征并生成可执行动作序列。首先利用视觉语言模型应用程序获取环境信息以确定场景中物体的相对位置，随后基于感知到的环境信息结合LLM-1的通用知识生成特定场景下的任务。接着将任务通过代码输入到热图模块，从而生成与每个任务步骤 l_i 对应的机器人运动轨迹 τ^e 。最后，进一步可以得到热图 $V_i^t = \text{heatmap}(o^t, l_i)$ ，其中 o^t 是 t 时刻的摄像头观察信息， l_i 是正在执行的任务。

为实现机器人运动轨迹的连续平滑优化，我们将任务的每个步骤表示为一个数学问题 $F_{\text{task}}(T_i, l_i)$ 。该模型通过微分方程约束实现轨迹规划，热值图能够显示环境中的物品位置以及是否为任务目标。任务目标的物体具有较高的热值，吸引机器人朝其移动，而非目标物品则显示为较低的热值，推动机器人远离低热值物体，在热值图中所有的目标都可以被计算路径。算法的目标是创建一个具有最高热值的轨迹，能够到达指定的子任务位置。在算法5中定义路径的“热值”为路径上所有节点的热值总和，其公式如式(4-3)所示。

$$R(P) = \sum_{i=1}^k H(v_i) \quad (4-3)$$

其中， k 为路径长度， $H(v)$ 为节点 v 的热值。我们希望找到一个使 $R(P)$ 的值最大化的路径 P ，利用贪婪算法的思想在每一步中选择下一个节点 v_i 使 $H(v_i)$ 最大，公式如(4-4)所示。

$$v_i = \operatorname{argmax}_{v \in N(v_{i-1})} H(v) \quad (4-4)$$

其中, $N(v_{i-1})$ 表示节点 v_{i-1} 的邻居节点集合, 这种方式可以找到路径 $P=\{v_1, v_2, \dots, v_k\}$, 其中 v_1 是起始点, v_i 是每一步选择的节点, 直到到达具有最高热量值的节点 $H(v_i)$ 为止。

表 4-1 基于热值图的动态导航算法

Tab. 4-1 Dynamic navigation algorithms for calorific heat maps

算法 5 基于热值图的动态导航算法

- 1: **Input** 自然语言形式的任务 $\mathcal{L}$
- 2: 任务拆解:
- 3: 将任务表示为最佳优化问题:
- 4: $\min_{\tau_i^t} \{F_{\text{task}}(T_i, l_i) + F_{\text{control}}(\tau_i^t)\} \quad \text{subject to} \quad C(T_i)$
- 5: 构建数学表示的热值图任务:
- 6: $F_{\text{task}} = -\sum_{j=1}^{|\tau_i^t|} V(p_j^e)$, 其中 $p_j^e \in \mathbb{N}^2$
- 7: 构建二维空间: $V \in \mathbb{R}^{w \times h}$
- 8: 构建热值图: $V_i^t = \text{heatmap}(o^t, l_i)$
- 9: 定义高热值路径:
- 10: $R(P) = \sum_{i=1}^k H(v_i)$ 我们将路径的热值定义为路径上所有节点的热值之和。
- 11: 路径生成:
- 12: $v_i = \arg\max_{v \in N(v_{i-1})} H(v)$ $N(v_{i-1})$ 表示节点的相邻节点的集合 v_{i-1}
- 13: **Output** $P = \{v_1, v_2, \dots, v_k\}$ 输出热值最高的路径

我们在空间中提供密集奖励函数, 可以生成规划路径。在机器人操作过程中, 由于2D视觉信息不能包含完整的空间信息, 它提供了机器人与环境之间的位置信息关系。机器人通过实时反馈的热图不断地逼近我们生成的运动轨迹。具体而言, 实时更新环境信息的过程为机器人提供了持续的反馈, 以便控制机器人运动节点, 使得机器人的运动轨迹不断接近我们生成的路径, 当机器人行为发生变化时, 可以通过反馈重新规划实际的机器人运动轨迹。具体步骤可以参考表 4-1 中的算法 5。

4.1.4 机器人导航规划实验

该实验分为基线实验和消融实验, 通过第五章节的5.1.2我们构建了多模型交互与环境感知方法, 并且在4.1.3章节中我描述了一种用于开放环境感知的热值图导航算法, 为了探究热值图导航算法在整个系统架构中的作用, 我们设计了一组基线实验和消融实验来进一步的探讨热值图在整个系统中的作用。

在基线实验中没有热值图这一功能模块, 基线实验的架构图如图 4-5 所示, 该架构旨在实现大型语言模型和多个机器人的策略生成和运动控制, 解决大型语言模型中任务分解不彻底的问题。该架构由云端、边缘端和设备端三个核心组件构成, 支持多大型语言模型协同工作以实现多机器人策略生成与路径规划。云端 LLM 接收任务需求后生成整体协作策略, 统筹多机器人动作分配, 同时为单个机器人制定具体执行方案, 确保协同作业与任务分解高效完成。

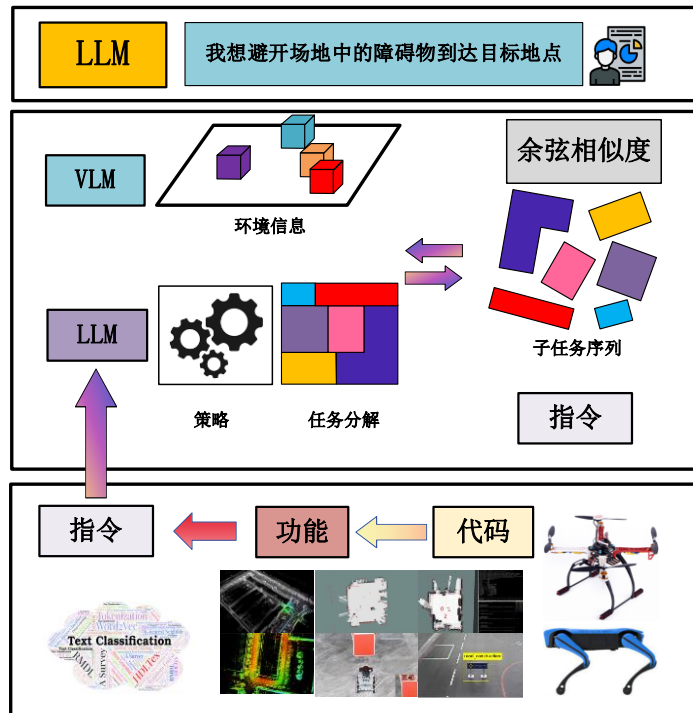


图 4-5 四足机器人策略生成基线实验

Fig. 4-5 A baseline experiment on task planning for a quadruped robot

在边缘端，我们部署了两个规模较小的关键模型，一个 VLM 和一个 LLM，VLM 负责采集局部环境数据，LLM 则解析云端生成的协作策略，将其分解为可执行的子任务指令序列。利用边缘端中的 LLM 理解云端的 LLM 生成的策略，将策略分解生成一系列子任务序列，并将其转换为可执行指令序列。这两个模型相互协同，基于局部环境信息进行任务分解，为设备侧上的机器人生成指令序列。该架构和我们在 4.1.2 我们构建的多模型交互与环境感知方法的区别是这个架构中没有使用热值图功能模块来提供更加精准的导航策略。

基线实验方法聚焦两个核心目标，验证任务分解结构的完备性和通过标准化封装机器人底层功能模块建立层次化功能体系。为保障生成任务与系统功能的精确匹配，采用余弦相似度量评估任务描述与功能接口特征向量的匹配程度。当子任务相似度低于阈值时，系统自动触发迭代分解机制对未达标子任务进行二次分解优化，直至所有子任务在功能库中找到对应实现模块。

基线实验中我们采用了 3.2.2 中提到的基于余弦相似度的方法来判断任务分解是否彻底，以优化任务分解的深度和准确性。该方法通过量化子任务相似性判断是否需要继续分解，在四足机器人与无人机协作的基线实验中专注于四足机器人的无地图策略生成与运动控制。结合云端大语言模型提供的用户需求与策略生成能力，依托边缘端的视觉与语言模型实现开放环境下的任务分解与自主运动执行。

如图 4-6 所示系统架构中，无人机通过 LLM 的指令驱动执行定点巡航扫描，同时同步构建环境的多模态感知数据流。环境特征数据经边缘计算层预处理后输入任务规划引擎

完成语义解析与策略生成。VLM 执行多级任务分解，利用余弦相似度动态评估矩阵验证分解完备性，若相似度低于阈值则自动触发策略迭代优化。最终任务指令序列经协议封装后通过低延迟通信传输至四足机器人执行终端。

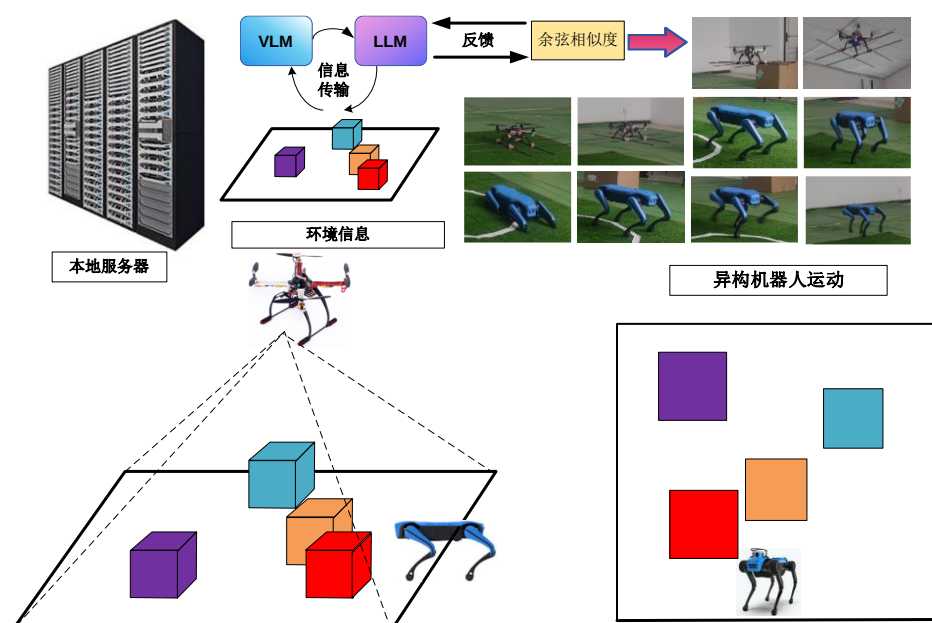


图 4-6 多机器人策略生成基线实验示例

Fig. 4-6 Example of a baseline experiment for multi-robot policy generation

如图 4-7 所示，我们在现实场景中完成了无人机和四足机器人的策略生成和运动控制，四足机器人通过提供无人机的场景环境信息，根据边缘计算层生成的任务序列完成任务。



图 4-7 基线实验中的无人机与四足机器人

Fig. 4-7 Drones and quadruped robots in baseline experiments

我们的基线实验取得了良好的成果，成功解决了任务分解不彻底的问题，顺利实现了在开放场景中多个机器人和用户之间的互动。在基线实验中我们成功实现了多机器人任务分解、多机器人策略生成以及运动控制。但是也存在一些问题，基线实验能够简单的获取简略的环境信息，通过大型语言模型为机器人提供一般性的动作指导，但是缺失了更加细致化的路径规划与环境量化手段，导致机器人在执行具体动作时出现不可挽回的偏差。

接下来进行消融实验部分,研究首先讨论了消融实验的整体设计和执行,并在现实的环境中实施了整体实验。在这个实验中,我们构建了一个真实世界的实验平台来验证基于热值图导航算法的有效性,且在服务器上部署了大型语言模型和视觉模型,通过调用 LLM 来提取通用知识。实验中使用了三辆无人车 and 一架无人机通过 ROS 系统实现了机器人的运动控制。通过无人机提供视觉信息,视觉信息被传回服务器用来感知环境信息,通过 VLM 和 LLM 结合环境信息生成机器人运动方案,最后通过热值图算法生成优化的轨迹,引导机器人完成相应的任务,如图 4-8 所示。

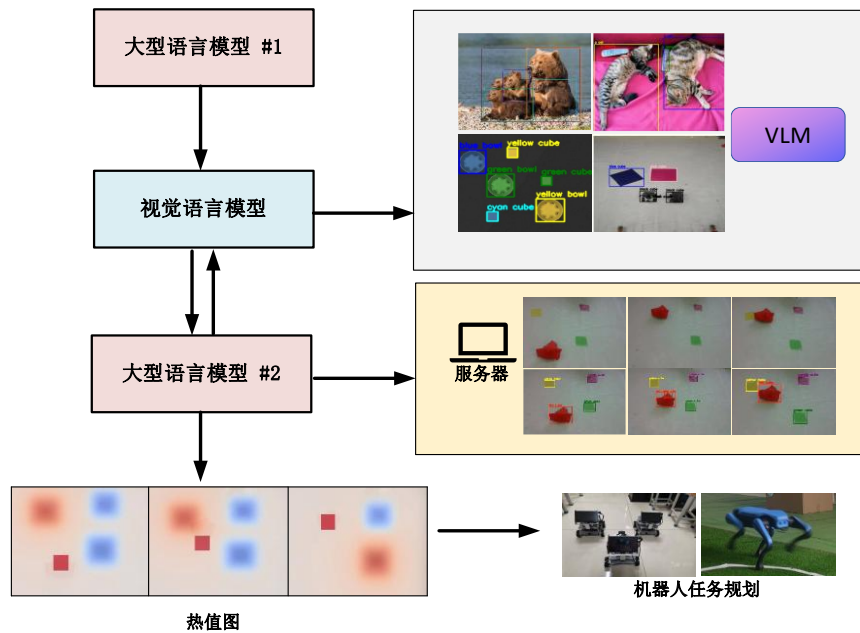


图 4-8 多机器人任务规划实验完整结构

Fig. 4-8 The complete structure of the multi-robot task planning experiment

在图 4-9 中,本文对实验进行了详细的描述,阐述了从自然语言指令到机器人运动热力轨迹图生成的技术路径。无人机搭载的摄像头能够采集场景信息,通过对视频进行关键帧提取,能够获得动态的场景变化。视觉语言模型对视频帧进行结构化解析,能够将图像信息转化为文本信息。具体来讲, VLM 在接收到无人机的摄像头采集到的视频帧后对图像进行判断,通过 VLM 能够识别出具有全局视野图像中的物品信息,并且将场景信息转化为文本信息传输至后续的 LLM。

LLM 接收到 VLM 传输的文本信息,将任务信息与环境信息进行分析推理,生成机器人任务序列。该过程采用 3.2 章节的基于向量空间的任務对齐方法,确保每个子任务与运动控制指令在空间语义上一致。机器人运动轨迹以热值图形式呈现,系统将离散动作指令转换为连续空间中的概率分布图,从而为机器人提供抵达目标点的路径导航。在图 4-9 的实验中,采用彩色标识块对无人车进行标记,其中初始目标区域用黄色标记,终止区域用绿色标记,移动载体则使用红色标识。

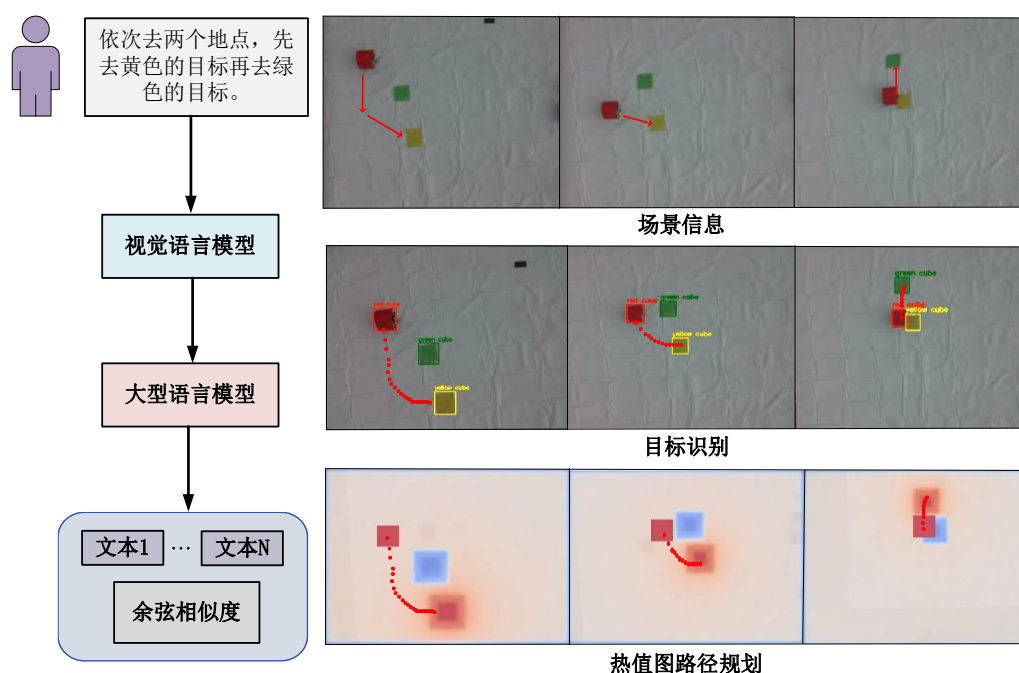


图 4-9 描述从图像信息到生成热值图运动轨迹过程的实验设计。

Fig. 4-9 Describe the experimental design of the process from image information to the generation of heat map motion trajectory.

图 4-10 展示了由我们设计的五个不同难度级别的任务。本研究采用自然语言对五个典型机器人导航任务进行系统性描述，所提出的框架能够有效指导机器人完成导航作业。实验设计遵循难度递增原则，逐步提升导航任务的复杂度，确保所有任务均属于机器人导航的标准应用场景。

在机器人导航规划实验中设计的 5 个实验分别为（1）机器人需要连续经过两个不同的目标点；（2）机器人需要按照先后顺序依次连续完成三个目标点；（3）机器人需要能够理解语义信息，成功规划出最短的路径；（4）机器人需要判断场景中是否有人为标记，如果有的话，就需要前往地点 A，如果没有的话需要前往目标地点 B；（5）有多个机器人 a、b、c，机器人 a 和机器人 b 各前往不同的人工指定位置，机器人 c 需要判断机器人 a 在到达指定位置后，也去往相同位置与机器人 a 汇合。

值得注意的是，上述所有任务都可以使用自然语言与真实的机器人进行交互，并可以实时提供动态反馈，以减少环境信息变化造成的干扰。我们在真实环境中成功完成了上述 5 个实验，图 4-10 中的图像都是在真实实验过程中捕获的。实验表明，大型语言模型的任务分解，加入环境信息和语义对齐，可以控制机器人执行更高复杂度的任务。通过这个消融实验，我们能够得到一个显而易见的结论，完整的无人集群复杂任务执行结构能够给机器人更加有效的指导。热值图模块能够为机器人提供精确的环境信息以及可以量化的计算空间，使机器人能够更加准确的理解人类意图，提升任务执行的准确率。

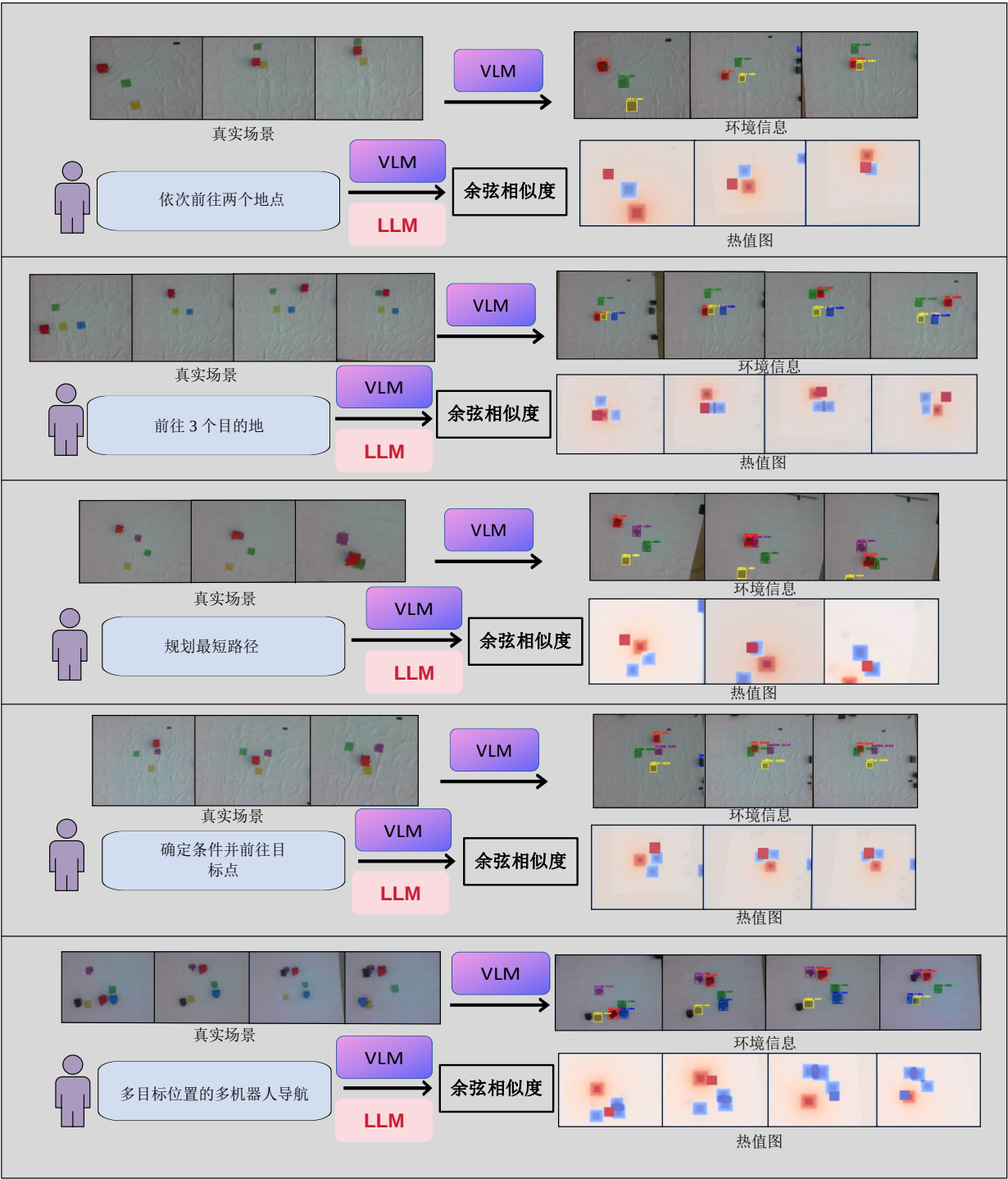


图 4-10 这 5 个实验的设计内容包括不同复杂性的任务。描述了 5 个实验的流程，包括我们从原始图像信息中识别目标，最后通过热值图完成导航任务。

Fig. 4-10 The design of these five experiments includes tasks of different complexity. The process of the five experiments is described, including identifying the target from the original image information and finally completing the navigation task through the heat map.

4.2 巡检机器人自回归循环语义对齐

4.2.1 大模型结果局部最优问题

大模型生成策略或者生成机器人动作指令时，经常会出现一种现象，那就是由于缺失

信息的上下文以及不完备的信息提供给大语言模型，导致大型语言模型输出错误的问题，其本质是在于大型语言模型没有正确理解正确的上下文所带来的误解。由于读取的信息不完备，或者过于长的上文使大型语言模型的理解产生了冲突。因为大型语言模型在理解需要生成的策略时，是通过多轮历史对话判断的，如果历史对话中的某些信息是错误的或者是相互冲突的，就会使大型语言模型产生逻辑混乱，从而不能提取到我们真正想要得到的信息。

产生的现象就是，大型语言模型由于得不到偏向正确答案的引导，会一直重复错误的答案，这种现象会随着对话轮数的增加而明显增强。有两种可能性，第一种就是因为上下文过于庞杂，导致的大型语言模型输出的结果是错误的，这种现象的原因是多个任务在历史数据中出现了混淆，大型语言模型没有办法判断上下文之间的正确关联，所以输出错误的答案。

另外一种情况是，上下文句意没有办法将真实的信息反馈给大型语言模型，因为反馈信息有可能并不包含完整信息，导致大型语言模型只能够通过缺失的予以信息进行判断，此时大型语言模型会因为幻觉现象以及不完整语义产生错误判断，生成局部最优的结果，所谓局部最优的结果就是，在处理某个子任务时，大型语言模型认为自己已经得到了最优的结果，多次出现同样的答案。这个答案也许是错误的，也许并不是最优解。但是在机器人执行过程中，人在回路的调节可以解决这个问题，但是这个过程非常的耗时。因此寻求一种自动化的反馈机制显得尤为迫切。如何设计一种有效的自动反馈环节，使其能够将真实情况反馈给大型语言模型，并引导模型的输出结果朝着正确的方向发展，是目前亟待解决的关键问题。

4.2.2 自回归循环语义对齐方法

根据上述描述，为提升任务规划的可解释性，我们在整个环节增加了一个任务规划的中间过程，当执行轨迹偏离预期时，允许操作者在关键节点进行人工修正。并且我们进行了基于语义相似度的反馈实验，并讨论了最优循环数。最后我们从实验中提出了一些失败的案例，并对这些案例进行了讨论和分析。

我们使用图 4-11 来直观地展示两个 LLM 的输出格式。这使得每个模块的输出具有可控性，允许在特定模块中进行人工干预，以生成与我们预期一致的结果。引入任务规划的中间步骤可以增强整体任务规划过程的可解释性。这一中间步骤为任务规划过程中的人工干预提供了机会，尤其是当生成的结果偏离预定的人工任务执行过程时。人工干预通过三个维度得以促进：粗粒度任务分解、细粒度任务分解和运动控制指令。可以预见我们采用第 4.1.2 章节中所提出的多模型交互环境感知方法，我们可以提高机器人控制的可靠性和安全性。

图 4-11 展示了中间任务规划过程的一个示例，研究以全遍历黄色目标物场景为验证对象。该框架采用双阶段解耦策略，首先由粗粒度任务解析层 LLM-1 进行领域知识驱动的任务拓扑构建，生成基于先验约束的初始任务序列，包括目标物定位策略与移动路径规

划，此阶段输出受限于动态环境感知的缺失。随后 LLM-2 注入实时场景数据，并合并了由 LLM-1 产生的序列中得到的细粒度任务分解。这种集成产生了包含场景信息的新任务序列。显然 LLM 可以通过整合环境信息来生成更接近真实情况的任务序列。随后通过与相应的运动控制命令对齐，在热值图中生成一个运动轨迹指导机器人执行任务。

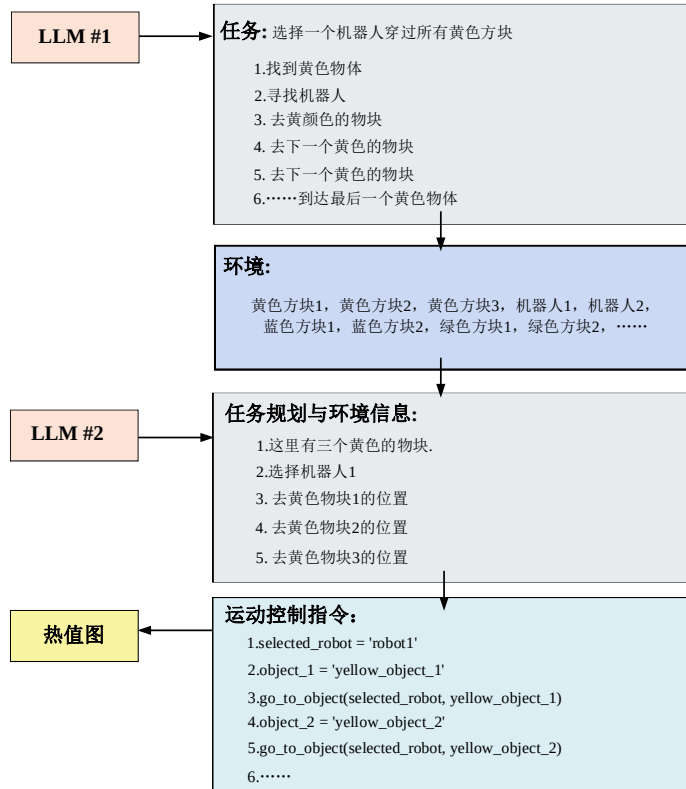


图 4-11 任务规划中间过程的一个示例

Fig. 4-11 An example of the intermediate process of mission planning

LLM 能够输出的语义相似的结果，表明 LLM 能够理解问题并生成准确的结果，这在语义级别的上已经是接受的。我们通过向量对齐，便于将大型语言模型生成的结果映射到机器人控制命令中。值得注意的是，当在语义级别上进行对齐时，大型语言模型擅长于分解语义正确的任务。这些任务可以通过语义对齐成功地映射到相应的运动命令上。

我们观察到由大型语言模型生成的结果并不始终是最优的，这种差异源于提示 LLM 的过程并不包含所有的工作条件。当使用详细的自然语言描述某些复杂的任务时，LLM 可能会出现理解偏差，这种现象会导致理解复杂任务时出现细微差别。由于问题表述过程中的理解偏差，大型语言模型的输出结果可能会偏离我们预期的方向，针对这种现象本章节提出了一种自回归语义对齐方法，能够有效的降低语义偏差导致的任务失败。

如图 4-12 所示，针对语义对齐失败的任务系统执行反馈修正流程。错误任务序列被重新输入至次级 LLM 进行新一轮任务分解，该迭代过程通过整合错误任务反馈信息使 LLM 输出更符合预设提示词的语义。由于提示词已包含正确的机器人运动控制指令，这种语义偏向能够有效引导 LLM 生成与机器人运动控制相匹配的文本输出。这种方法提高了在语义层面上输出正确结果的可能性。

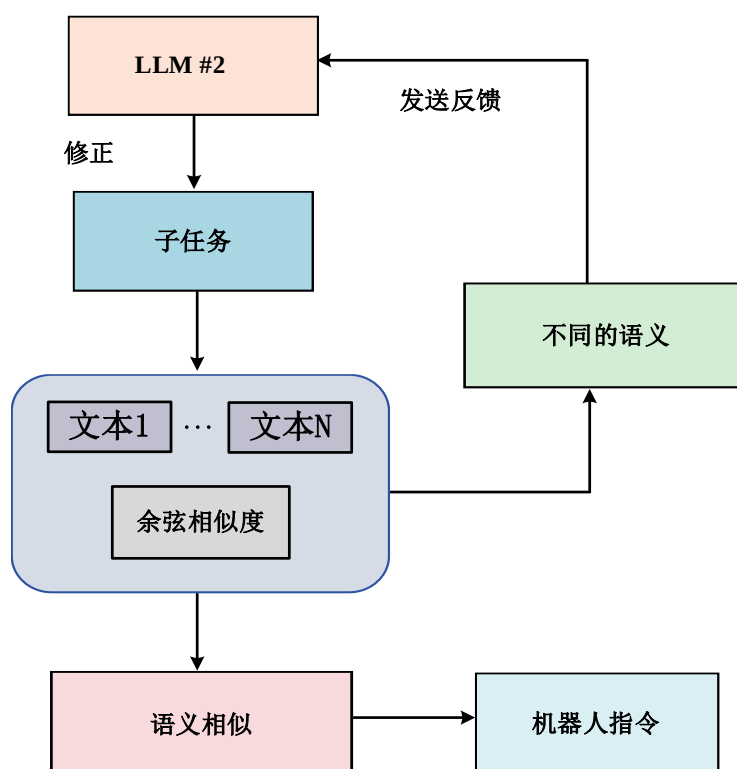


图 4-12 循环语义对齐方法的思想

Fig. 4-12 The idea of loop semantic alignment method.

如图 4-13 所示,针对第二个 LLM 的提示示例详细展示了当模型接收到语义不匹配的反馈后,其后续输出会显著倾向于遵循预设提示词的格式要求,这种模式有利于 LLM 产生偏向于我们所期望的输出。

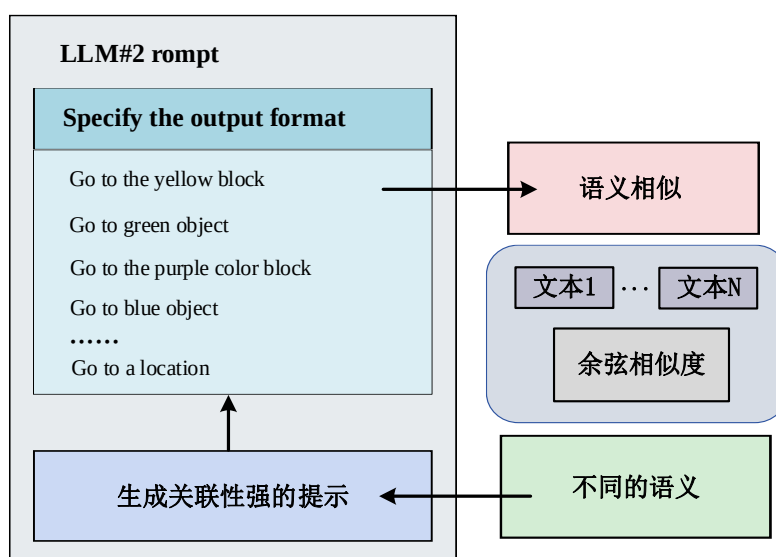


图 4-13 循环语义对齐方法中 LLM#2 的提示示例

Fig. 4-13 Example of hints for LLM#2 in the loop semantic alignment method

4.2.3 基于自回归循环语义对齐方法的反馈实验

我们进行了一个基于自回归循环语义对齐方法的反馈实验，并探索了最优迭代次数。如表 4-2 所示，证明了该反馈方法有利于提高任务执行的成功率。通过反馈告知 LLM 任务分解的报错信息，能够有效地减少信息的不确定性，并使 LLM 下一次分解的偏好倾向于提示词给定的期望结果。

表 4-2 基于自回归循环语义对齐方法的反馈实验结果

Tab. 4-2 Feedback experimental results based on autoregressive recurrent semantic alignment method

任务	反馈 0 次	反馈 1 次	反馈 2 次	反馈 3 次	反馈 4 次	反馈 5 次
任务 1	4/10	7/10	8/10	9/10	10/10	10/10
任务 2	3/10	6/10	8/10	8/10	8/10	8/10
任务 3	3/10	6/10	7/10	8/10	8/10	8/10
任务 4	4/10	5/10	7/10	7/10	7/10	7/10
任务 5	2/10	5/10	6/10	7/10	7/10	7/10
总数	32%	58%	72%	78%	80%	80%

实验结果表明，第三次反馈后，正确率达到 78%，从第四次反馈开始，任务成功率的上升速度变慢。我们考虑 3 或 4 次迭代作为反馈的最佳次数，证明了高系统效率。表 5.1 展示了在任务分解和向量对齐的两个步骤的中间，设置不同数量的反馈，并执行 5 个不同的任务来比较任务执行的成功率。

表 4-2 中的五个任务分别就对应了图 4-10 中的五个不同的任务。分别为任务 1：前往两个目标地点、任务 2：前往三个目标地点、任务 3：规划最短的路线、任务 4：目标定位、任务 5：多机器人到多目标的任务。

在实验过程中，在某些情况下机器人未能成功完成任务，从而导致实验的失败。本章节对这些失败案例进行了分析，经过分析部分实验失败的主要原因主要集中在 LLM 生成了错误的任务序列以及 VLM 没有正确的识别环境信息，导致热值图导航算法无法正常工作。需要注意的是，尽管正确的运动控制指令能够使机器人进行运动，但是在实际过程中由于 LLM 出现的幻觉现象会导致在语义传播的过程中出现偏差，所以会导致实验的失败。换句话说，LLM 和 VLM 之间的相互转换在语义理解上并不完全一致，导致了从自然语言到视觉信息的映射过程中产生误差，这是部分实验失败的原因。

当向 LLM 传递自然语言时，LLM 必须正确理解自然语言的语义并生成正确的策略。然而，LLM 并不总是能产生准确的结果。如图 4-14 所示，当给出“让机器人通过所有黄色方块”的任务时，正确的任务分解逻辑应该是让机器人通过它尚未到达的方块。然而，LLM 有时会生成混乱的任务分解过程，例如生成让机器人去到一个黄色方块，这显然是不正确的。任务分解过程中这种逻辑错误会导致机器人无法成功执行任务。

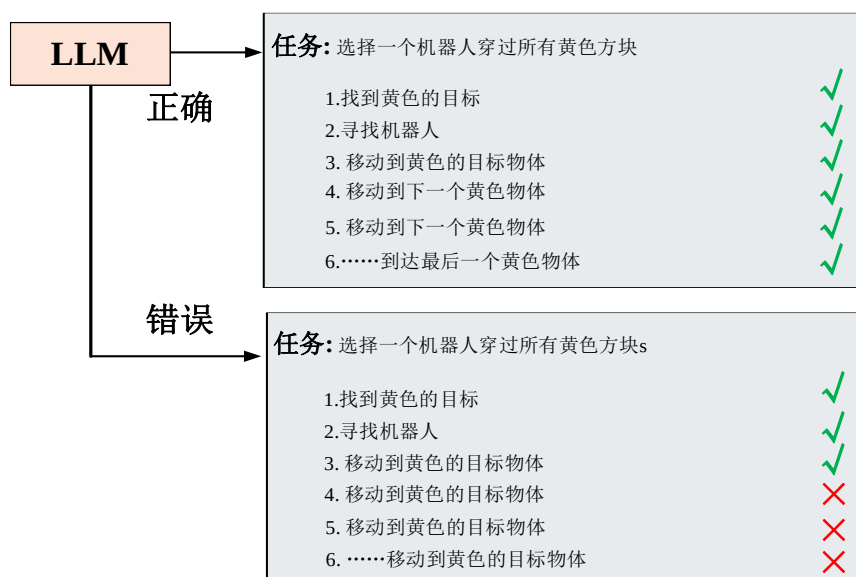


图 4-14 LLMS 生成策略错误的示例

Fig. 4-14 Example of LLMS generation strategy error

另一种情况是，VLMS 在环境感知过程中出现错误，如图 4-15 中的 (a) 所示。VLMS 会在光线变化时错误识别目标，导致 LLMS 接收到不正确的环境信息，这种情况下机器人无法执行任务从而导致任务失败。或者如图 4-15 中的 (b) 所示，VLM 识别到了场景中不需要识别的不相关目标，出现这种情况会在实验过程中造成干扰，并可能生成错误的热值图信息进一步的导致任务失败。

在这项工作中使大语言模型的输出与机器人控制指令在语义上对齐，并通过实验验证了这种方法可以通过自然语言控制机器人完成更复杂的任务。这样，人机交互变得更简单，现实中不需要专业技术或技能就能完成相关任务。人类可以用自然语言描述任务需求，通过本文设计的框架，这些需求可以在语义上被理解并转化为机器人能执行的任务序列，从而帮助机器人完成任务目标。值得注意的是，本文的方法通过多层次任务分解和与环境的互动，使机器人能够理解和完成包含复杂语义信息的任务。与传统的自然语言控制机器人技术只能处理简单任务不同，这种方法使机器人能够执行更复杂和抽象的任务。



图 4-15 任务分解过程中失败的两个案例。

Fig. 4-15 Two cases of failure in the task decomposition process.

4.3 本章小结

本章节针对电力系统巡检任务中模型与环境交互困难的问题,本文提出了多模型交互与环境感知方法,通过将大型语言模型与视觉语言模型结合,能够对真实环境信息进行解析与理解,同时为了实现更精确的环境感知,我们设计了一种用于开放环境感知的热值图导航算法,通过构建一个可计算的二维空间,将摄像头采集到的视觉信息映射到二维可计算空间,能够反映出物体在真实世界中的相对位置信息。并且通过贪心算法的思想计算出最优路径,在热值图内生成密集的点状轨迹来指导机器人运动,这种映射关系允许机器人根据环境变化实时调整运动轨迹。在 4.1.4 章节的无人集群复杂任务执行实验中我们分别设计了基线实验和消融实验,验证了本文所提出的多模型交互与环境感知方法以及开放环境感知的热值图导航算法的有效性。

同时,针对大模型生成结果出现的局部最优问题,本文提出了一种自回归循环语义对齐方法,旨在将语义对齐过程中出现语义偏差的任务文本进行修正,这个过程通过将错误任务和提示词合并构建一个反馈信息,使大型语言模型在生成新一轮结果时在语义层面上偏向提示词所述的正确结果,能够有效的提高大型语言模型输出结果与机器人控制指令之间的语义相似性。为了进一步验证该方法的有效性,本文进行了基于自回归循环语义对齐方法的反馈实验,4.2.3 章节的实验中讨论了最佳反馈次数,并且验证了该方法能够有效提升任务执行的成功率。

5 结论与展望

5.1 本文的主要研究成果

随着社会的快速发展和用电需求的持续增长,电力系统成为电力能源供应的支柱,保证电力系统的稳定运行成为关键步骤。机器人技术的引用为传统的电力系统人工巡检带来了新的机遇,机器人技术在物流运输、工业自动化等多个领域已经展现出巨大的潜力。电力巡检任务通常是在开放场景中执行,因其复杂且多变的环境情况以及任务的多样性和不确定性等因素对机器人的任务理解能力、动态任务规划能力和自主执行能力提出了极高的要求。在本文中,我们对电力巡检任务中面向自然语言交互的异构机器人复杂任务规划这一目标凝练出了以下核心问题。并且针对这些问题进行了深入的研究。

本文的主要研究成果如下:

(1) 针对电力系统巡检任务中面临的自然语言难以理解问题,本文提出了基于自然语言处理的任务语义解析框架,该框架通过 LangChain 架构接入了具备本地知识的向量库,能够将本地知识通过向量的形式与大型语言模型进行交互,为了解决大型语言模型难以获得自然语言中特定知识的困难,我们构建了一个电力系统巡检场景的数据库,并且在 LangChain 架构中我们使用了检索增强生成(RAG)这一技术,通过动态检索外部知识增强了大型语言模型的本地知识获取能力。实现了电力系统巡检任务的准确理解。通过 2.1.4 章节的电力系统知识库 LLM 对比测试,验证了在专业领域任务理解中,向量知识库的引入使模型在准确性、可靠性等关键指标上产生显著提升,有效降低 LLM 的幻觉现象。

(2) 针对电力系统巡检任务中的复杂任务难以分解的问题,本文提出了多智能体动态任务分解框架,该框架展示了一个多步骤的复杂任务经过主代理分解为细粒度的子任务序列,然后动态的将这些子任务分配给多个子代理的过程。该框架能够将复杂的多步骤任务分解为复杂度低的更加易于执行的多个子任务。这个过程降低了任务的复杂度,增加了每个任务的可执行性,最终目的是使子任务在细粒度层面逼近原子任务,从而在行为一致性上进行对齐。针对本文提出的多智能体动态任务分解框架,通过 2.2.4 章节中的对比实验成功的验证了该架构的性能高于传统单 LLM 任务分解架构。

(3) 针对巡检机器人个性化功能开发难以自动化的问题,本文提出了一种基于多智能体自协作的框架。该框架利用多个大型语言模型协同工作,共同制定机器人自动开发策略并生成可执行代码。通过调用 LLM API 构建不同角色的智能体,该框架能够高效引导机器人完成二次开发任务,使不具备专业知识或编程经验的用户也能实现复杂机器人功能开发。在 3.1.3 章节的机器人功能自动开发实验结果表明,多智能体自协作框架在效率和准确性上明显优于传统多体团队合作方法和单一大规模语言模型处理复杂任务时的表现。

(4) 针对巡检任务中机器人动作行为与任务策略之间的对齐问题,本章提出了一种

基于余弦相似度的原子任务对齐方法。通过量化任务规划与机器人运动控制指令之间的语义相似度,我们实现了任务与指令的精准匹配。本章提出了一种基于语义度量的细粒度评价体系,通过分析任务策略与原子任务之间的语义相似性来评估对齐过程的有效性。该方法确保任务分解与指令执行的无缝衔接,同时减少任务到指令转换过程中可能出现的语义偏差和传播错误。并且本文在 3.2.4 章节中进行了语义度量基线实验和消融实验,证明了该方法的有效性。

(5) 针对电力系统巡检任务中模型与环境交互困难的问题,本文提出了多模型交互与环境感知方法,将大型语言模型与视觉语言模型结合,能够对真实环境信息进行解析与理解,同时为了实现更精确的环境感知,我们设计了一种用于开放环境感知的热值图导航算法,通过构建一个可计算的二维空间,将摄像头采集到的视觉信息映射到二维可计算空间,能够反映出物体在真实世界中的相对位置信息。并且通过贪心算法的思想计算出最优路径,在热值图内生成密集的点状轨迹来指导机器人运动,这种映射关系允许机器人根据环境变化实时调整运动轨迹。在 4.1.4 章节的无人集群复杂任务执行实验中我们分别设计了基线实验和消融实验,验证了本文所提出的多模型交互与环境感知方法以及开放环境感知的热值图导航算法的有效性。

(6) 针对大模型生成结果出现的局部最优问题,本文提出了一种自回归循环语义对齐方法,旨在将语义对齐过程中出现语义偏差的任务文本进行修正,这个过程通过将错误任务和提示词合并构建一个反馈信息,使大型语言模型在生成新一轮结果时在语义层面上偏向提示词所述的正确结果,能够有效的提高大型语言模型输出结果与机器人控制指令之间的语义相似性。为了进一步验证该方法的有效性,本文进行了基于自回归循环语义对齐方法的反馈实验,4.2.3 章节的实验中讨论了最佳反馈次数,并且验证了该方法能够有效提升任务执行的成功率。

5.2 下一步的研究方向

在现有研究的基础上,可以从以下角度来开展本文研究的后续工作。

(1) 本文考虑了文本这种单一模态的数据的任务理解与任务分解,尽管目前的任务理解方法利用 LLM 结合领域知识库取得了初步的效果。但在复杂的电力系统巡检场景中,任务描述通常不止是自然语言形式的,还涉及到图像、视频以及多种传感器数据的信息形式。因此,下一步的研究方向可以进一步的探索多模态学习方法,集成视觉听觉以及多传感信息,以提升任务理解的精确度与全面性。融合自然语言处理与多模态大模型技术可提升任务理解与规划的智能化水平,使其在动态电力巡检环境中有效处理多样化的复杂任务。

(2) 本文讨论了机器人自动开发的自协作框架,当前机器人自动开发的自协作框架仍受限于大语言模型的智能水平和机器人硬件适配能力。面对日益复杂的巡检任务需求,未来研究应重点优化自协作框架下的机器人功能自动生成技术,特别是针对个性化需求的快速实现与调整。虽然 LLM 代码生成技术持续进步,但需结合硬件兼容性和环境约束来提升代码的适应性与执行可靠性。此外,还需优化多代理协作机制和自适应开发流程,以

更好地满足多样化巡检任务对机器人功能的要求。

(3) 本文考虑了机器人集群的任务规划与运动控制,但本质上是对多个独立机器人进行任务分配与规划,将每个机器人视为独立个体进行任务分配与规划,其他机器人作为动态环境变量处理。未来研究需构建更完善的多机器人集群控制框架,重点解决大规模动态不确定环境下的协同调度与任务优化问题。采用自适应算法和任务优先级调整策略可提升集群自主决策能力,有效处理集群内部冲突进而优化系统整体性能。

参考文献

- [1] 李明跃,黄镇,赵学文,等.一种提高变电站巡检机器人响应速度的智能联动系统[J].电工技术,2025,(03):39-40+45.
- [2] 李明.基于 PID 智能控制的电力巡检机器人路径跟踪系统研究[J].造纸装备及材料,2025,54(04):16-18.
- [3] Li Z, Zhang Y, Wu H, et al. Design and application of a UAV autonomous inspection system for high-voltage power transmission lines[J]. Remote Sensing, 2023, 15(3): 865.
- [4] Wang S, Zhao Z, Liu H. Power Corridor Safety Hazard Detection Based on Airborne 3D Laser Scanning Technology[J]. ISPRS International Journal of Geo-Information, 2024, 13(11): 392.
- [5] 林永,陈雪森,罗珩,等.电力巡检机器人导航技术发展现状[J].电工技术,2024,(S2):105-108.
- [6] 李耀贵.基于元强化学习的电力巡检机器人自主越障控制研究[J].科学技术创新,2024,(24):29-32.
- [7] Liu Y, Ge X, Jia H, et al. Obstacle Avoidance Path Planning for Power Inspection Robots based on Deep Learning Algorithms[J]. Scalable Computing: Practice and Experience, 2024, 25(5): 3288-3295.
- [8] Solmaz S, Innerwinkler P, Wójcik M, et al. Robust robotic search and rescue in harsh environments: An example and open challenges[C]//2024 IEEE International Symposium on Robotic and Sensors Environments (ROSE). IEEE, 2024: 1-8.
- [9] 王宏德,李晓盟.基于蚁群算法的电力巡检机器人移动轨迹自动化跟踪控制系统[J].自动化与仪表,2024,39(11):60-63+68.
- [10] Huang W, Abbeel P, Pathak D, et al. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents[C]//International conference on machine learning. PMLR, 2022: 9118-9147.
- [11] Tellex S, Gopalan N, Kress-Gazit H, et al. Robots that use language[J]. Annual Review of Control, Robotics, and Autonomous Systems, 2020, 3(1): 25-55.
- [12] Micheli V, Fleuret F. Language models are few-shot butlers[J]. arXiv preprint arXiv:2104.07972, 2021.

- [13] Thomason J, Zhang S, Mooney R J, et al. Learning to Interpret Natural Language Commands through Human-Robot Dialog[C]//IJCAI. 2015, 15: 1923-1929.
- [14] Shwartz V, West P, Bras R L, et al. Unsupervised commonsense question answering with self-talk[J]. arXiv preprint arXiv:2004.05483, 2020.
- [15] 完颜丹丹,孙士保,赵威.结合改进 A*和改进 DWA 的电力巡检机器人路径规划方法[J/OL].机械设计与制造,1-7[2025-04-26].
- [16] Brown T, Mann B, Ryder N, et al. Language models are few-shot learners[J]. Advances in neural information processing systems, 2020, 33: 1877-1901.
- [17] Jiang Y, Gu S S, Murphy K P, et al. Language as an abstraction for hierarchical deep reinforcement learning[J]. Advances in Neural Information Processing Systems, 2019, 32.
- [18] Li S, Puig X, Paxton C, et al. Pre-trained language models for interactive decision-making[J]. Advances in Neural Information Processing Systems, 2022, 35: 31199-31212.
- [19] Ahn M, Brohan A, Brown N, et al. Do as i can, not as i say: Grounding language in robotic affordances[J]. arXiv preprint arXiv:2204.01691, 2022.
- [20] Gramopadhye M, Szafir D. Generating executable action plans with environmentally-aware language models[C]//2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2023: 3568-3575.
- [21] Raman S S, Cohen V, Idrees I, et al. Cape: Corrective actions from precondition errors using large language models[C]//2024 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2024: 14070-14077.
- [22] 宁雪峰.变电站巡检机器人多场景巡视点检路线优化研究[J].自动化仪表,2025,46(03):106-110.
- [23] Kamath A, Singh M, LeCun Y, et al. Mdetr-modulated detection for end-to-end multi-modal understanding[C]//Proceedings of the IEEE/CVF international conference on computer vision. 2021: 1780-1790.
- [24] Minderer M, Gritsenko A, Stone A, et al. Simple open-vocabulary object detection[C]//European conference on computer vision. Cham: Springer Nature Switzerland, 2022: 728-755.
- [25] 贾子琦,王健宗,张旭龙,等.基于大模型的具身智能任务规划研究:从单智能体到多智能体[J].大数据,2025,11(02):73-90.

- [26] Zellers R, Lu X, Hessel J, et al. Merlot: Multimodal neural script knowledge models[J]. Advances in neural information processing systems, 2021, 34: 23634-23651.
- [27] Shah D, Osiński B, Levine S. Lm-nav: Robotic navigation with large pre-trained models of language, vision, and action[C]//Conference on robot learning. PMLR, 2023: 492-504.
- [28] Shen S, Zhu X, Dong Y, et al. Incorporating domain knowledge through task augmentation for front-end JavaScript code generation[C]//Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2022: 1533-1543.
- [29] 程淼海,郭涛,项明明,等.基于改进蚁群算法的电力信息网络设备智能巡检机器人路径规划方法[J].自动化与仪表,2023,38(03):40-43+58.
- [30] Dong Y, Jiang X, Jin Z, et al. Self-collaboration code generation via chatgpt[J]. ACM Transactions on Software Engineering and Methodology, 2024, 33(7): 1-38.
- [31] Austin J, Odena A, Nye M, et al. Program synthesis with large language models[J]. arXiv preprint arXiv:2108.07732, 2021.
- [32] Singh I, Blukis V, Mousavian A, et al. Progprompt: Generating situated robot task plans using large language models[C]//2023 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2023: 11523-11530.
- [33] Liang J, Huang W, Xia F, et al. Code as policies: Language model programs for embodied control[C]//2023 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2023: 9493-9500.
- [34] Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models[J]. Advances in neural information processing systems, 2022, 35: 24824-24837.
- [35] Kojima T, Gu S S, Reid M, et al. Large language models are zero-shot reasoners[J]. Advances in neural information processing systems, 2022, 35: 22199-22213.
- [36] Ouyang L, Wu J, Jiang X, et al. Training language models to follow instructions with human feedback[J]. Advances in neural information processing systems, 2022, 35: 27730-27744.
- [37] Chung H W, Hou L, Longpre S, et al. Scaling instruction-finetuned language models[J]. Journal of Machine Learning Research, 2024, 25(70): 1-53.
- [38] Austin J, Odena A, Nye M, et al. Program synthesis with large language models[J]. arXiv preprint arXiv:2108.07732, 2021.

- [39] Jain N, Vaidyanath S, Iyer A, et al. Jigsaw: Large language models meet program synthesis[C]//Proceedings of the 44th International Conference on Software Engineering. 2022: 1219-1231.
- [40] Nijkamp E, Pang B, Hayashi H, et al. Codegen: An open large language model for code with multi-turn program synthesis[J]. arXiv preprint arXiv:2203.13474, 2022.
- [41] Chen X, Lin M, Schärli N, et al. Teaching large language models to self-debug[J]. arXiv preprint arXiv:2304.05128, 2023.
- [42] Shin T, Razeghi Y, Logan IV R L, et al. Autoprompt: Eliciting knowledge from language models with automatically generated prompts[J]. arXiv preprint arXiv:2010.15980, 2020.
- [43] Reynolds L, McDonell K. Prompt programming for large language models: Beyond the few-shot paradigm[C]//Extended abstracts of the 2021 CHI conference on human factors in computing systems. 2021: 1-7.
- [44] Jiang Z, Xu F F, Araki J, et al. How can we know what language models know?[J]. Transactions of the Association for Computational Linguistics, 2020, 8: 423-438.
- [45] Cui Y, Karamcheti S, Palleti R, et al. No, to the right: Online language corrections for robotic manipulation via shared autonomy[C]//Proceedings of the 2023 ACM/IEEE International Conference on Human-Robot Interaction. 2023: 93-101.
- [46] Ma Y J, Kumar V, Zhang A, et al. Liv: Language-image representations and rewards for robotic control[C]//International Conference on Machine Learning. PMLR, 2023: 23301-23320.
- [47] 杨子迪,李建胜,王安成,等.多机器人协同视觉 SLAM 技术研究综述[J].测绘科学技术学报,2025,41(01):57-67.
- [48] 李玉峰,宗国庆,张东豪.基于具身大模型的多场景智能巡检机器人系统框架与多机器人协同应用集成研究[J/OL].智能计算机与应用,1-7[2025-04-26].
- [49] Lin K, Agia C, Migimatsu T, et al. Text2motion: From natural language instructions to feasible plans[J]. Autonomous Robots, 2023, 47(8): 1345-1365.
- [50] 鄢龙武,郑王里,林云汉.基于场景语义感知与大语言模型推理的行为树生成[J].计算机系统应用,2025,34(01):37-46.
- [51] Hu H, Sadigh D. Language instructed reinforcement learning for human-ai coordination[C]//International Conference on Machine Learning. PMLR, 2023: 13584-13598.

- [52] Zeng A, Attarian M, Ichter B, et al. Socratic models: Composing zero-shot multimodal reasoning with language[J]. arXiv preprint arXiv:2204.00598, 2022.
- [53] Liang J, Huang W, Xia F, et al. Code as policies: Language model programs for embodied control[C]//2023 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2023: 9493-9500.
- [54] Huang W, Xia F, Shah D, et al. Grounded decoding: Guiding text generation with grounded models for embodied agents[J]. Advances in Neural Information Processing Systems, 2023, 36: 59636-59661.
- [55] Wu J, Antonova R, Kan A, et al. Tidybot: Personalized robot assistance with large language models[J]. Autonomous Robots, 2023, 47(8): 1087-1102.
- [56] 杜明山,张海清,李代伟,等.基于多任务学习的短期风能发电功率预测研究[J].软件导刊,2025,24(04):32-41.
- [57] Driess D, Oguz O, Ha J S, et al. Deep visual heuristics: Learning feasibility of mixed-integer programs for manipulation planning[C]//2020 IEEE international conference on robotics and automation (ICRA). IEEE, 2020: 9563-9569.
- [58] 昌千琳,罗永捷,王强钢,等.基于 I-V 曲线全局特征提取的光伏组串 Swin-Transformer 故障诊断方法[J/OL].电工技术学报,1-13[2025-04-26].
- [59] Damen D, Doughty H, Farinella G M, et al. Scaling egocentric vision: The epic-kitchens dataset[C]//Proceedings of the European conference on computer vision (ECCV). 2018: 720-736.
- [60] Grauman K, Westbury A, Byrne E, et al. Ego4d: Around the world in 3,000 hours of egocentric video[C]//Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2022: 18995-19012.
- [61] Wang C, Xu D, Fei-Fei L. Generalizable task planning through representation pretraining[J]. IEEE Robotics and Automation Letters, 2022, 7(3): 8299-8306.
- [62] 王宁,罗中婧,吕跃祖,等.基于深度强化学习的仓库机器人寻路问题研究[J/OL].控制工程,1-7[2025-04-26].
- [63] De Groot O, Ferranti L, Gavrila D M, et al. Topology-driven parallel trajectory optimization in dynamic environments[J]. IEEE Transactions on Robotics, 2024.